Universidade Federal de Sergipe

Biblioteca de Algoritmos

Greedy
Grupo de Estudos em Programação Competitiva

August 22, 2026

Contents

1 Binary Search and Ternary Search

1.1 BinarySearch (3325ba)

```
#include <bits/stdc++.h>
using namespace std;
int n, x;
bool possible(int m) { return true; }; // O(M)
// Retorna o primeiro elemento que valida nossa propriedade
// lower_bound -> primeiro valor >= x (Primeiro Verdadeiro (MINIMIZAR))
// O nosso f(x) precisa ser: [F, F, F, F, T, T, T, T], ele para quando encontrar
     o primeiro T.
int bs(int x) { // O(M*log(n))
    int ans = -1, l = 0, r = 1e18; // r = MAX
    while (l <= r) {
        int m = l + (r-l)/2; // Piso
        if (possible(m)) {
            ans = m;
            r = m - 1;
        } else {
            l = m + 1;
        }
    }
    return ans;
}
// Retorna o ultimo elemento que valida nossa propriedade
// O nosso f(x) precisa ser: [T, T, T, T, F, F, F, F], ele para quando encontrar
     o ultimo T.
int bs_last(int x) {
    int ans = -1, l = 0, r = 1e18; // r = MAX
    while (l <= r) {
        int m = l + (r-l)/2; // Teto (Ultimo Verdadeiro (MAXIMIZAR))
        if (possible(m)) {
            ans = m;
            l = m + 1;
        } else {
            r = m - 1;
        }
    }
    return ans;
}
```

1.2 BinarySearchDouble (758367)

```
#include <bits/stdc++.h>
using namespace std;
const double EPS = 1e-6; // Precisao -> defino as casas no setprecision
const int N_ITERATIONS = 100;
bool is_possible(long double m) {return true;}
long double bs(int x) {
    long double l = 0, r = 1e7;
    // for (int i = 0; i < N_ITERATIONS; i++) { -> Mais seguro
    while ((r - l) > EPS) {
        // Abaixo voce pode ajustar para o problema
        long double m = l + (r-l)/2;
        if (is_possible(m)) l = m;
        else r = m;
    }
    cout <<  fixed << setprecision(6) << l << endl;
}
```

1.3 BinarySearchInRealValues (edc7d7)

```
// No problema abaixo, tinhamos que encontrar um range de 'o' que tivesse maior
     razao
// quantidade de os/tamanho do range, ou seja, win_rage = wins/(wins+losses)
// Binary Search in real values
int main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n,k; cin >> n >> k;
  string s; cin >> s;

  vi pref(n+1);
  for(int i=0;i<n;i++){
    pref[i+1]=pref[i]+(s[i]=='o');
  }

  ld l=0,r=n;
  for(int i=0;i<60;i++){
    ld mid = (l+r)/2;

    vector<ld> psum(n+1);
    for(int i=0;i<n;i++){
        if(s[i]=='o')psum[i+1]=psum[i]+mid;
        else psum[i+1]=psum[i]-1;
    }
    ld mn = LINF;
    int j=0;
    bool ok=0;
    for(int i=0;i<=n;i++){
        for(;j<i and pref[i]-pref[j]>=k;j++){
            mn=min(mn,psum[j]);
        }
        // mid * os - xs >= 0?
        if(psum[i]>=mn)ok=true;
    }
    if(ok)r=mid;
    else l=mid;
  }
  // win_rate = wins/(wins+losses) = 1/(1+losses/wins) = 1/(1+r)
  cout<<1.0/(r+1)<<endl;

  return 0;
}
```

1.4 LowerUpperBound (4d5e64)

```
#include <bits/stdc++.h>
using namespace std;
signed main() {
    int x; vector<int> v;
    // lower_bound: retorna um it para o PRIMEIRO elemento >= x
    auto it = lower_bound(v.begin(), v.end(), x);
    cout << "pos: " << it - v.begin() << endl;
    cout << "valor: " << *it << endl;
    // upper_bound: retorna um it para o PRIMEIRO elemento > x
    auto it = upper_bound(v.begin(), v.end(), x);
    // Quantidade de Elementos Validos = UPPER - LOWER
}
```

2 Classics - Advanced Graph Problems

2.1 strongly connected edges

```
#include <bits/stdc++.h>
using namespace std;
#define ff first
#define ss second
#define pb push_back
#define int long long
#define endl '\n' //<< flush
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cerr << #x << " = " << x << endl
#define uniq(v) sort(all(v)); v.erase(unique(all(v)), v.end())
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
typedef long long ll;
typedef pair<int, int> pi;
```

```cpp
// const int MOD = 1e9 + 7; // 998244353;
// const int INF = 0x3f3f3f3f;
// const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const int MAXN = 2e5+5;

vector<int> adj[MAXN];
vector<pi> span_edges, back_edges;
int n, m, bridges, lvl[MAXN], dp[MAXN], parent[MAXN];

void dfs(int v) {
    dp[v] = 0;
    for (auto u : adj[v]) {
        if (u == parent[v]) continue;
        if (lvl[u] == 0) {
            span_edges.pb({v, u});
            parent[u] = v;
            lvl[u] = lvl[v] + 1;
            dfs(u);
            dp[v] += dp[u];
        } else if (lvl[u] > lvl[v]) {
            back_edges.pb({u, v});
            dp[v]--;
        } else if (lvl[u] < lvl[v]) {
            dp[v]++;
        }
    }

    if (parent[v] != -1 and dp[v] == 0) {
        bridges++;
    }
}

void solve() {
    memset(parent, -1, sizeof(parent));
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int a, b; cin >> a >> b; a--, b--;
        adj[a].pb(b);
        adj[b].pb(a);
    }
    lvl[0] = 1;
    dfs(0);
    bool ok = true;
    for (int v = 0; v < n; v++) {
        if (lvl[v] == 0) {
            ok = false;
            break;
        }
    }
    if (!ok or (bridges > 0)) {
        cout << "IMPOSSIBLE" << endl;
        return;
    }
    // Todas as arestas da DFS Tree sao direcionadas para baixo (pai -> filho)
    for (auto [u, v] : span_edges) {
        cout << u+1 << " " << v+1 << endl;
    }
    // Todas as back edges sao direcionadas para cima (filho -> pai)
    for (auto [u, v] : back_edges) {
        cout << u+1 << " " << v+1 << endl;
    }
}

signed main() {
    ios_base::sync_with_stdio(0);cin.tie(0);
    int tc = 1;
    //cin >> tc;
    while(tc--) solve();
}
```

3 Classics - Advanced Techniques

3.1 dynamic connectivity

```cpp
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define pb push_back

typedef pair<int, int> pii;
typedef vector<int> vi;

const int N = 3e5 + 10;

vector<pii> seg[4 * N];
map<pii, int> entradas;
int ans[N];

struct persistent_dsu {
  struct state {
    int u, v, rnku, rnkv;
    state() {
      u = -1;
      v = -1;
      rnkv = -1;
      rnku = -1;
    }
    state(int _u, int _rnku, int _v, int _rnkv) {
      u = _u;
      rnku = _rnku;
      v = _v;
      rnkv = _rnkv;
    }
  };

  stack<state> st;
  int par[N], depth[N];
  int comp;

  persistent_dsu() {
    comp = 0;
    memset(par, -1, sizeof(par));
    memset(depth, 0, sizeof(depth));
  }

  int root(int x) {
    if (x == par[x])
      return x;
    return root(par[x]);
  }

  void init(int n) {
    comp = n;
    for (int i = 0; i <= n; i++) {
      par[i] = i;
      depth[i] = 1;
    }
  }

  bool connected(int x, int y) { return root(x) == root(y); }

  void unite(int x, int y) {
    int rx = root(x), ry = root(y);
    if (rx == ry) {
      st.push(state());
      return;
    }

    st.push(state(rx, depth[rx], ry, depth[ry]));

    if (depth[rx] < depth[ry]) {
      par[rx] = ry;
    } else if (depth[ry] < depth[rx]) {
      par[ry] = rx;
    } else {
      par[rx] = ry;
      depth[ry]++;
    }
    comp--;
  }
```

```
  void backtrack(int c) {
    while (!st.empty() && c) {
      if (st.top().u == -1) {
        st.pop();
        c--;
        continue;
      }
      par[st.top().u] = st.top().u;
      par[st.top().v] = st.top().v;
      depth[st.top().u] = st.top().rnku;
      depth[st.top().v] = st.top().rnkv;
      st.pop();
      c--;
      comp++;
    }
  }
};

persistent_dsu d;

void upd(int p, int l, int r, int ql, int qr, pii val) {
  if (l > qr or r < ql)
    return;
  if (ql <= l and r <= qr) {
    seg[p].pb(val);
    return;
  }
  int m = (l + r) / 2;
  upd(2 * p, l, m, ql, qr, val);
  upd(2 * p + 1, m + 1, r, ql, qr, val);
}

void query(int p, int l, int r) {
  if (l > r)
    return;
  int prev = d.st.size();

  for (auto &edge : seg[p])
    d.unite(edge.f, edge.sec);

  if (l == r) {
    ans[l] = d.comp;
    d.backtrack(d.st.size() - prev);
    return;
  }

  int m = (l + r) / 2;
  query(2 * p, l, m);
  query(2 * p + 1, m + 1, r);

  d.backtrack(d.st.size() - prev);
}

int main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);

  int n, m, k;
  cin >> n >> m >> k;

  d.init(n);
  for (int i = 1; i <= m; i++) {
    int u, v; cin >>u >>v;
    if(u>v) swap(u,v);
    entradas[{u,v}]=0;
  }
  vi queries;
  queries.pb(0);
  for (int i = 1; i <= k; i++) {
    int t;
    cin >> t;
    queries.pb(i);
    if (t == 1) {
      int u, v;
      cin >> u >> v;
      if (u > v)
        swap(u, v);
      entradas[{u, v}] = i;
    } else {
      int u, v;
      cin >> u >> v;
      if (u > v)
        swap(u, v);
      pii p = {u, v};
      upd(1, 0, k, entradas[p], i - 1, p);
      entradas.erase(p);
    }
  }

  for (auto &p : entradas) {
    upd(1, 0, k, p.second, k, p.first);
  }

  query(1, 0, k);

  for (auto i : queries) {
    cout << ans[i] << " ";
  }
  cout << endl;

  return 0;
}
```

3.2 reversals and sums

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back
#define int long long

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;
mt19937 rnd(chrono::steady_clock::now().time_since_epoch().count());

const int N = 2e5+10;

struct treap {
  //This is an implicit treap which investigates here on an array
  struct node {
    int val, sz, prior, lazy, sum, mx, mn, repl;
    bool repl_flag, rev;
    node *l, *r, *par;
    node() {
      lazy = 0;
      rev = 0;
      sum = 0;
      val = 0;
      sz = 0;
      mx = 0;
      mn = 0;
      repl = 0;
      repl_flag = 0;
      prior = 0;
      l = NULL;
      r = NULL;
      par = NULL;
```

```
  }
  node(int _val) {
    val = _val;
    sum = _val;
    mx = _val;
    mn = _val;
    repl = 0;
    repl_flag = 0;
    rev = 0;
    lazy = 0;
    sz = 1;
    prior = rnd();
    l = NULL;
    r = NULL;
    par = NULL;
  }
};
typedef node* pnode;
pnode root;
map<int, pnode> position;//positions of all the values
//clearing the treap
void clear() {
  root = NULL;
  position.clear();
}

treap() {
  clear();
}

int size(pnode t) {
  return t ? t->sz : 0;
}
void update_size(pnode &t) {
  if(t) t->sz = size(t->l) + size(t->r) + 1;
}

void update_parent(pnode &t) {
  if(!t) return;
  if(t->l) t->l->par = t;
  if(t->r) t->r->par = t;
}
//add operation
void lazy_sum_upd(pnode &t) {
  if( !t or !t->lazy )    return;
  t->sum += t->lazy * size(t);
  t->val += t->lazy;
  t->mx += t->lazy;
  t->mn += t->lazy;
  if( t->l )  t->l->lazy += t->lazy;
  if( t->r )  t->r->lazy += t->lazy;
  t->lazy = 0;
}
//replace update
void lazy_repl_upd(pnode &t) {
  if( !t or !t->repl_flag )   return;
  t->val = t->mx = t->mn = t->repl;
  t->sum = t->val * size(t);
  if( t->l ) {
    t->l->repl = t->repl;
    t->l->repl_flag = true;
  }
  if( t->r ) {
    t->r->repl = t->repl;
    t->r->repl_flag = true;
  }
  t->repl_flag = false;
  t->repl = 0;
}
//reverse update
void lazy_rev_upd(pnode &t) {
  if( !t or !t->rev ) return;
  t->rev = false;
  swap(t->l, t->r);
  if( t->l )  t->l->rev ^= true;
  if( t->r )  t->r->rev ^= true;
}
//reset the value of current node assuming it now
//represents a single element of the array
void reset(pnode &t) {
  if(!t) return;
  t->sum = t->val;
  t->mx = t->val;
  t->mn = t->val;
}
//combine node l and r to form t by updating corresponding queries
void combine(pnode &t, pnode l, pnode r) {
  if(!l) {
    t = r;
    return;
  }
  if(!r) {
    t = l;
    return;
  }
  //Beware!!!Here t can be equal to l or r anytime
  //i.e. t and (l or r) is representing same node
  //so operation is needed to be done carefully
  //e.g. if t and r are same then after t->sum=l->sum+r->sum operation,
  //r->sum will be the same as t->sum
  //so BE CAREFUL
  t->sum = l->sum + r->sum;
  t->mx = max(l->mx, r->mx);
  t->mn = min(l->mn, r->mn);
}
//perform all operations
void operation(pnode &t) {
  if( !t )    return;
  reset(t);
  lazy_rev_upd(t->l);
  lazy_rev_upd(t->r);
  lazy_repl_upd(t->l);
  lazy_repl_upd(t->r);
  lazy_sum_upd(t->l);
  lazy_sum_upd(t->r);
  combine(t, t->l, t);
  combine(t, t, t->r);
}
//split node t in l and r by key k
//so first k+1 elements(0,1,2,...k) of the array from node t
//will be split in left node and rest will be in right node
void split(pnode t, pnode &l, pnode &r, int k, int add = 0) {
  if(t == NULL) {
    l = NULL;
    r = NULL;
    return;
  }
  lazy_rev_upd(t);
  lazy_repl_upd(t);
  lazy_sum_upd(t);
  int idx = add + size(t->l);
  if(t->l) t->l->par = NULL;
  if(t->r) t->r->par = NULL;
  if(idx <= k)
    split(t->r, t->r, r, k, idx + 1), l = t;
  else
    split(t->l, l, t->l, k, add), r = t;

  update_parent(t);
  update_size(t);
  operation(t);
}
//merge node l with r in t
void merge(pnode &t, pnode l, pnode r) {
  lazy_rev_upd(l);
  lazy_rev_upd(r);
  lazy_repl_upd(l);
  lazy_repl_upd(r);
  lazy_sum_upd(l);
  lazy_sum_upd(r);
  if(!l) {
    t = r;
    return;
  }
  if(!r) {
    t = l;
    return;
  }
```

```
  if(l->prior > r->prior)
    merge(l->r, l->r, r), t = l;
  else
    merge(r->l, l, r->l), t = r;

  update_parent(t);
  update_size(t);
  operation(t);
}
//insert val in position a[pos]
//so all previous values from pos to last will be right shifted
void insert(int pos, int val) {
  if(root == NULL) {
    pnode to_add = new node(val);
    root = to_add;
    position[val] = root;
    return;
  }

  pnode l, r, mid;
  mid = new node(val);
  position[val] = mid;

  split(root, l, r, pos - 1);
  merge(l, l, mid);
  merge(root, l, r);
}
//erase from qL to qR indexes
//so all previous indexes from qR+1 to last will be left shifted qR-qL+1 times
void erase(int qL, int qR) {
  pnode l, r, mid;

  split(root, l, r, qL - 1);
  split(r, mid, r, qR - qL);
  merge(root, l, r);
}
//returns answer for corresponding types of query
int query(int qL, int qR) {
  pnode l, r, mid;

  split(root, l, r, qL - 1);
  split(r, mid, r, qR - qL);

  int answer = mid->sum;
  merge(r, mid, r);
  merge(root, l, r);

  return answer;
}
//add val in all the values from a[qL] to a[qR] positions
void update(int qL, int qR, int val) {
  pnode l, r, mid;

  split(root, l, r, qL - 1);
  split(r, mid, r, qR - qL);
  lazy_repl_upd(mid);
  mid->lazy += val;

  merge(r, mid, r);
  merge(root, l, r);
}
//reverse all the values from qL to qR
void reverse(int qL, int qR) {
  pnode l, r, mid;

  split(root, l, r, qL - 1);
  split(r, mid, r, qR - qL);

  mid->rev ^= 1;
  merge(r, mid, r);
  merge(root, l, r);
}
//replace all the values from a[qL] to a[qR] by v
void replace(int qL, int qR, int v) {
  pnode l, r, mid;

  split(root, l, r, qL - 1);
  split(r, mid, r, qR - qL);
  lazy_sum_upd(mid);
  mid->repl_flag = 1;
  mid->repl = v;
  merge(r, mid, r);
  merge(root, l, r);
}
//it will cyclic right shift the array k times
//so for k=1, a[qL]=a[qR] and all positions from ql+1 to qR will
//have values from previous a[qL] to a[qR-1]
//if you make left_shift=1 then it will to the opposite
void cyclic_shift(int qL, int qR, int k, bool left_shift = 0) {
  if(qL == qR) return;
  k %= (qR - qL + 1);

  pnode l, r, mid, fh, sh;
  split(root, l, r, qL - 1);
  split(r, mid, r, qR - qL);

  if(left_shift == 0) split(mid, fh, sh, (qR - qL + 1) - k - 1);
  else split(mid, fh, sh, k - 1);
  merge(mid, sh, fh);

  merge(r, mid, r);
  merge(root, l, r);
}
bool exist;
//returns index of node curr
int get_pos(pnode curr, pnode son = nullptr) {
  if(exist == 0) return 0;
  if(curr == NULL) {
    exist = 0;
    return 0;
  }
  if(!son) {
    if(curr == root) return size(curr->l);
    else return size(curr->l) + get_pos(curr->par, curr);
  }

  if(curr == root) {
    if(son == curr->l) return 0;
    else return size(curr->l) + 1;
  }

  if(curr->l == son) return get_pos(curr->par, curr);
  else return get_pos(curr->par, curr) + size(curr->l) + 1;
}
//returns index of the value
//if the value has multiple positions then it will
//return the last index where it was added last time
//returns -1 if it doesn't exist in the array
int get_pos(int value) {
  if(position.find(value) == position.end()) return -1;
  exist = 1;
  int x = get_pos(position[value]);
  if(exist == 0) return -1;
  else return x;
}
//returns value of index pos
int get_val(int pos) {
  return query(pos, pos);
}
//returns size of the treap
int size() {
  return size(root);
}
//inorder traversal to get indexes chronologically
void inorder(pnode cur) {
  if(cur == NULL) return;
  operation(cur);
  inorder(cur->l);
  cout << cur->val << ' ';
  inorder(cur->r);
}
//print current array values serially
void print_array() {
//      for(int i=0;i<size();i++) cout<<get_val(i)<<' ';
//      cout<<nl;
  inorder(root);
  cout << endl;
```

```
  }
  bool find(int val) {
    if(get_pos(val) == -1) return 0;
    else return 1;
  }
};

treap t;

signed main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);

  int n,q; cin >> n >> q;
  for(int i=0;i<n;i++){
    int x; cin >> x;
    t.insert(i,x);
  }

  while(q--){
    int c,l,r; cin >> c >> l >> r;
    l--,r--;
    if(c==1){
      t.reverse(l,r);
    }else{
      cout << t.query(l,r) << endl;
    }
  }

  return 0;
}
```

3.3 substring reversals

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;
mt19937 rnd(chrono::steady_clock::now().time_since_epoch().count());

const int N = 2e5+10;

struct treap {
  //This is an implicit treap which investigates here on an array
  struct node {
    int val, sz, prior, lazy, sum, mx, mn, repl;
    bool repl_flag, rev;
    node *l, *r, *par;
    node() {
      lazy = 0;
      rev = 0;
      sum = 0;
      val = 0;
      sz = 0;
      mx = 0;
      mn = 0;
      repl = 0;
      repl_flag = 0;
      prior = 0;
      l = NULL;
      r = NULL;
      par = NULL;
    }
    node(int _val) {
      val = _val;
      sum = _val;
      mx = _val;
      mn = _val;
      repl = 0;
      repl_flag = 0;
      rev = 0;
      lazy = 0;
      sz = 1;
      prior = rnd();
      l = NULL;
      r = NULL;
      par = NULL;
    }
  };
  typedef node* pnode;
  pnode root;
  map<int, pnode> position;//positions of all the values
  //clearing the treap
  void clear() {
    root = NULL;
    position.clear();
  }

  treap() {
    clear();
  }

  int size(pnode t) {
    return t ? t->sz : 0;
  }
  void update_size(pnode &t) {
    if(t) t->sz = size(t->l) + size(t->r) + 1;
  }

void lazy_rev_upd(pnode &t) {
    if( !t or !t->rev ) return;
    t->rev = false;
    swap(t->l, t->r);
    if( t->l )  t->l->rev ^= true;
    if( t->r )  t->r->rev ^= true;
  }

  //split node t in l and r by key k
  //so first k+1 elements(0,1,2,...k) of the array from node t
  //will be split in left node and rest will be in right node
  void split(pnode t, pnode &l, pnode &r, int k, int add = 0) {
    if(t == NULL) {
      l = NULL;
      r = NULL;
      return;
    }
    lazy_rev_upd(t);
    int idx = add + size(t->l);
    if(t->l) t->l->par = NULL;
    if(t->r) t->r->par = NULL;
    if(idx <= k)
      split(t->r, t->r, r, k, idx + 1), l = t;
    else
      split(t->l, l, t->l, k, add), r = t;

    update_size(t);
  }
  //merge node l with r in t
  void merge(pnode &t, pnode l, pnode r) {
    lazy_rev_upd(l);
    lazy_rev_upd(r);
    if(!l) {
      t = r;
      return;
```

```
    }
    if(!r) {
      t = l;
      return;
    }
    if(l->prior > r->prior)
      merge(l->r, l->r, r), t = l;
    else
      merge(r->l, l, r->l), t = r;
    update_size(t);
  }
  //insert val in position a[pos]
  //so all previous values from pos to last will be right shifted
  void insert(int pos, int val) {
    if(root == NULL) {
      pnode to_add = new node(val);
      root = to_add;
      position[val] = root;
      return;
    }

    pnode l, r, mid;
    mid = new node(val);
    position[val] = mid;

    split(root, l, r, pos - 1);
    merge(l, l, mid);
    merge(root, l, r);
  }
  //reverse all the values from qL to qR
  void reverse(int qL, int qR) {
    pnode l, r, mid;

    split(root, l, r, qL - 1);
    split(r, mid, r, qR - qL);

    mid->rev ^= 1;
    merge(r, mid, r);
    merge(root, l, r);
  }

  //inorder traversal to get indexes chronologically
  void inorder(pnode cur) {
    if(cur == NULL) return;
    lazy_rev_upd(cur);
    inorder(cur->l);
    char res = cur->val+'a';
    cout << res;
    inorder(cur->r);
  }
  //print current array values serially
  void print_array() {
    inorder(root);
    cout << endl;
  }
};

treap t;

signed main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);

  int n,q; cin >> n >> q;
  string s; cin >> s;
  for(int i=0;i<n;i++){
    char c = s[i];
    int x = c-'a';
    t.insert(i,x);
  }

  while(q--){
    int l,r; cin  >> l >> r;
    l--,r--;
    t.reverse(l,r);
  }

  t.print_array();

  return 0;
}
```

4 Classics - Dynamic Programming

4.1 array description

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back
#define int long long

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;

int dp[100005][105];

signed main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n, m;
  cin >> n >> m;

  vi a(n);
  for (int i = 0; i < n; i++) {
    cin >> a[i];
  }
  if (a[0] == 0) {
    fill(dp[0], dp[0] + 101, 1);
  } else {
    dp[0][a[0]] = 1;
  }
  for (int i = 1; i < n; i++) {
    if (a[i] != 0) {
      dp[i][a[i]] += dp[i - 1][a[i]];
      if (a[i] - 1 > 0)
        dp[i][a[i]] += dp[i - 1][a[i] - 1];
      if (a[i] + 1 <= m)
        dp[i][a[i]] += dp[i - 1][a[i] + 1];
      dp[i][a[i]] %= MOD;
    } else {
      for (int j = 1; j <= m; j++) {
        dp[i][j] += dp[i - 1][j];
        if (j - 1 > 0)
          dp[i][j] += dp[i - 1][j - 1];
        if (j+ 1 <= m)
          dp[i][j] += dp[i - 1][j + 1];
        dp[i][j]%=MOD;
      }
    }
  }
  int ans=0;
  for(int i=1;i<=m;i++){
    ans = (ans + dp[n-1][i])%MOD;
  }
```

```
  cout << ans << endl;

  return 0;
}
```

4.2 book shop

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define int long long

const int MAXN = 1e3 + 1;
const int MAXW = 1e5 + 1;

int n, x, dp[MAXW], preco[MAXN], v[MAXN];

signed main() {
        ios_base::sync_with_stdio(false); cin.tie(0);
    cin >> n >> x;

    for (int i = 1; i <= n; i++) {
        cin >> preco[i];
    }

    for (int i = 1; i <= n; i++) {
        cin >> v[i];
    }

    for (int i = 1; i <= n; i++) {
        int preco_atual = preco[i];
        int valor_atual = v[i];
        for (int j = x; j >= preco_atual; j--) {
            dp[j] = max(dp[j], dp[j - preco_atual] + valor_atual);
        }
    }

    cout << dp[x] << endl;
}
```

4.3 coin combinations i

```
#include <bits/stdc++.h>
using namespace std;

#define _ ios_base::sync_with_stdio(0);cin.tie(0);
#define endl '\n'
#define pb push_back
#define eb emplace_back
#define ff first
#define ss second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()

#define dbg(x) cout << #x << " = " << x << endl

typedef long long ll;
typedef pair<int, int> pi;

const ll MOD = 1000000007;
const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

#define MAX_N 1000000
#define MAX_X 1000000

ll dp[MAX_X + 1];
vector<ll> coins;

ll solve(ll sum) {
    for (int i = 1; i <= sum; i++) {
        for (auto c : coins) {
            if (i - c < 0) break;
            dp[i] = (dp[i] + (dp[i - c])) % MOD;
        }
    }

    return dp[sum];
}

signed main(){ _

    int n, x; cin >> n >> x;
    for (int i = 0; i < n; i++) {
        ll new_value; cin >> new_value;
        coins.push_back(new_value);
    }

    sort(all(coins));

    dp[0] = 1;

    cout << solve(x) << endl;
}
```

4.4 coin combinations ii

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back
#define int long long

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;

const int MX = 1e6 + 1;

signed main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n, x;
  cin >> n >> x;
  vi a(n);
  for (auto &x : a)
    cin >> x;

  vi dp(MX,0);
  dp[0]=1;
  for(int i=0;i<n;i++){
    for(int j = 0; j<=x;j++){
      if(j - a[i] >= 0){
        dp[j] += dp[j-a[i]];
        dp[j] %=MOD;
      }
    }
  }

  cout << dp[x]<<endl;
  return 0;
```

```
}
```

4.5 counting towers

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define ff first
#define ss second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define sz(x) (x).size()
#define pb push_back

typedef long long ll;
typedef double ld;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;
typedef vector<vl> vvl;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;

const int N = 1e6+5;

ll dp1[N], dp2[N];

void preprocess(){
    dp1[0] = dp2[0] = 1;
    for (int i = 1; i < N; i++) {
      dp2[i] = (dp2[i - 1] * 4) + dp1[i - 1];
      dp2[i] %= MOD;
      dp1[i] = (dp1[i - 1] * 2) + dp2[i - 1];
      dp1[i] %= MOD;
    }
}

int main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  preprocess();
  int t;
  cin >> t;
  while (t--) {
    int n;
    cin >> n;
    cout << (dp1[n - 1] + dp2[n - 1]) % MOD << endl;
  }

  return 0;
}
```

4.6 dice combinations

```
#include <bits/stdc++.h>
using namespace std;

#define _ ios_base::sync_with_stdio(0);cin.tie(0);
#define endl '\n'
#define pb push_back
#define eb emplace_back
#define ff first
#define ss second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()

#define dbg(x) cout << #x << " = " << x << endl

typedef long long ll;
typedef pair<int, int> pi;

const ll MOD = 1000000007;
const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

#define MAX_N 1000000

ll dp[MAX_N + 1];

ll solve(ll k) {
    for (int i = 1; i <= k; i++) {
        for (int j = 1; j <= 6 && i - j >= 0; j++) {
            dp[i] = (dp[i] + dp[i - j]) % MOD;
        }
    }

    return dp[k];
}

signed main(){ _
    ll n; cin >> n;

    dp[0] = 1;

    cout << solve(n) << endl;
}

/* Logica
Nosso objetivo e descobrir de quantas maneiras podemos chegar a um valor K
     somando x que pertencem a (1, 2, 3, 4, 5, 6).

solve(k) = solve(k - 1) + solve(k - 2) + solve(k - 3) + solve(k - 4) + solve(k -
      5) + solve(k - 6)
= sum{i = 1 ate 6, k - i >= 0}{solve(k - i)} --> Nossa recorrencia e essa aqui

a funcao solve(k) retorna o numero de vezes que podemos chegar em solve(k)
--------
Para k = 3
solve(3) = solve(2) + solve(1) + solve(0) # 0 -> Quantas maneiras ha de fazer a
     soma 0? Apenas uma, fazer nada.

Precisamos resolver solve(1) e solve(2):
- solve(1) = solve(0) = 1
- solve(2) = solve(1) + solve(0) = 1 + 1 = 2

Logo, solve(3) = 2 + 1 + 1 = 4!
*/
```

4.7 edit distance

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define ff first
#define ss second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define sz(x) (x).size()
#define pb push_back

typedef long long ll;
typedef double ld;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
```

```
typedef vector<pll> vpll;
typedef vector<vi> vvi;
typedef vector<vl> vvl;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3fll;
const double eps = 1e-9;
const ll MOD = 1e9 + 7;

const int N = 5e3+5;
int dp[N][N];

int main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  string n,m;
  cin >> n >> m;

  for(int i=0;i<=sz(n);i++) dp[i][0]=i;
  for(int j=0;j<=sz(m);j++) dp[0][j]=j;

  for(int i=1;i<=(int)n.size();i++){
      for(int j=1;j<=(int)m.size();j++){
          if(n[i-1] == m[j-1]){
              dp[i][j] = dp[i-1][j-1];
          }else{
              dp[i][j] = 1 + min({
                      dp[i][j-1],
                      dp[i-1][j],
                      dp[i-1][j-1]});
          }
      }
  }

  cout << dp[sz(n)][sz(m)] << endl;

  return 0;
}
```

4.8 grid paths i

```
#include <bits/stdc++.h>
using namespace std;

#define _ ios_base::sync_with_stdio(0);cin.tie(0);
#define endl '\n'
#define pb push_back
#define eb emplace_back
#define ff first
#define ss second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()

#define dbg(x) cout << #x << " = " << x << endl

typedef long long ll;
typedef pair<int, int> pi;

const ll MOD = 1e9 + 7;

signed main(){ _

    int n; cin >> n;
    char grid[n][n];
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cin >> grid[i][j];
        }
    }

    vector<vector<int>> dp(n, vector<int>(n, 0));
    dp[0][0] = 1;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (grid[i][j] == '.') {
                if (i > 0) dp[i][j] = (dp[i][j] + dp[i - 1][j]) % MOD;
                if (j > 0) dp[i][j] = (dp[i][j] + dp[i][j - 1]) % MOD;
            } else {
                dp[i][j] = 0;
            }
        }
    }

    cout << dp[n - 1][n - 1] << endl;
}
```

4.9 increasing subsequence ii

```
#include <bits/stdc++.h>
using namespace std;

#define _ ios_base::sync_with_stdio(0);cin.tie(0);
#define endl '\n'
#define all(x) (x).begin(), (x).end()

#define int long long

typedef long long ll;

const ll MOD = 1e9 + 7;

// Coordinate Compreesion
void coordinate_compression(vector<int> &a) {
    set<int> s; // Conjunto para armazenar todos os numeros unicos
    for (auto x : a) s.insert(x);

    int index = 0;
    map<int, int> mp; // Map para armazenar os novos elementos

    set<int>::iterator itr;
    for (itr = s.begin(); itr != s.end(); itr++) {
        index++;
        mp[*itr] = index;
    }

    // Alterando o valor de a
    for (int i = 0; i < a.size(); i++) {
        a[i] = mp[a[i]];
    }
}

// BIT ou SegTree
struct fenw {
    int n;
    vector<int> bit;

    fenw() {}
    fenw(int size) {
        n = size;
        bit.assign(size + 1, 0);
    }
    // query do prefixo a[0] + a[1] + ... + a[r]
    int qry(int r) {
        int ans = 0;
        for (int i = r + 1; i > 0; i -= i & -i) // i & -i retorna os bits menos
             signativos de i
            ans = (ans + bit[i]) % MOD;
        return ans;
    }

    // atualiza o valor a[r] = x
    void upd(int r, int x) {
        for (int i = r + 1; i <= n; i += i & -i)
            bit[i] = (bit[i] + x) % MOD;
    }
};

int count_lis(vector<int> &a) {
    coordinate_compression(a);
```

```
    int n = a.size();

    fenw tree(n + 1);

    ll ans = 0;
    for (int x : a) {
        ll subseq = (1 + tree.qry(x - 1)) % MOD;
        tree.upd(x, subseq);
        ans = (ans + subseq) % MOD;
    }

    return ans;
}

signed main(){ _

    int n; cin >> n;
    vector<int> a(n);
    for (auto &x : a) cin >> x;

    cout << count_lis(a) << endl;
}
```

4.10 increasing subsequence

```
#include <bits/stdc++.h>
using namespace std;

#define _ ios_base::sync_with_stdio(0);cin.tie(0);
#define endl '\n'
#define pb push_back
#define eb emplace_back
#define ff first
#define ss second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()

#define dbg(x) cout << #x << " = " << x << endl

typedef long long ll;
typedef pair<int, int> pi;

const ll MOD = 1000000007;
const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

int lis(const vector<int> &a) {
    int n = a.size();
    const int INF = 1e9;
    vector<int> dp(n + 1, INF);

    dp[0] = -INF;

    for (int i = 0; i < n; i++) {
        int l = upper_bound(all(dp), a[i]) - dp.begin();
        if (dp[l - 1] < a[i] && a[i] < dp[l]) {
            dp[l] = a[i];
        }
    }

    int ans = 0;
    for (int l = 0; l <= n; l++) {
        if (dp[l] < INF) ans = l;
    }
    return ans;
}

signed main(){ _

    int n; cin >> n;
    vector<int> a(n);
    for (auto &x : a) cin >> x;

    cout << lis(a) << endl;
}
```

4.11 longest common subsequence

```
#include <bits/stdc++.h>

using namespace std;

#define _ ios_base::sync_with_stdio(0);cin.tie(0);
#define endl '\n'
#define f first
#define s second

typedef long long ll;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

const int maxN=1001;
ll A[maxN]={0};
ll B[maxN]={0};
ll dp[maxN][maxN]={0};

int main(){_
  int n,m; cin >> n >> m;

  for(int i=1;i<=n;i++) cin >> A[i];
  for(int i=1;i<=m;i++) cin >> B[i];

  vector<ll> ans;

  for(int i=1;i<=m;i++){
    for(int j=1;j<=n;j++){
      if(A[j] == B[i]){
        dp[i][j] = dp[i-1][j-1] + 1;
      }else{
        dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
      }
    }
  }

  cout << dp[m][n] << endl;


  int i=m,j=n;
  while(i>0 && j>0){
    if(A[j] == B[i]){
      ans.push_back(A[j]);
      i--, j--;
    }else if(dp[i-1][j] > dp[i][j-1]) i--;
    else j--;
  }


  reverse(ans.begin(), ans.end());
  for(auto x : ans) cout << x << " ";
  cout << endl;

  return 0;
}
```

4.12 minimizing coins

```
#include <bits/stdc++.h>
using namespace std;

#define _ ios_base::sync_with_stdio(0);cin.tie(0);
#define endl '\n'
#define pb push_back
#define eb emplace_back
#define ff first
#define ss second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
```

```
#define dbg(x) cout << #x << " = " << x << endl

typedef long long ll;
typedef pair<int, int> pi;

const ll MOD = 1000000007;

#define MAX_N 1000000

ll dp[MAX_N + 1];
vector<ll> coins;

signed main(){ _

        ll n, x; cin >> n >> x;
        for (int i = 0; i < n; i++) {
                ll c; cin >> c;
                coins.pb(c);
        }

        sort(all(coins));

        dp[0] = 0;
        for (int i = 1; i <= x; i++) {
                dp[i] = INT_MAX;
                for (auto c : coins) {
                        if (i - c >= 0) {
                                dp[i] = min(dp[i], dp[i - c] + 1);
                        }
                }
        }

        cout << ((dp[x] == INT_MAX) ? -1 : dp[x]) << endl;
}
```

4.13 money sums

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back
#define int long long

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;

const int MX = 1e5+1;

int dp[MX];
signed main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n; cin >> n;
  vi a(n);
  for(auto &x:a) cin >> x;

  dp[0]=1;
  for(int i=0;i<n;i++){
    for(int j=MX-1; j>=0;j--){
      if(dp[j] > 0){
        dp[a[i]+j] = 1;
      }
    }
  }

  vi ans;
  for(int i=1;i<MX;i++){
    if(dp[i] > 0) ans.pb(i);
  }
  cout << ans.size() << endl;
  for(auto i : ans) cout << i << ' ';
  cout << endl;


  return 0;
}
```

4.14 removal game

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back
#define int long long

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;

const int N = 5e3+1;

int v[N];
vvi dp1(N, vi(N,LINF));
vvi dp2(N, vi(N,LINF));

int solve(int l, int r, bool t){
  if(t){
    if(l==r) return v[l];
    if(dp1[l][r] != LINF) return dp1[l][r];
    return dp1[l][r] = max(v[l] + solve(l+1,r,!t),
        v[r]+ solve(l,r-1,!t));
  }else{
    if(l==r) return 0;
    if(dp2[l][r] != LINF) return dp2[l][r];
    return dp2[l][r] = min(solve(l+1,r,!t), solve(l,r-1,!t));
  }
}

signed main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n; cin >> n;
  for(int i=0;i<n;i++) cin >> v[i];

  cout << solve(0,n-1,true) << endl;

  return 0;
}
```

4.15 removing digits

```cpp
#include <bits/stdc++.h>
using namespace std;

#define endl '\n' //<< flush

int main() {
    ios_base::sync_with_stdio(0);cin.tie(0);
    int n; cin >> n;
    int contador = 0;
    while (n != 0) {
        // mx -> maior digito de n
        // temp -> uma copia de n
        int mx = 0, temp = n;

        // Percorro todos os digitos de n (sem alterar n)
        // e pego maior digito
        while (temp != 0) {
            int digito = temp % 10;
            mx = max(mx, digito);
            temp = temp / 10;
        }

        n -= mx;
        contador++;
    }
    cout << contador << endl;
}
```

4.16 two sets ii

```cpp
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back
#define int long long

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;

signed main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);

  int t = 1;
  while (t--) {
    int n;
    cin >> n;
    int total = (n * (n + 1)) / 2;
    if (total % 2 != 0)
      cout << 0 << endl;
    else {
      int mx = total / 2;
      int dp[mx + 1];
      fill(dp, dp + mx + 1, 0);
      dp[0] = 1;
      for (int i = 1; i <= n; i++) {
        for (int j = mx; j >= 0; j--) {
          if (dp[j] > 0) {
            if (j + i <= mx) {
              dp[j + i] = (dp[j + i] + dp[j]) % MOD;
            }
          }
        }
      }
      cout << (dp[mx] * 500000004) % MOD << endl;
    }
  }

  return 0;
}
```

5 Classics - Geometry

5.1 convex hull

```cpp
#include <bits/stdc++.h>
using namespace std;

#define FAST_IO ios_base::sync_with_stdio(0);cin.tie(0);
#define endl '\n'
#define pb push_back
#define eb emplace_back
#define ff first
#define ss second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()

#define dbg(x) cout << #x << " = " << x << endl

typedef long long ll;
typedef pair<int, int> pi;

const ll MOD = 1e9 + 7;
const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

#define int long long int

struct pt {
    int x, y, id;
    pt(int x, int y, int id) : x(x), y(y), id(id) {}
    pt() {}
    pt operator-(pt const &o) const {
        return {x - o.x, y - o.y, -1};
    }
    bool operator<(pt const &o) const {
        if (x == o.x) {
            return y < o.y;
        }
        return x < o.x;
    }
    int operator^(pt const &o) const {
        return (int)x * o.y - (int)y * o.x;
    }
};
int ccw(pt const &a, pt const &b, pt const &x) {
    auto p = (b - a) ^ (x - a);
    return (p > 0) - (p < 0);
}
vector<pt> convex_hull(vector<pt> P, bool include_collinear) {
        // include_collinear: define se vai retornar os pontos colineares as
            bordas
        // ou seja, pontos da aresta do poligono
    sort(P.begin(), P.end());
    vector<pt> L, U;
    int pop_threshold = include_collinear ? -1 : 0;
    for (auto p : P) {
```

```
        while (L.size() >= 2 && ccw(L.end()[-2], L.end()[-1], p) <=
             pop_threshold) {
            L.pop_back();
        }
        L.push_back(p);
    }
    reverse(P.begin(), P.end());
    for (auto p : P) {
        while (U.size() >= 2 && ccw(U.end()[-2], U.end()[-1], p) <=
             pop_threshold) {
            U.pop_back();
        }
        U.push_back(p);
    }
    L.insert(L.end(), U.begin() + 1, U.end() - 1);
    return L;
}
signed main(){
    FAST_IO
    int n; cin >> n;
    vector<pt> arr;
    for (int i = 0; i < n; i++) {
                int x, y; cin >> x >> y;
                arr.emplace_back(x, y, i+1);
        }
        vector<pt> ans = convex_hull(arr, true);
        cout << ans.size() << endl;
    for (auto [x, y, id] : ans) {
                cout << x << " " << y << endl;
        }
}
```

5.2 intersection points

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define all(x) (x).begin(), (x).end()
#define int long long

#define HORIZONTAL_INICIO 0
#define VERTICAL 1
#define HORIZONTAL_FIM 2

// 10^-6 <= y <= 10^6
#define MAX_Y 2 * 1e6 + 1

struct fenw {
    int n;
    vector<int> bit;

    fenw() {}
    fenw(int size) {
        n = size;
        bit.assign(size + 1, 0);
    }
    // query do prefixo a[0] + a[1] + ... + a[r]
    int qry(int r) {
        int ans = 0;
        for (int i = r + 1; i > 0; i -= i & -i) // i & -i retorna os bits menos
             signativos de i
            ans += bit[i];
        return ans;
    }

    // atualiza o valor a[r] = x
    void upd(int r, int x) {
        for (int i = r + 1; i <= n; i += i & -i) bit[i] += x;
    }
};

signed main(){
    ios_base::sync_with_stdio(0);cin.tie(0);
    int n; cin >> n;
    vector<tuple<int, int, int>> ev;
    int y_horizontal[n];
    pair<int, int> y_vertical[n];
    for (int i = 0; i < n; i++) {
        int x1, y1, x2, y2; cin >> x1 >> y1 >> x2 >> y2;
        if (x1 == x2) {
            // Reta Vertical
            ev.emplace_back(x1, VERTICAL, i);
            y_vertical[i] = {y1, y2};
        } else {
            // Reta Horizontal
            ev.emplace_back(x1, HORIZONTAL_INICIO, i);
            ev.emplace_back(x2, HORIZONTAL_FIM, i);
            y_horizontal[i] = y1;
        }
    }

    sort(all(ev));

    fenw tree(MAX_Y);

    int ans = 0;
    for (auto [x, tipo, id] : ev) {
        if (tipo == HORIZONTAL_INICIO) {
            int yh = y_horizontal[id];
            tree.upd(yh + 1e6, +1); // Liga a reta Horizontal
        } else if (tipo == HORIZONTAL_FIM) {
            int yh = y_horizontal[id];
            tree.upd(yh  + 1e6, -1); // Desliga a reta Horizontal
        } else { // VERTICAL
            auto [y1, y2] = y_vertical[id];
            ans += tree.qry(y2  + 1e6) - tree.qry(y1  + 1e6);
        }
    }

    cout << ans << endl;
}
```

5.3 line segment intersection

```
#include <bits/stdc++.h>
using namespace std;
#define ff first
#define ss second
#define int long long
#define endl '\n' //<< flush
#define dbg(x) cerr << #x << " = " << x << endl
#define pdbg(x) cerr << #x << " = " << x.ff << "," << x.ss << endl
typedef long long ll;
/* PRIMITIVAS */
#define int long long
#define sq(x) ((x)*(int)(x))
struct pt {
    int x, y;
    pt(int a=0, int b=0) : x(a), y(b) {}
    friend istream &operator>>(istream &in, pt &p) {
                int x, y; in >> p.x >> p.y; return in;
        }
    bool operator < (const pt p) const {
                if (x != p.x) return x < p.x; return y < p.y;
        }
        bool operator == (const pt p) const {
                return x == p.x and y == p.y;
        }
        pt operator + (const pt p) const { return pt(x+p.x, y+p.y); }
        pt operator - (const pt p) const { return pt(x-p.x, y-p.y); }
    // multiplicar um escalar ao vetor
        pt operator * (const int c) const { return pt(x*c, y*c); }
    // produto escalar
        int operator * (const pt p) const { return x*(int)p.x + y*(int)p.y; }
    // produto vetorial (norma de u x v)
    int operator ^ (const pt p) const { return x*(int)p.y - y*(int)p.x; }
};
struct line {
```

```
    pt p, q;
    line() {}
    line(pt _p, pt _q) : p(_p), q(_q) {}
    friend istream &operator>>(istream &in, line &r) {
        int x, y; in >> r.p >> r.q; return in;
    }
};
/* PONTO e VETOR */
int dist2(pt p, pt q) { return sq(p.x - q.x) + sq(p.y - q.y); }
// area do paralelogramo formado pelos vetores p->q e p->r
// dividir por 2 retorna a area do triangulo formado pelos pontos
int area2(pt p, pt q, pt r) { return (q - p) ^ (r - q); }
// verifica se p, q, r sao colineares
bool col(pt p, pt q, pt r) { return area2(p, q, r) == 0; }
// verifica se p -> q -> r e no sentido anti-horario
bool ccw(pt p, pt q, pt r) { return area2(p, q, r) > 0; }
// quadrante de um ponto -> 1, 2, 3, 4
int quad(pt p) { return (p.x<0)^3*(p.y<0); }
// retorna se ang(p) < ang(q) -> Radial Sort (ordena pelos angulos)
bool compare_angle(pt p, pt q) {
        if (quad(p) != quad(q)) return quad(p) < quad(q);
        return ccw(q, pt(0, 0), p);
}
// rotaciona 90 graus o vetor
// vetor resultante e o normal ao original
pt rotate90(pt p) { return pt(-p.y, p.x); }
/* RETA */
// verifica se P pertence ao segmento R
bool in_seg(pt p, line r) {
    pt a = r.p - p, b = r.q - p;
    // (a * b) <= garante que p esta entre [r.p, r.q]
    return (a ^ b) == 0 and (a * b) <= 0;
}
// se o seg de r intersecta o seg de s
bool inter_seg(line r, line s) {
        if (in_seg(r.p, s) or in_seg(r.q, s)
                or in_seg(s.p, r) or in_seg(s.q, r)) return 1;

        return ccw(r.p, r.q, s.p) != ccw(r.p, r.q, s.q) and
                        ccw(s.p, s.q, r.p) != ccw(s.p, s.q, r.q);
}
signed main() {
    ios_base::sync_with_stdio(0); cin.tie(0);
    int tc; cin >> tc;
    while (tc--) {
        line r, s;
        cin >> r >> s;
        if (inter_seg(r, s)) cout << "YES";
        else cout << "NO";
        cout << endl;
    }
}
```

5.4 maximum manhattan distances

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define ff first
#define ss second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define sz(x) (x).size()
#define pb push_back

typedef long long ll;
typedef double ld;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;
typedef vector<vl> vvl;
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f1ll;
const double eps = 1e-9;
const ll MOD = 1e9 + 7;

bool eq(ld a, ld b) {
        return abs(a - b) <= eps;
}

struct pt { // ponto
        ld x, y;
        pt(ld x_ = 0, ld y_ = 0) : x(x_), y(y_) {}
        bool operator < (const pt p) const {
                if (!eq(x, p.x)) return x < p.x;
                if (!eq(y, p.y)) return y < p.y;
                return 0;
        }
        bool operator == (const pt p) const {
                return eq(x, p.x) and eq(y, p.y);
        }
        pt operator + (const pt p) const { return pt(x+p.x, y+p.y); }
        pt operator - (const pt p) const { return pt(x-p.x, y-p.y); }
        pt operator * (const ld c) const { return pt(x*c  , y*c  ); }
        pt operator / (const ld c) const { return pt(x/c  , y/c  ); }
        ld operator * (const pt p) const { return x*p.x + y*p.y; }
        ld operator ^ (const pt p) const { return x*p.y - y*p.x; }
        friend istream& operator >> (istream& in, pt& p) {
                return in >> p.x >> p.y;
        }
};

int main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n; cin >> n;
  ll x_mi=LINF, y_mi = LINF;
  ll x_ma = -LINF, y_ma = -LINF;
  for(int i=0;i<n;i++){
    ll ox, oy; cin >> ox >> oy;
    ll x = (ox-oy);
    ll y = (ox+oy);
    x_mi =min(x_mi, x), y_mi = min(y_mi, y);
    x_ma = max(x_ma, x), y_ma = max(y_ma,y);
    ll ans = max(x_ma-x_mi, y_ma-y_mi);
    cout << ans << endl;
  }
  return 0;
}
```

5.5 minimum euclidean distance

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define ff first
#define ss second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define sz(x) (x).size()
#define pb push_back

typedef long long ll;
typedef double ld;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;
typedef vector<vl> vvl;
```

```
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double eps = 1e-9;
const ll MOD = 1e9 + 7;

ll get_dist(pll a, pll b) {
    return (a.ff - b.ff) * (a.ff - b.ff) + (a.ss - b.ss) * (a.ss - b.ss);
}

int main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n; cin >> n;
  vector<pll> p(n);
  for(int i=0;i<n;i++) cin >> p[i].ff >> p[i].ss;

  sort(all(p), [](pii a, pii b){
      if(a.ff == b.ff) return a.ss < b.ss;
      return a.ff < b.ff;
      });

  set<pll> st;
  st.insert({p[0].ss, p[0].ff});
  ll min_dist = LLONG_MAX;
  int j=0;
  for(int i=1;i<n;i++){
    ll d = (min_dist == LLONG_MAX) ? 4e9 : sqrt(min_dist) + 1;
    while(j < i and p[j].ff < p[i].ff - d){
      st.erase({p[j].ss, p[j].ff});
      j++;
    }

    auto l = st.lower_bound({p[i].ss - d,-4e9});
    auto r = st.upper_bound({p[i].ss + d, 4e9});

    for(auto it = l; it != r;++it){
      ll dist = get_dist(p[i], {it->ss, it->ff});
      if(min_dist > dist){
        min_dist = dist;
        dist = sqrt(min_dist);
      }
    }
    st.insert({p[i].ss, p[i].ff});
  }
  cout << min_dist << endl;

  return 0;
}
```

5.6 point in polygon

```
#include <bits/stdc++.h>
using namespace std;
#define ff first
#define ss second
#define int long long
#define endl '\n' //<< flush
#define dbg(x) cerr << #x << " = " << x << endl
#define pdbg(x) cerr << #x << " = " << x.ff << "," << x.ss << endl
typedef long long ll;

struct pt {
    int x, y;
    pt(int a=0, int b=0) : x(a), y(b) {}
    friend istream &operator>>(istream &in, pt &p) {
                int x, y; in >> p.x >> p.y; return in;
        }
    bool operator < (const pt p) const {
                if (x != p.x) return x < p.x; return y < p.y;
        }
        bool operator == (const pt p) const {
                return x == p.x and y == p.y;
        }
        pt operator + (const pt p) const { return pt(x+p.x, y+p.y); }
        pt operator - (const pt p) const { return pt(x-p.x, y-p.y); }
    // multiplicar um escalar ao vetor
        pt operator * (const int c) const { return pt(x*c, y*c); }
    // produto escalar
        int operator * (const pt p) const { return x*(int)p.x + y*(int)p.y; }
    // produto vetorial (norma de u x v)
    int operator ^ (const pt p) const { return x*(int)p.y - y*(int)p.x; }
};

int inpol(vector<pt>& v, pt p) { // O(|v|)
    int qt = 0;
    for (int i = 0; i < v.size(); i++) {
        if (p == v[i]) return 2;
        int j = (i + 1) % v.size();
        if (p.y == v[i].y and p.y == v[j].y) {
                        if ((v[i]-p)*(v[j]-p) <= 0) return 2;
                }
        bool baixo = v[i].y < p.y;
                if (baixo == (v[j].y < p.y)) continue;
                auto t = (p-v[i])^(v[j]-v[i]);
                if (!t) return 2;
                if (baixo == (t > 0)) qt += (baixo ? 1 : -1);
    }
    return qt != 0;
}

signed main() {
    ios_base::sync_with_stdio(0); cin.tie(0);
    int n, m; cin >> n >> m;
    vector<pt> pol(n);
    for (auto &x : pol) cin >> x;
    while (m--) {
        pt x; cin >> x;
        int flag = inpol(pol, x);
        if (flag == 2) {
            cout << "BOUNDARY";
        } else if (flag == 1) {
            cout << "INSIDE";
        } else {
            cout << "OUTSIDE";
        }
        cout << endl;
    }
}
```

5.7 point location test

```
#include <bits/stdc++.h>
using namespace std;

#define ff first
#define ss second
#define int long long
#define endl '\n' //<< flush
#define dbg(x) cerr << #x << " = " << x << endl
#define pdbg(x) cerr << #x << " = " << x.ff << "," << x.ss << endl

typedef long long ll;
typedef pair<int, int> pt;

int cross(pt v, pt u) {
    return (v.ff * u.ss) - (v.ss * u.ff);
}

signed main() {
    int tc; cin >> tc;
    while (tc--) {
        int x1, y1, x2, y2, x3, y3;
        cin >> x1 >> y1 >> x2 >> y2 >> x3 >> y3;
        pt a = {x1, y1}, b = {x2, y2}, c = {x3, y3};
        // vetores v e u
        pt v = {b.ff - a.ff, b.ss - a.ss}, u = {c.ff - a.ff, c.ss - a.ss};
        int prod = cross(v, u);
        if (prod == 0) cout << "TOUCH";
```

```
        else if (prod > 0) cout << "LEFT";
        else cout << "RIGHT";
        cout << endl;
    }
}
/*
Dados os tres pontos A, B e C, considere os vetores v = B - A e u = C - A.
Queremos descobrir se:
- A, B e C sao colineares
- Se C esta a direita da reta AB
- Se C esta a esquerda da reta AB
Podemos descobrir isso a partir da norma do produto vetorial v x u.
Lembre-se que a norma de v x u e dado por det(v.x   v.y) = (v.x * u.y) - (u.x *
     v.y)
                                              u.x   u.y
Caso o determinante seja:
- = 0, temos que v e u sao paralelos, logo A, B e C sao colineares
- < 0, temos que a area e negativa (regra da mao), C esta a direita.
- > 0, temos que a area e positiva (regra da mao), C esta a esquerda.
*/
```

5.8 polygon area

```
#include <bits/stdc++.h>
using namespace std;
#define ff first
#define ss second
#define int long long
#define endl '\n' //<< flush
#define dbg(x) cerr << #x << " = " << x << endl
#define pdbg(x) cerr << #x << " = " << x.ff << "," << x.ss << endl
typedef long long ll;

struct pt {
    int x, y;
    pt(int a=0, int b=0) : x(a), y(b) {}
    friend istream &operator>>(istream &in, pt &p) {
                int x, y; in >> p.x >> p.y; return in;
        }
    bool operator < (const pt p) const {
                if (x != p.x) return x < p.x; return y < p.y;
        }
        bool operator == (const pt p) const {
                return x == p.x and y == p.y;
        }
        pt operator + (const pt p) const { return pt(x+p.x, y+p.y); }
        pt operator - (const pt p) const { return pt(x-p.x, y-p.y); }
    // multiplicar um escalar ao vetor
        pt operator * (const int c) const { return pt(x*c, y*c); }
    // produto escalar
        int operator * (const pt p) const { return x*(int)p.x + y*(int)p.y; }
    // produto vetorial (norma de u x v)
    int operator ^ (const pt p) const { return x*(int)p.y - y*(int)p.x; }
};

int area2(pt p, pt q, pt r) { return (q - p) ^ (r - q); }

int polarea2(vector<pt> v) {
    int ret = 0;
    for (int i = 0; i < v.size(); i++)
        ret += area2(pt(0, 0), v[i], v[(i+1) % v.size()]);
    return abs(ret);
}

signed main() {
    ios_base::sync_with_stdio(0); cin.tie(0);
    int n; cin >> n;
    vector<pt> v(n);
    for (int i = 0; i < n; i++) cin >> v[i];
    cout << polarea2(v) << endl;
}
```

5.9 polygon lattice points

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define ff first
#define ss second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define sz(x) (x).size()
#define pb push_back

typedef long long ll;
typedef double ld;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;
typedef vector<vl> vvl;
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f1ll;
const double eps = 1e-9;
const ll MOD = 1e9 + 7;

bool eq(ld a, ld b) {
        return abs(a - b) <= eps;
}

struct pt { // ponto
        ll x, y;
        pt(ld x_ = 0, ld y_ = 0) : x(x_), y(y_) {}
        bool operator < (const pt p) const {
                if (!eq(x, p.x)) return x < p.x;
                if (!eq(y, p.y)) return y < p.y;
                return 0;
        }
        bool operator == (const pt p) const {
                return eq(x, p.x) and eq(y, p.y);
        }
        pt operator + (const pt p) const { return pt(x+p.x, y+p.y); }
        pt operator - (const pt p) const { return pt(x-p.x, y-p.y); }
        pt operator * (const ld c) const { return pt(x*c  , y*c  ); }
        pt operator / (const ld c) const { return pt(x/c  , y/c  ); }
        ll operator * (const pt p) const { return x*p.x + y*p.y; }
        ll operator ^ (const pt p) const { return x*p.y - y*p.x; }
        friend istream& operator >> (istream& in, pt& p) {
                return in >> p.x >> p.y;
        }
};


int main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n; cin >> n;
  vector<pt> points(n);
  for(auto &[x,y]: points) cin >> x >> y;

  ll area=0;
  for(int i=0;i<n;i++) area+=points[i] ^ points[(i+1)%n];
  area = abs(area);
  ll ans=0;
  for(int i=0;i<n;i++){
    int j = (i+1)%n;
    pt diff = points[j] - points[i];
    int g = gcd(abs(diff.x), abs(diff.y));
    ans += g;
  }
  ll interior = (area - ans + 2)/2;
  cout << interior << " " << ans << endl;
```

```
  return 0;
}
```

6 Classics - Graph Algorithms

6.1 building roads

```
#include <bits/stdc++.h>
using namespace std;
vector<int> adj[1000000];
bool vis[1000000];
void dfs(int v){
    vis[v] = true;

    for(auto u : adj[v]) if(!vis[u])dfs(u);
}
int main(){
    int n, m; cin >> n >> m;
    for(int i = 0; i < m ; i++)
    {
        int a, b; cin >> a >> b;

        adj[a].push_back(b);
        adj[b].push_back(a);

    }
    vector<int> comp;
    for(int i = 1; i <= n; i++)
    {
        if(!vis[i]){
            dfs(i);
            comp.push_back(i);
        }
    }
    cout << comp.size() - 1 << endl;
    int v = comp[0];
    for(int i =1; i < comp.size(); i++){

        cout << v << " " << comp[i] << endl;
    }
    return 0;
}
```

6.2 building teams

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define pb push_back
#define eb emplace_back
#define ff first
#define ss second
#define sz(x) (int) (x).size()
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cout << #x << " = " << x << endl

typedef long long ll;
typedef pair<int, int> pi;

const ll MOD = 1e9 + 7;
const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

const int MAX = 1e5 + 10;

int n, m, c;
vector<int> adj[MAX];
int color[MAX];

bool bfs(int s) {
    queue<int> q; q.push(s); color[s] = 0;
    while (!q.empty()) {
        int v = q.front(); q.pop();
        for (auto u : adj[v]) {
            if (color[u] == -1) { // nao visitado
                color[u] = !color[v];
                q.push(u);
            } else if (color[u] == color[v]) {
                return false;
            }
        }
    }
    return true;
}

signed main(){
    ios_base::sync_with_stdio(0);cin.tie(0);
    memset(color, -1, sizeof(color));
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int a, b; cin >> a >> b; a--, b--;
        adj[a].pb(b);
        adj[b].pb(a);
    }
    // Definindo as cores
    bool ok = true;
    for (int i = 0; i < n; i++) if(color[i] == -1) {
        ok = bfs(i);
        if (!ok) break;
    }
    if (!ok) {
        cout << "IMPOSSIBLE" << endl;
    } else {
        for (int i = 0; i < n; i++) {
            cout << color[i]+1 << " ";
        }
        cout << endl;
    }
}
```

6.3 counting rooms

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define pb push_back
#define eb emplace_back
#define ff first
#define ss second
#define sz(x) (int) (x).size()
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cout << #x << " = " << x << endl

typedef long long ll;
typedef pair<int, int> pi;

const ll MOD = 1e9 + 7;
const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

const int MAX = 1e3 + 10;
const pi mov[4] = {{-1, 0}, {1, 0}, {0, -1}, {0, 1}};

int n, m;
char grid[MAX][MAX];
bool vis[MAX][MAX];

bool valid(int i, int j) {
    return i>=0 and j>=0 and i<n and j<m
            and !vis[i][j] and grid[i][j] == '.';
```

```
}

void dfs(pi v) {
    auto [i, j] = v;
    vis[i][j] = true;
    for (auto [di, dj] : mov) {
        int ni = i + di, nj = j + dj;
        if (valid(ni, nj)) {
            vis[ni][nj] = true;
            dfs({ni, nj});
        }
    }
}

signed main(){
    ios_base::sync_with_stdio(0);cin.tie(0);
    cin >> n >> m;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            cin >> grid[i][j];
        }
    }
    int ans = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (!vis[i][j] and grid[i][j] == '.') {
                ans++;
                dfs({i, j});
            }
        }
    }
    cout << ans << endl;
}
```

6.4 course schedule

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define int long long
#define pb push_back
#define eb emplace_back
#define ff first
#define ss second
#define sz(x) (int) (x).size()
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cout << #x << " = " << x << endl

const int MAXN = 1e5 + 10;

int n, m;
vector<int> adj[MAXN], color(MAXN, 0);
deque<int> order(MAXN);

void dfs(int v) {
    color[v] = 1;
    for (auto u : adj[v]) {
        if (color[u] == 0) dfs(u);
        if (color[u] == 1) {
            cout << "IMPOSSIBLE" << endl;
            exit(0);
        }
    }
    order.push_front(v);
    color[v] = 2;
}

signed main(){
    ios_base::sync_with_stdio(0);cin.tie(0);
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int u, v; cin >> u >> v; u--, v--;
        adj[u].pb(v);
    }
    for (int v = 0; v < n; v++) {
        if (color[v] == 0) dfs(v);
    }
    for (int i = 0; i < n; i++)
        cout << order[i] + 1 << " ";
    cout << endl;
}
```

6.5 cycle finding

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back
#define int long long

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;

struct Edge {
  int a, b, cost;
};

signed main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);

  int n, m;
  cin >> n >> m;
  vector<Edge> edges;
  for (int i = 0; i < m; i++) {
    int u, v, w;
    cin >> u >> v >> w;
    edges.pb({u, v, w});
  }
  vi d(n + 1, 0);
  d[1] = 0;
  vi p(n + 1, -1);
  int x;
  for (int i = 0; i <= n; ++i) {
    x = -1;
    for (Edge e : edges)
      if (d[e.b] > d[e.a] + e.cost) {
        d[e.b] = max(LONG_LONG_MIN, d[e.a] + e.cost);
        p[e.b] = e.a;
        x = e.b;
      }
  }

  if (x == -1)
    cout << "NO" << endl;
  else {
    cout << "YES" << endl;
    int y = x;
    for (int i = 0; i < n; ++i)
      y = p[y];

    vi path;
```

```
    for (int cur = y;; cur = p[cur]) {
      path.pb(cur);
      if (cur == y && path.size() > 1)
        break;
    }
    reverse(all(path));

    for (int u : path)
      cout << u << ' ';
  }
  return 0;
}
```

6.6 distinct routes

```
// Source: https://usaco.guide/general/io

#include <bits/stdc++.h>
using namespace std;

#define ff first
#define ss second
#define pb push_back
#define int long long
#define endl '\n'
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cerr << #x << " = " << x << endl
#define uniq(v) sort(all(v)); v.erase(unique(all(v)), v.end())

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

typedef long long ll;
typedef pair<int, int> pi;

const int INF = 4e18;
const int MAXN = 1000 + 5;

int n, m;
vector<int> g[MAXN];
vector<int> parent;
int capacity[MAXN][MAXN];
int capacityCopy[MAXN][MAXN];
int used[MAXN][MAXN];

int bfs(int s, int t){
    fill(all(parent), -1);

    queue<pi> q;
    parent[s] = -2;

    q.push({s, INF});

    while(!q.empty()){
        auto [u, flow] = q.front();
        q.pop();

        for(int v : g[u]){
            if(parent[v] == -1 && capacity[u][v] > 0){
                parent[v] = u;

                int newFlow = min(flow, capacity[u][v]);

                if(v == t){
                    return newFlow;
                }

                q.push({v, newFlow});
            }
        }
    }

    return 0;
}

int maxFlow(int s, int t){
    int flow = 0;

    while(true){
        int bottleneck = bfs(s, t);

        if(bottleneck == 0){
            break;
        }

        flow += bottleneck;

        int cur = t;

        while(cur != s){
            int prev = parent[cur];

            capacity[prev][cur] -= bottleneck;
            capacity[cur][prev] += bottleneck;

            cur = prev;
        }
    }

    return flow;
}

vector<int> getPath(){
    vector<int> path;

    int cur = 0;
    path.pb(cur);

    while(cur != n - 1){
        for(int v : g[cur]){
            if(used[cur][v] > 0){
                used[cur][v]--;
                cur = v;
                path.pb(cur);
                break;
            }
        }
    }
    return path;
}

void solve(){
    cin >> n >> m;
    parent.assign(n, -1);

    for(int i = 0; i < m; i++){
        int a, b;
        cin >> a >> b;
        a--;
        b--;

        g[a].pb(b);
        g[b].pb(a);

        capacity[a][b] = 1;
        capacityCopy[a][b] = 1;

    }


    int k = maxFlow(0, n - 1);

      for(int i = 0; i < n; i++){
        for(int j = 0; j < n; j++){
           used[i][j] = capacityCopy[i][j] - capacity[i][j];
        }
    }

    cout << k << endl;

    for(int i = 0; i < k; i++){
        vector<int> path = getPath();

        cout << path.size() << endl;
```

```
        for(int v : path){
            cout << v + 1 << " ";
        }

        cout << endl;
    }
}

signed main(){
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);

    solve();
}
```

6.7 download speed

```
// Source: https://usaco.guide/general/io

#include <bits/stdc++.h>
using namespace std;
#define ff first
#define ss second
#define pb push_back
#define int long long
#define endl '\n'
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cerr << #x << " = " << x << endl
#define uniq(v) sort(all(v)); v.erase(unique(all(v)), v.end())

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

typedef long long ll;
typedef pair<int, int> pi;

const int INF = 4e18;
const int MAXN = 500 + 5;
const int MAX = 1e9 + 5;

int n, m;
vector<int> g[MAXN];
vector<int> parent;
int capacity[MAXN][MAXN];

int bfs(int s, int t){
    fill(all(parent), - 1);

    queue<pi> q;
    parent[s] = -2;

    q.push({s, INF});

    while(!q.empty()){
        auto[u, flow] = q.front();
        q.pop();

        for(int v : g[u]){
            if(parent[v] == -1 && capacity[u][v] > 0){
                parent[v] = u;

                int newFlow = min(flow, capacity[u][v]);

                if(v == t){
                    return newFlow;
                }

                q.push({v, newFlow});
            }
        }
    }

    return 0;
}

int maxFlow(int s, int t){
    int flow = 0;

    while(true){
        int bottleneck = bfs(s, t);

        if(bottleneck == 0){
            break;
        }

        flow += bottleneck;

        int cur = t;

        while(cur != s){
            int prev = parent[cur];

            capacity[prev][cur] -= bottleneck;
            capacity[cur][prev] += bottleneck;

            cur = prev;
        }
    }

    return flow;
}

void solve(){
    cin >> n >> m;
    parent.assign(n, -1);

    for(int i = 0; i < m; i++){
        int a, b, c;
        cin >> a >> b >> c;

        a--;
        b--;

        g[a].pb(b);
        g[b].pb(a);

        capacity[a][b] += c;
    }

    cout << maxFlow(0, n - 1) << endl;
}

signed main(){
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);

    solve();
}
```

6.8 flight discount

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define ff first
#define ss second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define sz(x) (int)(x).size()
#define pb push_back

typedef long long ll;
typedef double ld;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
```

```
typedef vector<pll> vpll;
typedef vector<vi> vvi;
typedef vector<vl> vvl;
//mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double eps = 1e-9;
const ll MOD = 1e9 + 7;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n,m; cin >> n >> m;

  vector<vpll> g(n);
  for(int i=0;i<m;i++){
    int u,v,w; cin >> u >> v >> w;
    u--,v--;
    g[u].emplace_back(v,w);
  }

  priority_queue<array<ll,3>> q;
  q.push({0,0,0});
  vvl dist(n, vl(2,LINF));
  vector<vb> vis(n, vb(2,false));
  dist[0][0]=0;
  while(!q.empty()){
    auto [w, u,c] = q.top();
    q.pop();
    if(dist[u][c]==LINF or vis[u][c]) continue;
    vis[u][c]=1;
    for(auto [v,cost] : g[u]){
        if(c==0){
            ll cur1=dist[u][c]+cost/2LL;
            ll cur2=dist[u][c]+cost;
            if(cur1<dist[v][1]){
                dist[v][1]=cur1;
                q.push({-cur1, v, 1});
            }
            if(cur2<dist[v][0]){
                dist[v][0]=cur2;
                q.push({-cur2,v,0});
            }
        }else{
            ll cur = dist[u][c]+cost;
            if(cur<dist[v][c]){
                dist[v][c]=cur;
                q.push({-cur,v,1});
            }
        }
    }
  }

  cout<<min(dist[n-1][0], dist[n-1][1])<<endl;

  return 0;
}
```

6.9 game routes

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define ff first
#define ss second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define sz(x) (x).size()
#define pb push_back

typedef long long ll;
typedef double ld;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;
typedef vector<vl> vvl;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;

const int N = 1e5 + 10;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n, m;
  cin >> n >> m;
  vvi adj(n + 1);
  vi in(n+1,0);
  for (int i = 0; i < m; i++) {
    int u, v;
    cin >> u >> v;
    adj[u].pb(v);
    in[v]++;
  }

  vi dp(n+1,0);
  vi acc(n+1,0);
  dp[1] = 1;
  queue<int>q;
  q.push(1);
  for(int i=2;i<=n;i++) if(in[i]==0)q.push(i);
  while(!q.empty()){
      int u = q.front(); q.pop();
      for(auto v : adj[u]){
          in[v]--;
          if(in[v] == 0){
              dp[v] = (dp[v]+dp[u] + acc[v])%MOD;
              q.push(v);
          }else acc[v] = (acc[v] + dp[u])%MOD;
      }
  }

  cout << dp[n] << endl;

  return 0;
}
```

6.10 hamiltonian flights

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define ff first
#define ss second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define sz(x) (x).size()
#define pb push_back

typedef long long ll;
typedef double ld;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;
```

```
typedef vector<vl> vvl;
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n, m;
  cin >> n >> m;

  vvi adj(n);
  for (int i = 0; i < m; i++) {
    int u, v;
    cin >> u >> v;
    u--,v--;
    adj[v].pb(u);
  }


  int sz = (1 << n);
  vvi dp(sz, vi(n, 0));
  dp[1][0] = 1;
  for (int mask = 2; mask < sz; mask++) {
    if((mask & 1)==0) continue;
    if((mask & (1<<(n-1))) and mask != ((1<<n)-1)) continue;

    for(int i=1;i<=n;i++){
      if((mask & (1<<i))==0) continue;

      int prev = mask ^ (1<<i);
      for(auto v : adj[i]){
        if((mask & (1 << v))){
          dp[mask][i] += dp[prev][v];
          dp[mask][i] %= MOD;
        }
      }
    }
  }
  cout << dp[sz - 1][n-1] << endl;

  return 0;
}
```

6.11 high score

```
#include <bits/stdc++.h>

#define nl '\n'
#define ll long long
#define ull unsigned long long
#define ld long double
#define pii pair<long long, long long>
#define pb push_back

using namespace std;

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

typedef __gnu_pbds::tree<int, __gnu_pbds::null_type, less<int>, __gnu_pbds::
    rb_tree_tag, __gnu_pbds::tree_order_statistics_node_update> ordered_set;

const ll INF = 1e14 + 5;
const ll P = 13;
const ll M = 1e8;
const ll LG = 32;
const ll MAX = 1e2 + 5;

int n, m, k, h, w, q;

vector<vector<ll>> g;
vector<bool> vis;

struct Edge {
    ll a, b, cost;
};

void dfs(ll u) {
    vis[u] = true;
    for (ll v : g[u]) {
        if (!vis[v]) dfs(v);
    }
}

void solve() {
    cin >> n >> m;

    vis.assign(n, false);
    g.assign(n, vector<ll>());
    vector<Edge> edges(m);

    for (ll i = 0; i < m; i++) {
        cin >> edges[i].a >> edges[i].b >> edges[i].cost;
        edges[i].a--, edges[i].b--;
        g[edges[i].a].pb(edges[i].b);
    }

    vector<ll> d(n, -INF);
    d[0] = 0;

    vector<bool> bad(n, false);
    for (int i = 0; i < n; ++i) {
        for (Edge e : edges)
            if (d[e.a] > -INF)
                if (d[e.b] < d[e.a] + e.cost) {
                    d[e.b] = min(INF, d[e.a] + e.cost);
                    if (i == n-1) bad[e.b] = true;
                }
    }

    for (ll i = 0; i < n; i++) {
        if (!vis[i] && bad[i]) {
            dfs(i);
        }
    }

    if (vis[n-1]) {
        cout << -1 << nl;
    } else {
        cout << d[n-1] << nl;
    }
}

int main()
{
        ios::sync_with_stdio(0), cin.tie(0);
        int t = 1;
    //cin >> t;
        for (int i = 1; i <= t; i++) {
            solve();
        }
        return 0;
}
```

6.12 labyrinth

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define pb push_back
#define eb emplace_back
#define ff first
#define ss second
#define sz(x) (int) (x).size()
#define all(x) (x).begin(), (x).end()
```

```
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cout << #x << " = " << x << endl

typedef long long ll;
typedef pair<int, int> pi;

const ll MOD = 1e9 + 7;
const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

const int MAX = 1e3 + 10;
const tuple<int, int, char> mov[4] = {
    {-1, 0, 'U'}, {1, 0, 'D'}, {0, -1, 'L'}, {0, 1, 'R'}
};

int n, m;
char grid[MAX][MAX], directions[MAX][MAX];
int dist[MAX][MAX];
pi parent[MAX][MAX];

bool valid(int i, int j) {
    return i>=0 and j>=0 and i<n and j<m
            and (dist[i][j] == -1) and (grid[i][j] != '#');
}

void bfs(pi s) {
    queue<pi> q; q.push(s);
    dist[s.ff][s.ss] = 0;
    parent[s.ff][s.ss] = {-1, -1};
    while (!q.empty()) {
        auto [i, j] = q.front(); q.pop();
        for (auto [di, dj, d] : mov) {
            int ni = i + di, nj = j + dj;
            if (valid(ni, nj)) {
                q.push({ni, nj});
                dist[ni][nj] = dist[i][j] + 1;
                parent[ni][nj] = {i, j};
                directions[ni][nj] = d;
            }
        }
    }
}

signed main(){
    memset(dist, -1, sizeof(dist));
    ios_base::sync_with_stdio(0);cin.tie(0);
    cin >> n >> m;
    pi a, b;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            cin >> grid[i][j];
            if (grid[i][j] == 'A') a = {i, j};
            else if (grid[i][j] == 'B') b = {i, j};
        }
    }
    bfs(a);
    if (dist[b.ff][b.ss] == -1) { // Nao visitado
        cout << "NO" << endl;
    } else {
        cout << "YES" << endl;
        cout << dist[b.ff][b.ss] << endl;
        vector<char> path;
        for (pi v = b; parent[v.ff][v.ss] != pi{-1, -1}; v = parent[v.ff][v.ss])
            {
            path.pb(directions[v.ff][v.ss]);  // movimento que levou do pai a v
        }
        reverse(all(path));
        for (auto x : path) cout << x;
        cout << endl;
    }
}
```

6.13 longest flight route

```
#include <bits/stdc++.h>

using namespace std;

#define endl '\n'
#define ff first
#define ss second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define sz(x) (int)(x).size()
#define pb push_back

typedef long long ll;
typedef double ld;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;
typedef vector<vl> vvl;
//mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double eps = 1e-9;
const ll MOD = 1e9 + 7;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n,m; cin >> n >> m;
  vvi adj(n+1);
  vi indg(n+1,0);
  for(int i=0;i<m;i++){
    int u,v; cin >> u >> v;
    adj[u].pb(v);
    indg[v]++;
  }

  vi dist(n+1,-1);
  vi par(n+1,-1);
  dist[1]=0;
  queue<int> q;
  for(int i=1;i<=n;i++) if(indg[i]==0) q.push(i);

  while(!q.empty()){
    int u=q.front();q.pop();
    for(auto v : adj[u]){
      if(dist[u] != -1 and dist[v]<dist[u]+1){
        par[v]=u;
        dist[v]=dist[u]+1;
      }
      indg[v]--;
      if(indg[v]==0){
        q.push(v);
      }
    }
  }

  if(dist[n]==-1){
    cout << "IMPOSSIBLE"<<endl;
    return 0;
  }

  vi ans;
  int cur=n;
  while(cur!=-1){
    ans.pb(cur);
    cur=par[cur];
  }

  cout << sz(ans)<<endl;
  reverse(all(ans));
  for(auto x : ans){
    cout << x << " ";
  }
  cout << endl;
```

```
  return 0;
}
```

6.14 message route

```
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 1e6;
int n, m, dist[MAXN];
vector<int> g[MAXN];
int par[MAXN];
void bfs(int s) {
    queue<int> q; q.push(s); dist[s] = 0;
    while (!q.empty() ){
        int v=q.front(); q.pop();
        for (auto u : g[v]) if (dist[u] == -1) {
            q.push(u);
            dist[u] = dist[v] + 1;
            par[u] = v;
        }
    }
}

int main() {
    cin >> n >> m;
    memset(dist, -1, sizeof(dist));
    for (int i = 0; i < m; i++) {
        int a, b; cin >> a >> b; a--, b--;
        g[a].push_back(b);
        g[b].push_back(a);
    }
    bfs(0);
    if (dist[n-1] == -1) {
        cout << "IMPOSSIBLE" << endl;
        return 0;
    }

    int cur = n-1;
    par[0] = -1;
    vector<int> path;
    cout << dist[n-1] +1<< endl;
    while(cur != -1  ) {
        path.push_back(cur+1);
        cur = par[cur];
    }
    reverse(path.begin(), path.end());
    for (auto x : path) cout << x << " ";
    cout << endl;
}
```

6.15 monsters

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define pb push_back
#define eb emplace_back
#define ff first
#define ss second
#define sz(x) (int) (x).size()
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cout << #x << " = " << x << endl

typedef long long ll;
typedef pair<int, int> pi;

const ll MOD = 1e9 + 7;
const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

const int MAX = 1e3+10;
const tuple<int, int, char> mov[4] = {
    {-1, 0, 'U'}, {1, 0, 'D'}, {0, -1, 'L'}, {0, 1, 'R'}
};

int n, m;
char grid[MAX][MAX], dir[MAX][MAX];
int dist_person[MAX][MAX], dist_monster[MAX][MAX];
pi parent[MAX][MAX];
bool vis[MAX][MAX];

bool valid(int i, int j) {
    return i >= 0 and j >= 0 and i < n and j < m
            and !vis[i][j] and grid[i][j] != '#';
}

void bfs_ms(vector<pi> ms) {
    queue<pi> q;
    for (auto [i, j] : ms) {
        q.push({i, j});
        vis[i][j] = true;
        dist_monster[i][j] = 0;
    }
    while (!q.empty()) {
        auto [i, j] = q.front(); q.pop();
        for (auto [di, dj, c] : mov) {
            int ni = i + di, nj = j + dj;
            if (valid(ni, nj)) {
                q.push({ni, nj});
                vis[ni][nj] = true;
                dist_monster[ni][nj] = dist_monster[i][j] + 1;
            }
        }
    }
}

pi bfs(pi s) {
    queue<pi> q; q.push(s);
    vis[s.ff][s.ss] = true;
    dist_person[s.ff][s.ss] = 0;
    parent[s.ff][s.ss] = {-1, -1};
    dir[s.ff][s.ss] = '?';
    while (!q.empty()) {
        auto [i, j] = q.front(); q.pop();
        if ((i == 0) or (j == 0) or (i == n-1) or (j == m-1)) {
            return {i, j};
        }
        for (auto [di, dj, c] : mov) {
            int ni = i + di, nj = j + dj;
            if (valid(ni, nj) and (dist_person[i][j] + 1 < dist_monster[ni][nj])
                ) {
                q.push({ni, nj});
                vis[ni][nj] = true;
                dist_person[ni][nj] = dist_person[i][j] + 1;
                dir[ni][nj] = c;
                parent[ni][nj] = {i, j};
            }
        }
    }
    return {-1, -1};
}

signed main(){
    ios_base::sync_with_stdio(0);cin.tie(0);
    memset(dist_monster, INF, sizeof(dist_monster));
    cin >> n >> m;
    vector<pi> ms; pi s;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            cin >> grid[i][j];
            if (grid[i][j] == 'M') ms.eb(i, j);
            else if (grid[i][j] == 'A') s = {i, j};
        }
    }
    bfs_ms(ms);
    // for (int i = 0; i < n; i++) {
    //     for (int j = 0; j < m; j++) {
    //         cout << dist_monster[i][j];
    //     }
```

```
    //     cout << endl;
    // }
    memset(vis, false, sizeof(vis));
    pi ans = bfs(s);
    if (ans == pi{-1, -1}) {
        cout << "NO" << endl;
    } else {
        cout << "YES" << endl;
        vector<char> path;
        for (pi v = ans; v != pi{-1, -1}; v = parent[v.ff][v.ss]) {
            path.pb(dir[v.ff][v.ss]);
        }
        reverse(all(path));
        cout << sz(path)-1 << endl;
        for (int i = 1; i < sz(path); i++) {
            cout << path[i];
        }
        cout << endl;
    }
}
```

6.16 planets queries i

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back
#define int long long

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;

const int N = 2e5+10;
const int LOG = 31;

int t[N];
int anc[N][LOG];
vb vis;


void bl(int n){
  for(int k=1;k<LOG;k++){
    for(int i=0;i<n;i++){
      anc[i][k]= anc[anc[i][k-1]][k-1];
    }
  }
}

int query(int a, int k){

  for(int i=LOG-1;i>=0;i--){
    if((k>>i)&1) {
      a = anc[a][i];
    }
  }

  return a;
}
```

```
signed main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n,q; cin >> n >> q;
  vis.assign(n,false);
  for(int i=0;i<n;i++){
    cin >> t[i];
    t[i]--;
    anc[i][0]=t[i];
  }


  bl(n);
  while(q--){
    int x,k; cin >> x >> k;
    x--;
    if(t[x]==x)cout << x+1 << endl;
    else cout << query(x,k)+1 << endl;
  }

  return 0;
}
```

6.17 police chase

```
// Source: https://usaco.guide/general/io

#include <bits/stdc++.h>
using namespace std;
#define ff first
#define ss second
#define pb push_back
#define int long long
#define endl '\n'
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cerr << #x << " = " << x << endl
#define uniq(v) sort(all(v)); v.erase(unique(all(v)), v.end())

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

typedef long long ll;
typedef pair<int, int> pi;

const int INF = 4e18;
const int MAXN = 500 + 5;
const int MAX = 1e9 + 5;

int n, m;
vector<int> g[MAXN];
vector<int> parent;
vector<bool> vis;
vector<pi> edges;
int capacity[MAXN][MAXN];

int bfs(int s, int t){
    fill(all(parent), - 1);

    queue<pi> q;
    parent[s] = -2;

    q.push({s, INF});

    while(!q.empty()){
        auto[u, flow] = q.front();
        q.pop();

        for(int v : g[u]){
            if(parent[v] == -1 && capacity[u][v] > 0){
                parent[v] = u;

                int newFlow = min(flow, capacity[u][v]);

                if(v == t){
```

```
                return newFlow;
            }

            q.push({v, newFlow});
        }
    }
}

return 0;
}

int maxFlow(int s, int t){
    int flow = 0;

    while(true){
        int bottleneck = bfs(s, t);

        if(bottleneck == 0){
            break;
        }

        flow += bottleneck;

        int cur = t;

        while(cur != s){
            int prev = parent[cur];

            capacity[prev][cur] -= bottleneck;
            capacity[cur][prev] += bottleneck;

            cur = prev;
        }
    }

    return flow;
}

void dfsCut(int v){
    vis[v] = true;

    for(auto u : g[v]){
        if(!vis[u] && capacity[v][u] > 0){
            dfsCut(u);
        }
    }
}

void solve(){
    cin >> n >> m;
    parent.assign(n, -1);
    vis.assign(n, false);

    for(int i = 0; i < m; i++){
        int a, b;
        cin >> a >> b;

        a--;
        b--;

        g[a].pb(b);
        g[b].pb(a);

        capacity[a][b] += 1;
        capacity[b][a] += 1;

        edges.pb({a, b});
    }

    int ans = maxFlow(0, n - 1);
    dfsCut(0);

    cout << ans << endl;

    for(auto[u, v] : edges){
        if(vis[u] != vis[v]){
            cout << u + 1 << " " << v + 1 << endl;
        }
    }
}

signed main(){
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);

    solve();
}
```

6.18 road construction

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back
#define int long long

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;

struct DSU {
  vector<int> par, rnk, sz;
  int c;
  DSU(int n) : par(n + 1), rnk(n + 1, 0), sz(n + 1, 1), c(n) {
    for (int i = 1; i <= n; ++i) par[i] = i;
  }
  int find(int i) {
    return (par[i] == i ? i : (par[i] = find(par[i])));
  }
  bool same(int i, int j) {
    return find(i) == find(j);
  }
  int get_size(int i) {
    return sz[find(i)];
  }
  int count() {
    return c;
  }
  int merge(int i, int j) {
    if ((i = find(i)) == (j = find(j))) return -1;
    else --c;
    if (rnk[i] > rnk[j]) swap(i, j);
    par[i] = j;
    sz[j] += sz[i];
    if (rnk[i] == rnk[j]) rnk[j]++;
    return j;
  }
};

signed main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n,q; cin >> n >> q;
  DSU dsu(n);
  int mx =0;
  while(q--){
    int u,v; cin >> u >> v;
    dsu.merge(u,v);
```

```
    mx = max(mx,dsu.get_size(dsu.find(u)));
    cout << dsu.count() << " " << mx << endl;
  }

  return 0;
}
```

6.19 round trip

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define pb push_back
#define eb emplace_back
#define ff first
#define ss second
#define sz(x) (int) (x).size()
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cout << #x << " = " << x << endl

typedef long long ll;
typedef pair<int, int> pi;

const ll MOD = 1e9 + 7;
const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

const int MAX = 1e5 + 10;

int n, m;
vector<int> adj[MAX];
bool vis[MAX];
int parent[MAX], cycle_start, cycle_end;

bool dfs(int v) {
    vis[v] = true;
    for (auto u : adj[v]) {
        if (u==parent[v]) continue;
        if (vis[u]) {
            cycle_end = v;
            cycle_start = u;
            return true;
        }
        parent[u] = v;
        if (dfs(u)) return true;
    }
    return false;
}

// Encontrar ciclos simples!
signed main(){
    ios_base::sync_with_stdio(0);cin.tie(0);
    memset(parent, -1, sizeof(parent));
    cycle_start = -1;


    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int a, b; cin >> a >> b; a--, b--;
        adj[a].pb(b);
        adj[b].pb(a);
    }
    for (int v = 0; v < n; v++) {
        if (!vis[v] and dfs(v)) {
            // Encontrou um ciclo
            break;
        }
    }

    if (cycle_start == -1) {
        cout << "IMPOSSIBLE" << endl;
    } else {
        vector<int> cycle;
        cycle.push_back(cycle_start);
        for (int v = cycle_end; v != cycle_start; v = parent[v])
            cycle.push_back(v);
        cycle.push_back(cycle_start);
        cout << sz(cycle) << endl;
        for (auto x : cycle) cout << x+1 << " ";
        cout << endl;
    }
}
```

6.20 school dance

```
#include <bits/stdc++.h>
using namespace std;
#define pb push_back
#define eb emplace_back
// #define int long long
#define ff first
#define ss second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define nl '\n'

typedef long long ll;
typedef long double ld;
typedef pair<int, int> pi;
typedef vector<ll> vl;

const int MAXN = 1e3 + 5;
const ll INF = 2e9;

ll n, m, k, q;

vector<int> g[501];
vector<int> mt;
vector<bool> used;

bool try_kuhn(int v) {
    if (used[v])
        return false;
    used[v] = true;
    for (int to : g[v]) {
        if (mt[to] == -1 || try_kuhn(mt[to])) {
            mt[to] = v;
            return true;
        }
    }
    return false;
}

void solve() {
    cin >> n >> m >> k;

    for (ll i = 0; i < k; i++) {
        ll a, b; cin >> a >> b;
        a--, b--;
        g[a].pb(b);
    }

    mt.assign(m, -1);
    for (int v = 0; v < n; ++v) {
        used.assign(n, false);
        try_kuhn(v);
    }

    ll cnt = 0;
    vector<pair<ll, ll>> ans;
    for (int i = 0; i < m; ++i) {
        if (mt[i] != -1) {
            cnt++;
            ans.eb(mt[i] + 1, i + 1);
        }
    }

    cout << cnt << nl;
```

```
    for (auto [a, b] : ans) cout << a << " " << b << nl;
}

signed main() {
    ios_base::sync_with_stdio(0);cin.tie(0);
    int tc = 1;
    // cin >> tc;
    while (tc--) solve();
}
```

6.21 shortest routes i

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define pb push_back
#define eb emplace_back
#define ff first
#define ss second
#define sz(x) (int) (x).size()
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cout << #x << " = " << x << endl
#define int long long

typedef pair<int, int> pi;

const int INF = 0x3f3f3f3f3f3f3f3fll;
const int MAX = 1e5+10;

int n, m;
vector<pi> adj[MAX]; // {vizinho, peso}
bool vis[MAX];
int dist[MAX];

void dijkstra(int s) {
    fill(dist, dist+MAX, INF);
    fill(vis, vis+MAX, false);
    // priority_queue de {distancia, vertice}
    priority_queue<pi, vector<pi>, greater<pi>> pq;
    pq.push({0, s}); dist[s] = 0;
    while (!pq.empty()) {
        auto [d, v] = pq.top(); pq.pop();
        if (vis[v]) continue;
        vis[v] = true;
        for (auto [u, w] : adj[v]) if (dist[u] > dist[v] + w) {
            dist[u] = dist[v] + w;
            pq.push({dist[u], u});
        }
    }
}

signed main(){
    ios_base::sync_with_stdio(0);cin.tie(0);
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int a, b, c; cin >> a >> b >> c; a--, b--;
        adj[a].pb({b, c});
    }
    dijkstra(0);
    for (int i = 0; i < n; i++) {
        cout << dist[i] << " ";
    }
    cout << endl;
}
```

6.22 shortest routes ii

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define pb push_back
#define eb emplace_back
#define int long long

typedef pair<int, int> pi;

const int INF = 0x3f3f3f3f3f3f3f3fll;
const int MAX = 5e2 + 10;

int n, m, q;
int w[MAX][MAX], dist[MAX][MAX];

void initialize() {
    for (int i = 0 ; i < n ; i++) {
        for (int j = 0 ; j < n ; j++) {
            if (i == j) {
                dist[i][j] = 0;
            } else {
                dist[i][j] = INF ;
            }
        }
    }
}
void floyd_warshall() {
    for (int k = 0 ; k < n ; k++) {
        for (int i = 0 ; i < n ; i++) {
            for (int j = 0 ; j < n ; j++) {
                dist[i][j] = min(dist[i][j] , dist[i][k] + dist[k][j]) ;
            }
        }
    }
}

signed main(){
    ios_base::sync_with_stdio(0);cin.tie(0);
    cin >> n >> m >> q;
    initialize();
    for (int i = 0; i < m; i++) {
        int a, b, c; cin >> a >> b >> c; a--, b--;
        w[a][b] = c;
        dist[a][b] = min(dist[a][b], c);
        w[b][a] = c;
        dist[b][a] = min(dist[b][a], c);
    }
    floyd_warshall();
    while (q--) {
        int a, b; cin >> a >> b; a--, b--;
        if (dist[a][b] == INF) {
            cout << -1 << endl;
        } else {
            cout << dist[a][b] << endl;
        }
    }
}
```

7 Classics - Interactive Problems

7.1 hidden integer

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define ff first
#define ss second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define sz(x) (x).size()
#define pb push_back

typedef long long ll;
typedef double ld;
```

```
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;
typedef vector<vl> vvl;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3fll;
const double eps = 1e-9;
const ll MOD = 1e9 + 7;

string check(int m){
    cout << "? "<< m << endl;
    cout.flush();
    string response;
    cin >> response;
    return response;
}

int main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int l=1,r=1e9;
  int ans=0;
  while(l<=r){
      int m = l+(r-l)/2;
      if(check(m)=="YES"){
          l=m+1;
      }else{
          ans=m;
          r=m-1;
      }
  }
  cout << "! " << ans << endl;
  cout.flush();

  return 0;
}
```

7.2 hidden permutation

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define ff first
#define ss second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define sz(x) (x).size()
#define pb push_back

typedef long long ll;
typedef double ld;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;
typedef vector<vl> vvl;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3fll;
const double eps = 1e-9;
const ll MOD = 1e9 + 7;

bool cmp(int x, int y){
    cout << "? " << x << " " << y << endl;
    cout.flush() << endl;
    string resp; cin >> resp;
    return resp=="YES";
}

int main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n; cin >> n;
  vi a(n+1);
  for(int i=1;i<=n;i++) a[i]=i;
  stable_sort(a.begin()+1, a.end(), cmp);
  vi ans(n+1);
  for(int i=1;i<=n;i++) ans[a[i]]=i;

  cout << "!";
  for(int i=1;i<=n;i++){
      cout << " " << ans[i];
  }
  cout << endl;
  cout.flush();

  return 0;
}
```

8 Classics - Introductory Problems

8.1 apple division

```
def all_subsets(acc=0, i=0):
    global minimo
    if i == len(apples):
        minimo = min(minimo, abs(total - 2*acc))
        return

    all_subsets(acc + apples[i], i+1)
    all_subsets(acc, i+1)

n = int(input())
apples = [int(x) for x in input().split()]
total = sum(apples)
minimo = float('inf')

all_subsets()
print(minimo)
```

8.2 bit strings

```
#include <bits/stdc++.h>
using namespace std;

#define int long long

const int MAXN = 1e6 + 10;
const int MOD = 1e9 + 7;

int dp[MAXN];

int solve(int n) {
    if (n == 0) return dp[0] = 1;
    if (n == 1) return dp[1] = 2;
    return dp[n] = (2 * solve(n-1)) % MOD;
}

signed main() {
        int n; cin >> n;
    cout << solve(n) << endl;
}
```

8.3 chessboard and queens

```
count = 0

def solve():
    board = []
    for _ in range(8):
        board.append(list(input()))

    eight_queens(0, board)

def in_board(i, j):
    return 0 <= i <= 7 and 0 <= j <= 7

def attacks_another_queen(i, j, board):
    for k in range(i+1):
        if board[k][j] == 'Q':
            return True

        if in_board(i-k, j-k) and board[i-k][j-k] == 'Q':
            return True

        if in_board(i-k, j+k) and board[i-k][j+k] == 'Q':
            return True

    return False

def eight_queens(i, board):
    global count

    if i == 8:
        count += 1
        return

    for j in range(8):
        if board[i][j] == '*':
            continue

        if attacks_another_queen(i, j, board):
            continue

        board[i][j] = 'Q'
        eight_queens(i+1, board)
        board[i][j] = '.'

solve()
print(count)
```

8.4 coin piles

```
#include <bits/stdc++.h>
using namespace std;

#define _ ios_base::sync_with_stdio(0);cin.tie(0);
#define endl '\n'
#define ff first
#define ss second
#define pb push_back
#define all(x) (x).begin(), (x).end()

#define dbg(x) cout << #x << " = " << x << endl

typedef long long ll;
typedef pair<int, int> ii;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

int main(){ _
    int n; cin >> n;
    while (n--) {
        ll a, b; cin >> a >> b;
        ll eq1 = 2*a - b, eq2 = 2*b - a;
        if (eq1 >= 0 && eq2 >= 0 && (eq1 % 3 == 0) && (eq2 % 3 == 0)) {
            cout << "YES" << endl;
        } else {
            cout << "NO" << endl;
        }
    }

    return 0;
}
```

8.5 creating strings

```
#include <bits/stdc++.h>
using namespace std;

#define _ ios_base::sync_with_stdio(0);cin.tie(0);
#define endl '\n'
#define ff first
#define ss second
#define pb push_back
#define all(x) (x).begin(), (x).end()

#define dbg(x) cout << #x << " = " << x << endl

typedef long long ll;
typedef pair<int, int> ii;

const ll P = 1000000007;
const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

int main(){ _

    string s; cin >> s;
    sort(all(s));

    set<string> permutations;
    do {
        permutations.insert(s);
    } while(next_permutation(all(s)));

    cout << permutations.size() << endl;
    for (auto x : permutations) cout << x << endl;

    return 0;
}
```

8.6 digit queries

```
from math import ceil

def solve():
    t = int(input())

    for _ in range(t):
        n = int(input())
        k = 0

        while n - 9*(10**k)*(k+1) >= 0:
            n -= 9*(10**k)*(k+1)
            k += 1

        start = 10 ** k - 1
        jump = n // (k + 1)

        if n % (k + 1) != 0:
            jump += 1

        number = str(start + jump)
        print(number[(n % (k + 1)) - 1])

solve()
```

8.7 gray code

```cpp
#include <bits/stdc++.h>
using namespace std;

#define _ ios_base::sync_with_stdio(0);cin.tie(0);
#define endl '\n'
#define ff first
#define ss second
#define pb push_back
#define all(x) (x).begin(), (x).end()

#define dbg(x) cout << #x << " = " << x << endl

typedef long long ll;
typedef pair<int, int> ii;

const ll P = 1000000007;
const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;


string to_binary(uint16_t x, int size) {
    string s;
    for (int i = size - 1; i >= 0; i--) s += ((x >> i) & 1) ? '1' : '0';
    return s;
}

int main(){ _

    int n; cin >> n;
    uint16_t gray;
    for (int i = 0; i < (1 << n); i++) { // (1 << n) == 2^n
        gray = i ^ (i >> 1);
        cout << to_binary(gray, n) << endl;
    }

    return 0;
}

// 10
// 010
// 11

// 11
//  11
// 10
```

8.8 increasing array

```cpp
#include <bits/stdc++.h>
using namespace std;

#define _ ios_base::sync_with_stdio(0);cin.tie(0);
#define endl '\n'
#define ff first
#define ss second
#define pb push_back
#define all(x) (x).begin(), (x).end()

#define dbg(x) cout << #x << " = " << x << endl

typedef long long ll;
typedef pair<int, int> ii;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

int main(){ _

    int n; cin >> n;
    vector<ll> arr(n); for (int i = 0; i < n; i++) cin >> arr[i];

    ll ans = 0;
    for (int i = 1; i < n; i++) {
        if (arr[i - 1] > arr[i]) {
            ll diff = arr[i - 1] -  arr[i];
            ans += diff;
            arr[i] += diff;
        }
    }

    cout << ans << endl;

    return 0;
}
```

8.9 missing number

```cpp
#include <bits/stdc++.h>
using namespace std;

#define _ ios_base::sync_with_stdio(0);cin.tie(0);
#define endl '\n'
#define ff first
#define ss second
#define pb push_back
#define all(x) (x).begin(), (x).end()

#define dbg(x) cout << #x << " = " << x << endl

typedef long long ll;
typedef pair<int, int> ii;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

int main(){ _

    int n; cin >> n;
    vector<bool> arr(n, false);
    for (int i = 0; i < n - 1; i++) {
        int temp; cin >> temp;
        arr[temp - 1] = true;
    }

    for (int i = 0; i < n; i++) {
        if (!arr[i]) cout << i + 1 << endl;
    }

    return 0;
}
```

8.10 number spiral

```cpp
#include <bits/stdc++.h>
using namespace std;

#define _ ios_base::sync_with_stdio(0);cin.tie(0);
#define endl '\n'
#define ff first
#define ss second
#define pb push_back
#define all(x) (x).begin(), (x).end()

#define dbg(x) cout << #x << " = " << x << endl

typedef long long ll;
typedef pair<int, int> ii;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

int main(){ _
```

```
    int t; cin >> t;
    while (t--) {
        ll y, x; cin >> y >> x;

        ll area, ans;
        if (y > x) {
            area = (y - 1) * (y - 1);
            if (y % 2 == 0) ans = area + (2 * y - x);
            else ans = area + x;
        } else {
            area = (x - 1) * (x - 1);
            if (x % 2 == 0) ans = area + y;
            else ans = area + (2 * x - y);
        }

        cout << ans << endl;
    }

    return 0;
}
```

8.11 palindrome reorder

```
#include <bits/stdc++.h>
using namespace std;

#define _ ios_base::sync_with_stdio(0);cin.tie(0);
#define endl '\n'
#define ff first
#define ss second
#define pb push_back
#define all(x) (x).begin(), (x).end()

#define dbg(x) cout << #x << " = " << x << endl

typedef long long ll;
typedef pair<int, int> ii;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

int main(){ _

    string s; cin >> s;
    map<char, int> mp;
    for (auto c : s) mp[c]++;

    // Verifica se existe solucao para a string s
    if (s.length() % 2 == 0) {
        for (const auto& x : mp) {
            if (x.ss % 2 != 0) {
                cout << "NO SOLUTION" << endl;
                return 0;
            }
        }
    } else {
        bool flag = false;
        for (const auto& x : mp) {
            if (x.ss % 2 != 0 && flag) {
                cout << "NO SOLUTION" << endl;
                return 0;
            } else if (x.ss % 2 != 0) {
                flag = true;
            }
        }
    }

    string half, mid;
    for (const auto& x : mp) {
        if (x.ss % 2 == 0) {
            for (int i = 0; i < x.ss / 2; i++) half += x.ff;
        } else {
            for (int i = 0; i < x.ss; i++) mid += x.ff;
        }
    }

    cout << half << mid;
    for (int i = half.size() - 1; i >= 0; i--) cout << half[i];
    cout << endl;

    return 0;
}
```

8.12 permutations

```
#include <bits/stdc++.h>
using namespace std;

#define _ ios_base::sync_with_stdio(0);cin.tie(0);
#define endl '\n'
#define ff first
#define ss second
#define pb push_back
#define all(x) (x).begin(), (x).end()

#define dbg(x) cout << #x << " = " << x << endl

typedef long long ll;
typedef pair<int, int> ii;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

int main(){ _

    int n; cin >> n;

    if (n > 1 && n < 4) cout << "NO SOLUTION" << endl;
    else {
        for (int i = 1; i <= n; i++) {
            if (i % 2 == 0) cout << i << " ";
        }
        for (int i = 1; i <= n; i++) {
            if (i % 2 != 0) cout << i << " ";
        }
        cout << endl;
    }

    return 0;
}
```

8.13 repetitions

```
#include <bits/stdc++.h>
using namespace std;

#define _ ios_base::sync_with_stdio(0);cin.tie(0);
#define endl '\n'
#define ff first
#define ss second
#define pb push_back
#define all(x) (x).begin(), (x).end()

#define dbg(x) cout << #x << " = " << x << endl

typedef long long ll;
typedef pair<int, int> ii;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

int main(){ _

    string s; cin >> s;

    int ans = 1, count = 1;
```

```
    for (int i = 1; i < s.size(); i++) {
        if (s[i - 1] != s[i]) count = 1;
        else count++;
        ans = max(ans, count);
    }

    cout << ans << endl;

    return 0;
}
```

8.14 tower of hanoi

```
def solve():
    n = int(input())
    print(2**n - 1)
    hanoi(1, 3, n)

def hanoi(start, end, n):
    if n == 1:
        print(start, end)
        return

    helper = 6 - (start + end)
    hanoi(start, helper, n-1)
    print(start, end)
    hanoi(helper, end, n-1)

solve()
```

8.15 trailing zeros

```
#include <bits/stdc++.h>
using namespace std;

#define _ ios_base::sync_with_stdio(0);cin.tie(0);
#define endl '\n'
#define ff first
#define ss second
#define pb push_back
#define all(x) (x).begin(), (x).end()

#define dbg(x) cout << #x << " = " << x << endl

typedef long long ll;
typedef pair<int, int> ii;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

ll solve(int n) {
    if (n == 0) return 0;
    else return (n / 5L) + solve(n / 5L);
}

int main(){ _

    ll n; cin >> n;
    cout << solve(n) << endl;

    return 0;
}
```

8.16 two knights

```
#include <bits/stdc++.h>
using namespace std;

#define _ ios_base::sync_with_stdio(0);cin.tie(0);
#define endl '\n'
#define ff first
#define ss second
#define pb push_back
#define all(x) (x).begin(), (x).end()

#define dbg(x) cout << #x << " = " << x << endl

typedef long long ll;
typedef pair<int, int> ii;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

int main(){ _

    ll n; cin >> n;
    for (ll k = 1; k <= n; k++) {
        // (k   * (k   - 1)) / 2
        ll total = ((k * k) * ((k * k) - 1)) / 2;
        ll attacking_ways = 4 * (k - 1) * (k - 2); // 3x2 or 2x3
        ll ans = total - attacking_ways;
        cout << ans << endl;
    }
    return 0;
}
```

8.17 two sets

```
#include <bits/stdc++.h>
using namespace std;

#define _ ios_base::sync_with_stdio(0);cin.tie(0);
#define endl '\n'
#define ff first
#define ss second
#define pb push_back
#define all(x) (x).begin(), (x).end()

#define dbg(x) cout << #x << " = " << x << endl

typedef long long ll;
typedef pair<int, int> ii;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

int main(){ _

    ll n; cin >> n;

    ll total_sum = (n * (n + 1L)) / 2;
    if (total_sum % 2 != 0) cout << "NO" << endl;
    else {
        cout << "YES" << endl;
        vector<int> a, b;
        vector<bool> visited(n + 1, false);
        ll sum_a = 0, current = n;
        while (sum_a < total_sum / 2) {
            ll remain = total_sum / 2 - sum_a;
            if (remain > current) {
                a.pb(current);
                sum_a += current;
                visited[current] = true;
                current--;
            } else {
                a.pb(remain);
                visited[remain] = true;
                break;
            }
        }

        for (int i = 1; i <= n; i++) {
            if (!visited[i]) b.pb(i);
        }
```

```
        // print a
        sort(all(a));
        cout << a.size() << endl;
        for (auto x : a) cout << x << " ";
        cout << endl;
        // print b
        cout << b.size() << endl;
        for (auto x : b) cout << x << " ";
        cout << endl;
    }

    return 0;
}
```

8.18 weird algorithm

```
#include <bits/stdc++.h>
using namespace std;

#define _ ios_base::sync_with_stdio(0);cin.tie(0);
#define endl '\n'
#define ff first
#define ss second
#define pb push_back
#define all(x) (x).begin(), (x).end()

#define dbg(x) cout << #x << " = " << x << endl

typedef long long ll;
typedef pair<int, int> ii;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

int main(){ _

    ll n; cin >> n;
    cout << n << " ";
    while (n != 1) {
        if (n % 2 == 0) n = n / 2;
        else n = 3*n + 1;
        cout << n << " ";
    }
    cout << endl;

    return 0;
}
```

9 Classics - Mathematics

9.1 binomial coefficients

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back
#define int long long

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;

int const N = 1e6+1;
int fat[N];

ll modpow(ll base, ll exp) {
    ll res = 1;
    base %= MOD;
    while (exp > 0) {
        if (exp % 2 == 1) res = (res * base) % MOD;
        base = (base * base) % MOD;
        exp /= 2;
    }
    return res;
}

int inv_modular(int n){
  return modpow(n,MOD-2);
}

void fatorial(){
  fat[0]=1;
  for(int i=1;i<N;i++){
    fat[i] = (fat[i-1]*i)%MOD;
  }
}


signed main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  fatorial();
  int t; cin >> t;
  while(t--){
    int a,b; cin >> a >> b;
    int c=fat[a-b];
    a=fat[a];
    b=fat[b];
    int ans = a;
    ans = (ans*inv_modular(b))%MOD;
    ans = (ans*inv_modular(c))%MOD;
    cout << ans << endl;
  }

  return 0;
}
```

9.2 common divisors

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back
#define int long long

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;
```

```
const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;

const int N = 1e6;

signed main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n;
  cin >> n;
  int mx = 0;
  vi a(n);
  vi freq(N + 1, 0);
  for (int i = 0; i < n; i++) {
    cin >> a[i];
    freq[a[i]]++;
  }

  for(int i=N;i>=1;i--){
    int div=0;
    for(int j=i;j<=N;j+=i){
      div += freq[j];
    }

    if(div >1){
      cout << i << endl;
      break;
    }
  }


  return 0;
}
```

9.3 counting divisors

```
#include <bits/stdc++.h>
using namespace std;
int divisores(int x) {
    int ans = 0;
    for (int a = 1; a*a <= x; a++) {
        if (x % a == 0) {
            int b = x / a;
            ans++;
            if (a!=b) ans++;
        }
    }
    return ans;
}
signed main() {
    int n; cin >> n;
    for (int i = 0; i < n; i++) {
        int x; cin >> x;
        cout << divisores(x) << endl;
    }
}
```

9.4 exponentiation

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back
#define int long long

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;

ll modpow(ll base, ll exp, ll mod) {
    ll res = 1;
    base %= mod;
    while (exp > 0) {
        if (exp % 2 == 1) res = (res * base) % mod;

        base = (base * base) % mod;

        exp /= 2;
    }
    return res;
}

signed main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n; cin >> n;
  while(n--){
    int a,b; cin >> a >> b;
    if(a==0&&b==0){
      cout << 1 << endl;
      continue;
    }
    cout << modpow(a,b,MOD) << endl;
  }
  return 0;
}
```

9.5 nim game i

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'

const int MAXN = 2e5 + 10;

int v[MAXN];

signed main() {
        ios_base::sync_with_stdio(false); cin.tie(0);
    int t; cin >> t;
    while (t--) {
        int n; cin >> n;
        for (int i = 0; i < n; i++) {
            cin >> v[i];
        }
        // caso o xor de todos os x[i] == 0 -> Adversario ganha. Caso contrario,
             Breno ganha.
        int xr = 0;
        for (int i = 0; i < n; i++) {
            xr ^= v[i];
        }
        if (xr != 0) {
            cout << "first" << endl;
        } else {
            cout << "second" << endl;
        }
    }
}
```

9.6 nim game ii

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define ff first
#define ss second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define sz(x) (x).size()
#define pb push_back

typedef long long ll;
typedef double ld;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;
typedef vector<vl> vvl;
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double eps = 1e-9;
const ll MOD = 1e9 + 7;

void solve(){
  int n; cin >> n;
  int sum=0;
  for(int i=0;i<n;i++){
    int x; cin>> x;
    sum ^= (x%4);
  }

   cout << (sum > 0 ? "first" : "second") << endl;
}

int main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int t; cin >> t;
  while(t--)solve();

  return 0;
}
```

9.7 stick game

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define ff first
#define ss second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define sz(x) (x).size()
#define pb push_back

typedef long long ll;
typedef double ld;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;
typedef vector<vl> vvl;
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double eps = 1e-9;
const ll MOD = 1e9 + 7;

int main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n,k; cin >> n >> k;
  vi a(k);
  for(auto &x:a) cin >> x;

  vb dp(n+1,false);
  for(int i=0;i<k;i++) dp[a[i]]=true;

  for(int i=1;i<=n;i++){
    bool flag=false;
    for(int j=0;j<k;j++){
      if(i-a[j] > 0){
        if(!dp[i-a[j]]) flag=true;
      }
    }
    if(flag) dp[i]=true;
  }

  for(int i=1;i<=n;i++){
    if(dp[i]) cout << 'W';
    else cout << 'L';
  }

  return 0;
}
```

9.8 sum of divisors

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back
#define int long long

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;

int inv(int a) {
    return a <= 1 ? a : MOD - (long long)(MOD/a) * inv(MOD % a) % MOD;
}

signed main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n;
  cin >> n;
  int r=0;
```

```
  int ans=0;
  int inv2 = inv(2);
  for(int i=1;i<=n;i=r+1){
    int d = n/i;
    r = n/d;
    int pa = ((((i+r)%MOD)
    *((r-i+1)%MOD))
    %MOD)
    *inv2
    %MOD;
    int cont = (pa*(d%MOD))%MOD;
    ans = (ans+cont)%MOD;
  }
  cout << ans << endl;
  return 0;
}
```

10 Classics - Range Queries

10.1 distinct values queries

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back
#define int long long

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;

template<class T> struct SegmentTree {
  const T ID{};
  T cmb(T a, T b) { return a + b; }
  int N = 1; vector<T> seg;
  SegmentTree(int _N) {
    while (N < _N) N *= 2;
    seg.assign(2*N, ID);
  }
  void update(int p, T val) { // set val at position p
    seg[p += N] = val;
    for (p /= 2; p > 0; p /= 2) {
      seg[p] = cmb(seg[2*p], seg[2*p+1]);
    }
  }
  T query(int l, int r) { // zero-indexed, inclusive
    T lf = ID, rf = ID;
    for (l += N, r += N + 1; l < r; l /= 2, r /= 2) {
      if (l & 1) lf = cmb(lf, seg[l++]);
      if (r & 1) rf = cmb(seg[--r], rf);
    }
    return cmb(lf, rf);
  }
};

signed main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n, q;
  cin >> n >> q;
  vi a(n),ans(q);
  for(auto &x:a)cin >>x;

  map<int, vpii> Queries;
  for(int i=0;i<q;i++){
    int l,r; cin >> l >> r;
    Queries[r-1].pb({l-1,i});
  }

  map<int,int> lstOc;
  SegmentTree<int> st(n);
  for(int i=0;i<n;i++){
    if(lstOc.count(a[i])){
      st.update(lstOc[a[i]],0);
    }
    st.update(i,1);
    lstOc[a[i]]=i;
    for(auto [l,ind]:Queries[i]){
      ans[ind]=st.query(l,i);
    }
  }

  for(auto i : ans) cout << i << endl;

  return 0;
}
```

10.2 dynamic range sum queries

```
// Source: https://usaco.guide/general/io

#include <bits/stdc++.h>
using namespace std;
#define int long long

const int N = 2e5+5;
int a[N];
int tree[N];

void build(int idx,int value){
    while(idx<=N){
        tree[idx]+=value;
        idx += idx & (-idx);
    }
}

int query(int idx){
    int s=0;
    while(idx>0){
        s+=tree[idx];
        idx -= idx&(-idx);
    }
    return s;
}

int query(int a, int b){
    if(a>b) return 0;
    return query(b) - query(a-1);
}


signed main() {
    int n,q;
    cin >> n >> q;
    memset(a,0,sizeof(a));
    for(int i=1;i<=n;i++) {
        cin >> a[i];
        build(i,a[i]);
    }

    while(q--){
        int op; cin >> op;
```

```
        if(op==1){
            int u,value; cin >> u >> value;
            int tmp=value;
            value = value-a[u];
            a[u] = tmp;
            build(u,value);
        }else{
            int a,b; cin >> a >> b;
            cout << query(a,b) << endl;
        }
    }
}
```

10.3 forest queries

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back
#define int long long

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;

signed main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);

  int n,q; cin >> n >> q;
  vvi pref(n+1, vi(n+1,0));
  for(int i=1;i<=n;i++){
    for(int j=1;j<=n;j++){
      char c; cin >> c;
      int val = (c=='*'?1:0);
      pref[i][j] = val + pref[i-1][j] + pref[i][j-1] -pref[i-1][j-1];
    }
  }

  while(q--){
    int x1,y1,x2,y2;
    cin >> x1 >> y1 >> x2 >> y2;
    cout << pref[x2][y2] - pref[x1-1][y2] - pref[x2][y1-1] + pref[x1-1][y1-1] <<
         endl;
  }

  return 0;
}
```

10.4 prefix sum queries

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back
#define int long long

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;

struct Node{
  int prefix,sum;
};

const int N = 2e5+10;

Node seg[N*4];
int v[N];

Node combina(Node a, Node b){
  Node resp;
  resp.prefix = max({a.prefix,a.sum+b.prefix, a.sum+b.sum});
  resp.sum = a.sum+b.sum;
  return resp;
}

Node build(int p, int l, int r){
  if(l==r){
    return seg[p]={v[l],v[l]};
  }
  int m=(l+r)/2;
  return seg[p]=combina(build(2*p,l,m),build(2*p+1,m+1,r));
}

Node update(int i, int x, int p, int l, int r){
  if(i < l or i > r) return seg[p];
  if(l==r){
    v[l]=x;
    return seg[p]={x,x};
  }
  int m=(l+r)/2;
  return seg[p]=combina(update(i,x,2*p,l,m),update(i,x,2*p+1,m+1,r));
}

Node query(int ql, int qr, int p, int l, int r){
  if(ql > r or qr < l) return {0,0};
  if(ql <= l and r <= qr) return seg[p];

  int m=(l+r)/2;
  return combina(query(ql,qr,2*p,l,m),query(ql,qr,2*p+1,m+1,r));
}

signed main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n,q; cin >> n >> q;
  for(int i=0;i<n;i++) cin >> v[i];

  build(1,0,n-1);
  while(q--){
    int t; cin >> t;
    if(t==1){
      int i,x; cin >> i >> x;
      i--;
      update(i,x,1,0,n-1);
    }else{
      int l,r; cin >> l >> r;
      l--,r--;
```

```
      cout << max(0LL,query(l,r,1,0,n-1).prefix) << endl;
    }
  }
  return 0;
}
```

10.5 range queries and copies

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back
#define int long long

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;

template <typename T> struct PersistentSegTree {
  struct Node {
    T sum;
    int l, r; // Indices para o filho esquerdo e direito no vector 'tree'
    Node(T sum = 0, int l = 0, int r = 0) : sum(sum), l(l), r(r) {}
  };

  int n;
  vector<Node> tree;
  vector<int> roots; // Armazena o indice da raiz de cada versao
  T neutro = 0;      // Elemento neutro para a soma

  // Construtor
  PersistentSegTree(int n) : n(n) {
    // O indice 0 sera o nosso no "nulo" ou vazio
    tree.push_back(Node(neutro, 0, 0));
    roots.push_back(0); // A versao 0 aponta para o no nulo inicialmente
  }

  T combina(T a, T b) { return a + b; }

  int clone(int node_idx) {
    tree.push_back(tree[node_idx]);
    return tree.size() - 1;
  }

  int copy_version(int version_id) {
    roots.push_back(roots[version_id]);
    return roots.size() - 1;
  }

  // Build interno (Constroi a versao inicial baseada em um array)
  int build(int l, int r, const vector<T> &arr) {
    int node = tree.size();
    tree.push_back(Node()); // Cria um novo no

    if (l == r) {
      tree[node].sum = arr[l];
      return node;
    }
    int mid = l + (r - l) / 2;
    tree[node].l = build(l, mid, arr);
    tree[node].r = build(mid + 1, r, arr);
    tree[node].sum = combina(tree[tree[node].l].sum, tree[tree[node].r].sum);

    return node;
  }

  // Update interno (Retorna o indice da nova raiz)
  int update(int curr_root, int l, int r, int pos,
             T val) {
    int node = clone(curr_root);
    // 3. Chegamos na folha: modifica in-place e RETORNA O INDICE
    if (l == r) {
      tree[node].sum = val; // Altere para '=' se for substituicao
      return node;
    }

    int mid = l + (r - l) / 2;

    if (pos <= mid) {
      // O filho esquerdo do clone recebe o novo caminho atualizado
      tree[node].l = update(tree[node].l, l, mid, pos, val);
    } else {
      // O filho direito do clone recebe o novo caminho atualizado
      tree[node].r = update(tree[node].r, mid + 1, r, pos, val);
    }

    // Recalcula a soma usando os filhos
    tree[node].sum =
        combina(tree[tree[node].l].sum, tree[tree[node].r].sum);

    // Retorna o indice do no que acabamos de modificar
    return node;
  }

  // Query interna no intervalo [ql, qr] em uma raiz especifica
  T query(int node, int l, int r, int ql, int qr) {
    if (node == 0 || ql > r || qr < l)
      return neutro;
    if (ql <= l && r <= qr)
      return tree[node].sum;

    int mid = l + (r - l) / 2;
    return combina(query(tree[node].l, l, mid, ql, qr),
                   query(tree[node].r, mid + 1, r, ql, qr));
  }

  // --- FUNCOES PARA USAR NA MAIN ---

  // Constroi a versao 0 a partir de um array.
  // Retorna o indice da versao criada (sempre sera 1 se chamado logo apos
  // instanciar)
  int build(const vector<T> &arr) {
    int root_idx = build(0, n - 1, arr);
    roots.push_back(root_idx);
    return roots.size() - 1; // Retorna o ID da versao recem-criada
  }

  // Atualiza um elemento partindo de uma versao base.
  // Retorna o ID da nova versao criada.
  void update(int base_version, int pos, T val) {
    roots[base_version] = update(roots[base_version], 0, n - 1, pos, val);
  }

  // Consulta no intervalo [l, r] de uma versao especifica
  T query(int version, int l, int r) {
    return query(roots[version], 0, n - 1, l, r);
  }
};

signed main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n, q;
  cin >> n >> q;
  vi a(n);
  for (auto &x : a)
```

```
    cin >> x;
  PersistentSegTree<int> st(n);
  st.build(a);
  while (q--) {
    int t;
    cin >> t;
    int k;
    cin >> k;
    if (t == 1) {
      int i, x;
      cin >> i >> x;
      i--;
      st.update(k, i, x);
    } else if (t == 2) {
      int l, r;
      cin >> l >> r;
      l--, r--;
      cout << st.query(k, l, r) << endl;
    } else {
      st.copy_version(k);
    }
  }

  return 0;
}
```

10.6 range update queries

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back
#define int long long

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;

template <class T> struct BIT {
  int n;
  vector<T> bit;

  BIT(int n) : n(n), bit(n + 1, 0) {}

  void add(int i, T val) {
    for (++i; i <= n; i += i & -i)
      bit[i] += val;
  }

  void range_add(int l, int r, T val) {
    add(l, val);
    if (r + 1 <= n) add(r + 1, -val);
  }

  T point_query(int idx) {
    T resp = 0;
    for (++idx; idx > 0; idx -= idx & -idx)
      resp += bit[idx];
    return resp;
  }
};

signed main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n, q;
  cin >> n >> q;

  BIT<int> bt(n + 1);
  vi a(n+1,0);
  for (int i = 1; i <= n; i++) {
    cin >> a[i];
  }

  for(int i=1;i<=n;i++){
    bt.add(i,a[i]-a[i-1]);
  }

  while (q--) {
    int t;
    cin >> t;
    if (t == 1) {
      int l, r, val;
      cin >> l >> r >> val;
      bt.range_add(l, r, val);
    } else {
      int i;
      cin >> i;
      cout << bt.point_query(i) << endl;
    }
  }

  return 0;
}
```

10.7 salary queries

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back
#define int long long

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;

const int MINV=1;
const int MAXV=1e9+5;

struct Node {
  int sum;
  Node *l, *r;
  Node(): sum(0), l(nullptr), r(nullptr){}
};

void update(Node* &node, int l, int r, int idx, int val){
  if(!node) node = new Node();
```

```
  if(l==r){
    node->sum += val;
    return;
  }

  int m= (l+r)/2;
  if(idx<=m){
    update(node->l, l,m,idx,val);
  }else update(node->r, m+1,r,idx,val);

  int leftSum = node->l ? node->l->sum : 0;
  int rightSum = node->r ? node->r->sum : 0;

  node->sum = leftSum + rightSum;
}

int query(Node* node, int l, int r, int ql, int qr){
  if(!node || ql > r || qr <l ) return 0;
  if (ql <= l && r <= qr) return node->sum;

  int m =(l+r)/2;
  return query(node->l, l,m,ql,qr)+query(node->r, m+1,r,ql,qr);
}

signed main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n,q; cin >> n >> q;

  vi v(n);
  Node* root = nullptr;

  for(int i=0;i<n;i++){
    cin >> v[i];
    update(root, MINV, MAXV, v[i],+1);
  }

  while(q--){
    char t; cin >> t;

    if(t=='?'){
      int a,b; cin >> a >> b;
      cout << query(root, MINV, MAXV, a,b) << endl;
    }else{
      int k,x; cin >> k >> x;
      k--;

      update(root, MINV, MAXV, v[k],-1);

      v[k]=x;
      update(root, MINV, MAXV, v[k],+1);
    }
  }

  return 0;
}
```

10.8 static range minimum queries

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define all(x) (x).begin(), (x).end()
#define pb push_back
#define int long long

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;

int n;
vector<int> a;
vector<vector<int>> st;
vector<int> lg;

void build() {
  int K = __lg(n) + 1;
  st.assign(K, vector<int>(n));
  st[0] = a;

  for(int k = 1; k < K; k++) {
    for(int i = 0; i + (1 << k) <= n; i++) {
      st[k][i] = min(st[k-1][i], st[k-1][i + (1 << (k-1))]);
    }
  }

  lg.assign(n+1, 0);
  for(int i = 2; i <= n; i++)
    lg[i] = lg[i/2] + 1;
}

int query(int l, int r) {
  int k = lg[r - l + 1];
  return min(st[k][l], st[k][r - (1 << k) + 1]);
}

signed main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  int q;
  cin >> n >> q;

  a.resize(n);
  for(auto &x : a) cin >> x;

  build();

  while(q--){
    int x, b;
    cin >> x >> b;
    x--; b--;
    cout << query(x, b) << '\n';
  }
}
```

10.9 static range sum queries

```
#include <bits/stdc++.h>
using namespace std;

int main() {

    int n, q; cin >> n >> q;
    vector<int> v(n);
    for(auto& x : v)
        cin >> x;

    vector<long long> s(n+1);
    s[0] = 0;

    for(int i = 1; i <= n; i++){
        s[i] = v[i-1] + s[i-1];
    }
```

```
    while(q--){
        int a, b;
        cin >> a >> b;
        cout << s[b] - s[a-1] << endl;
    }

    return 0;
}
```

11 Classics - Sliding Window Problems

11.1 sliding window distinct values

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef pair<int, int> pi;
#define int ll
#define ff first
#define ss second
#define endl '\n'
#define pb push_back
#define sz(x) (int) (x).size()
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cout << #x << " = " << x << endl
#define uniq(v) sort(all(v)); v.erase(unique(all(v)), v.end())
// const int MOD = 1e9 + 7; // 998244353;
// const int INF = 0x3f3f3f3f;
// const ll LINF = 0x3f3f3f3f3f3f3f3fll;

signed main(){ // O(nlogn)
    ios_base::sync_with_stdio(0);cin.tie(0);
    int n, k; cin >> n >> k;
    vector<int> a(n);
    for (int i = 0;i < n; i++) cin >> a[i];
    map<int, int> freq;
    vector<int> ans; ans.reserve(n-k+1);
    // primeira janela
    for (int i = 0; i < k; i++) {
        freq[a[i]]++;
    }
    ans.pb(sz(freq));

    // demais janelas
    for (int i = k; i < n; i++) {
        // adiciona um elemento
        freq[a[i]]++;

        // remove o primeiro elemento
        freq[a[i-k]]--;

        if (freq[a[i-k]] == 0) {
            freq.erase(a[i-k]);
        }

        // atualiza a respota
        ans.pb(sz(freq));
    }
    for (auto x : ans) cout << x << " ";
    cout << endl;
}
```

11.2 sliding window median

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define ff first
#define ss second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define sz(x) (int)(x).size()
#define pb push_back

typedef long long ll;
typedef double ld;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;
typedef vector<vl> vvl;
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double eps = 1e-9;
const ll MOD = 1e9 + 7;

int n, m;
int a[200010];
multiset<int> up, low;

void ins(int val) {
  int a = *low.rbegin();
  if (a < val) {
    up.insert(val);
    if (sz(up) > m / 2) {
      low.insert(*up.begin());
      up.erase(up.begin());
    }
  } else {
    low.insert(val);
    if (sz(low) > (m + 1) / 2) {
      up.insert(*low.rbegin());
      low.erase(--low.end());
    }
  }
}

void er(int val) {
  if (up.find(val) != up.end())
    up.erase(up.find(val));
  else
    low.erase(low.find(val));
  if (low.empty()) {
    low.insert(*up.begin());
    up.erase(up.begin());
  }
}

int main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  cin >> n >> m;
  for (int i = 0; i < n; i++) cin >> a[i];
  low.insert(a[0]);
  for (int i = 1; i < m; i++) ins(a[i]);
  cout << *low.rbegin() << " ";
  for (int i = m; i < n; i++) {
    if (m == 1) {
      ins(a[i]);
      er(a[i - m]);
    } else {
      er(a[i - m]);
      ins(a[i]);
    }
    cout << *low.rbegin() << " ";
  }
  cout << endl;

  return 0;
}
```

11.3 sliding window minimum

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef pair<int, int> pi;
#define int ll
#define ff first
#define ss second
#define endl '\n'
#define pb push_back
#define sz(x) (int) (x).size()
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cout << #x << " = " << x << endl
#define uniq(v) sort(all(v)); v.erase(unique(all(v)), v.end())
const int MOD = 1e9 + 7; // 998244353;
const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

struct MinQueue {
    deque<pi> dq;
    int added = 0, removed = 0;
    // adiciona x no final da fila
    void push(int x) {
        while (!dq.empty() and x < dq.back().ff) {
            dq.pop_back();
        }
        dq.push_back({x, added});
        added++;
    }
    // remove o elemento do inicio da final
    void pop() {
        if (!dq.empty() and dq.front().ss == removed) {
            dq.pop_front();
        }
        removed++;
    }
    int get_min() {
        return dq.front().ff;
    }
};

int mult(int a, int b, int m) {
    return ((a % m) * (b % m)) % m;
}

int add(int a, int b, int m) {
    return ((a % m) + (b % m)) % m;
}

signed main(){
    ios_base::sync_with_stdio(0);cin.tie(0);
    int n, k; cin >> n >> k;
    int x, a, b, c; cin >> x >> a >> b >> c;
    vector<int> arr(n); arr[0] = x;
    for (int i = 1; i < n; i++) {
        arr[i] = add(mult(a, arr[i-1], c), b, c);
    }
    // for (int i = 0; i < n; i++) cout << arr[i] << " ";
    // cout << endl;
    int ans = 0;
    MinQueue mq;
    int mn = LLONG_MAX;
    // primeira janela
    for (int i = 0; i < k; i++) {
        mn = min(mn, arr[i]);
        mq.push(arr[i]);
    }
    ans ^= mn;

    for (int i = k; i < n; i++) {
        // adiciona novo elemento
        mq.push(arr[i]);
        // remove elemento antigo
        mq.pop();
        // atualiza o minimo
        mn = mq.get_min();
        ans ^= mn;
    }

    cout << ans << endl;
}
```

11.4 sliding window mode

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef pair<int, int> pi;
#define int ll
#define ff first
#define ss second
#define endl '\n'
#define pb push_back
#define sz(x) (int) (x).size()
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cout << #x << " = " << x << endl
#define uniq(v) sort(all(v)); v.erase(unique(all(v)), v.end())
// const int MOD = 1e9 + 7; // 998244353;
// const int INF = 0x3f3f3f3f;
// const ll LINF = 0x3f3f3f3f3f3f3f3fll;

const int MAXN = 2e5 + 10;

vector<int> a;
int freq_mode = -1;
multiset<int> modes[MAXN];
unordered_map<int, int> freq, freq_freq;

void add(int i) {
    freq_freq[freq[a[i]]]--;
    freq[a[i]]++;
    freq_freq[freq[a[i]]]++;

    modes[freq[a[i]]].insert(a[i]);
    auto it = modes[freq[a[i]]-1].find(a[i]);
    if (it != modes[freq[a[i]]-1].end()) {
        modes[freq[a[i]]-1].erase(it);
    }

    if (freq[a[i]] > freq_mode) {
        freq_mode = freq[a[i]];
    }
}

void remove(int i) {
    if ((freq[a[i]] == freq_mode) and (freq_freq[freq_mode] == 1)) {
        freq_mode--;
    }

    freq_freq[freq[a[i]]]--;
    freq[a[i]]--;
    freq_freq[freq[a[i]]]++;

    modes[freq[a[i]]].insert(a[i]);
    auto it = modes[freq[a[i]]+1].find(a[i]);
    if (it != modes[freq[a[i]]+1].end()) {
        modes[freq[a[i]]+1].erase(it);
    }
}

signed main(){ // O(nlogn)
    ios_base::sync_with_stdio(0);cin.tie(0);
    int n, k; cin >> n >> k;
    a = vector<int>(n);
    for (int i = 0;i < n; i++) cin >> a[i];

    // primeira janela
```

```
    for (int i = 0; i < k; i++) add(i);

    cout << *modes[freq_mode].begin() << " ";

    // demais janelas
    for (int i = k; i < n; i++) {
        // adiciona um elemento
        add(i);

        // remove o primeiro elemento
        remove(i-k);

        cout << *modes[freq_mode].begin() << " ";
    }
    cout << endl;
}
```

11.5 sliding window or

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef pair<int, int> pi;
#define int ll
#define ff first
#define ss second
#define endl '\n'
#define pb push_back
#define sz(x) (int) (x).size()
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cout << #x << " = " << x << endl
#define uniq(v) sort(all(v)); v.erase(unique(all(v)), v.end())
// const int MOD = 1e9 + 7; // 998244353;
// const int INF = 0x3f3f3f3f;
// const ll LINF = 0x3f3f3f3f3f3f3f3fll;

// Esse codigo pode ser alterado para min, max, gcd, or, ...
// e mais flexivel que a MinQueue com deque
// Pilha modificada que mantem o minimo
// modifique o op para a operacao que vc quer
int op(int a, int b) { return a | b; }
// int op(int a, int b) { return min(a, b); }
struct AggStack {
    stack<pi> st;
    // adiciona um novo elemento (O(1))
    void push(int x) {
        int cur = st.empty() ? x : op(st.top().ss, x);
        st.push({x, cur});
    }
    // remove o elemento do topo (O(1))
    void pop() { st.pop(); }
    // retorna o valor agregado do op (minimo, or, ...)
    int agg() { return st.top().ss; };
    // checa se a pilha esta vazia
    bool empty() { return st.empty(); }
    // retorna o topo da pilha
    pi top() { return st.top(); }
};
struct AggQueue {
    AggStack in, out;
    // adiciona um elemento na fila (O(1) amortizado)
    void push(int x) { in.push(x); }
    // remove o primeiro elemento (O(1) amortizado)
    void pop() {
        if (out.empty()) {
            while (!in.empty()) {
                auto [x, cur] = in.top();
                in.pop();
                out.push(x);
            }
        }
        out.pop();
    }
    // retorna o valor agregado da fila (O(1))
    int query() {
        if (in.empty()) return out.agg();
        if (out.empty()) return in.agg();
        return op(in.agg(), out.agg());
    }
};

int mult(int a, int b, int m) {
    return ((a % m) * (b % m)) % m;
}

int add(int a, int b, int m) {
    return ((a % m) + (b % m)) % m;
}

signed main(){
    ios_base::sync_with_stdio(0);cin.tie(0);
    int n, k; cin >> n >> k;
    int x, a, b, c; cin >> x >> a >> b >> c;
    vector<int> arr(n); arr[0] = x;
    for (int i = 1; i < n; i++) {
        arr[i] = add(mult(a, arr[i-1], c), b, c);
    }
    int ans = 0, cur = 0;

    AggQueue q;

    // primeira janela
    for (int i = 0; i < k; i++) {
        cur |= arr[i];
        q.push(arr[i]);
    }
    ans ^= cur;

    // demais janelas
    for (int i = k; i < n; i++) {
        // adiciona um elemento
        q.push(arr[i]);
        // remove o primeiro elemento
        q.pop();
        // atualiza a respota
        ans ^= q.query();
    }

    cout << ans << endl;
}
```

11.6 sliding window sum

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef pair<int, int> pi;
#define int ll
#define ff first
#define ss second
#define endl '\n'
#define pb push_back
#define sz(x) (int) (x).size()
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cout << #x << " = " << x << endl
#define uniq(v) sort(all(v)); v.erase(unique(all(v)), v.end())
const int MOD = 1e9 + 7; // 998244353;
const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

int mult(int a, int b, int m) {
    return ((a % m) * (b % m)) % m;
}

int add(int a, int b, int m) {
    return ((a % m) + (b % m)) % m;
}
```

```
signed main(){
    ios_base::sync_with_stdio(0);cin.tie(0);
    int n, k; cin >> n >> k;
    int x, a, b, c; cin >> x >> a >> b >> c;
    vector<int> arr(n); arr[0] = x;
    for (int i = 1; i < n; i++) {
        arr[i] = add(mult(a, arr[i-1], c), b, c);
    }
    // for (int i = 0; i < n; i++) {
    //     cout << arr[i] << " ";
    // }
    // cout << endl;

    // janelas de tamanho k
    vector<int> win;
    int sum = 0;
    for (int i = 0; i < k; i++) {
        sum += arr[i];
    }
    win.pb(sum);

    for (int i = k; i < n; i++) {
        sum += arr[i]; // adiciona novo elemento
        sum -= arr[i - k]; // remove o primeiro elemento
        win.pb(sum);
    }

    int ans = 0;
    for (auto x : win) ans ^= x;
    cout << ans << endl;
}
```

11.7 sliding window xor

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef pair<int, int> pi;
#define int ll
#define ff first
#define ss second
#define endl '\n'
#define pb push_back
#define sz(x) (int) (x).size()
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cout << #x << " = " << x << endl
#define uniq(v) sort(all(v)); v.erase(unique(all(v)), v.end())
const int MOD = 1e9 + 7; // 998244353;
const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

int mult(int a, int b, int m) {
    return ((a % m) * (b % m)) % m;
}

int add(int a, int b, int m) {
    return ((a % m) + (b % m)) % m;
}

signed main(){
    ios_base::sync_with_stdio(0);cin.tie(0);
    int n, k; cin >> n >> k;
    int x, a, b, c; cin >> x >> a >> b >> c;
    vector<int> arr(n); arr[0] = x;
    for (int i = 1; i < n; i++) {
        arr[i] = add(mult(a, arr[i-1], c), b, c);
    }
    int ans = 0, cur = 0;
    for (int i = 0; i < k; i++) {
        cur ^= arr[i];
    }
    ans ^= cur;
    // e possivel fazer cur ^ a[i] para remover a[i] de cur
    // pois x ^ x = 0 e y ^ 0 = y
    for (int i = k; i < n; i++) {
        // adiciona elemento
        cur ^= arr[i];
        // remove elemento
        cur ^= arr[i-k];
        // atualiza a resposta
        ans ^= cur;
    }
    cout << ans << endl;
}
```

12 Classics - Sorting And Searching

12.1 apartments

```
#include <bits/stdc++.h>
using namespace std;

#define _ ios_base::sync_with_stdio(0);cin.tie(0);
#define endl '\n'
#define pb push_back
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()

#define dbg(x) cout << #x << " = " << x << endl

typedef long long ll;
typedef pair<int, int> pi;

const ll MOD = 1000000007;
const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

int main(){ _

    ll n, m, k; cin >> n >> m >> k;
    vector<ll> a(n), b(m);
    for (auto &x : a) cin >> x;
    for (auto &x : b) cin >> x;

    sort(all(a));
    sort(all(b));

    ll ans = 0;
    int ptrA = 0, ptrB = 0;

    while (ptrA < n && ptrB < m) {
        if (abs(a[ptrA] - b[ptrB]) <= k) {
            ans++;
            ptrA++;
            ptrB++;
        } else if (a[ptrA] > b[ptrB]) {
            ptrB++;
        } else {
            ptrA++;
        }
    }

    cout << ans << endl;

    return 0;
}
```

12.2 collecting numbers

```
#include <iostream>
#include <stdlib.h>
#include <vector>
#include <map>
#include <algorithm>
```

```
using namespace std;
using ll = long long;

int main() {
    ll n;
    cin >> n;

    vector<ll> a(n);
    map<ll, ll> mp;

    for (auto &e : a) cin >> e;

    for (ll i = 0; i < n; i++) {
        mp[a[i]] = i;
    }

    ll ans = 1;
    for (ll i = 1; i < n; i++) {
        if (mp[i + 1] < mp[i]) ans++;
    }

    cout << ans << endl;
    return 0;
}
```

12.3 concert tickets

```
#include <bits/stdc++.h>

using namespace std;

#define _                                                \
  ios_base::sync_with_stdio(0);                          \
  cin.tie(0);
#define endl '\n'
#define f first
#define s second

typedef long long ll;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

int main() {
  _

      int n,
      m;
  cin >> n >> m;
  multiset<ll> tickets;
  for (int i = 0; i < n; i++) {
    ll x;
    cin >> x;
    tickets.insert(x);
  }

  vector<ll> t(m);
  for (int i = 0; i < m; i++) {
    cin >> t[i];
  }



  for (int k = 0; k < m; k++) {
    auto it = tickets.upper_bound(t[k]);
    if(it == tickets.begin()) cout << -1 << endl;
    else {
      it--;
      cout << *it << endl;
      tickets.erase(it);
    }
  }


  return 0;
}
```

12.4 distinct numbers

```
#include <bits/stdc++.h>
using namespace std;

#define _ ios_base::sync_with_stdio(0);cin.tie(0);
#define endl '\n'
#define pb push_back
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()

#define dbg(x) cout << #x << " = " << x << endl

typedef long long ll;
typedef pair<int, int> pi;

const ll MOD = 1000000007;
const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

int main(){ _

    int n; cin >> n;
    set<int> s;
    while (n--) {
        int aux; cin >> aux;
        s.insert(aux);
    }

    cout << s.size() << endl;

    return 0;
}
```

12.5 factory machines

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define pb push_back
#define eb emplace_back
#define ff first
#define ss second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()

#define dbg(x) cout << #x << " = " << x << endl

typedef long long ll;
typedef pair<int, int> pi;

const ll MOD = 1e9 + 7;
const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

const int MAXN = 2e5 + 10;
ll n, t, k[MAXN];

bool possible(ll m) {
    ll prod = 0;
    for (int i = 0; i < n; i++) {
        prod += (m / k[i]);
        if (prod >= t) return true;
    }
    return false;
}

signed main() {
    ios_base::sync_with_stdio(0);cin.tie(0);
```

```
    cin >> n >> t;
    for (int i = 0; i < n; i++) cin >> k[i];
    sort(k, k+n);
    // r -> Uma maquina que demora 1e9 para produzir sozinha 1e9 produtos
    ll l = 0, r = 1e18;
    while (l < r) {
        ll m = l + (r-l)/2;
        if (possible(m)) r = m;
        else l = m + 1;
    }
    cout << l << endl;
}
```

12.6 ferris wheel

```
#include <bits/stdc++.h>
using namespace std;
#define ff first
#define ss second
#define pb push_back
#define int long long
#define endl '\n' //<< flush
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cout << #x << " = " << x << endl
#define uniq(v) sort(all(v)); v.erase(unique(all(v)), v.end())
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
typedef long long ll;
typedef pair<int, int> pi;
// const int MOD = 1e9 + 7; // 998244353;
// const int INF = 0x3f3f3f3f;
// const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const int MAXN = 2e5+5;

void solve() {
    int n, x; cin >> n >> x;
    vector<int> a(n);
    for (int i = 0; i < n; i++) cin >> a[i];
    sort(all(a));
    // cada negocio cabe no maximo 2 pessoas
    // caso o mais leve e o mais pesado caibam juntos, levo os dois
    // caso contrario, levo apenas o mais pesado (pi <= x)
    int i = 0, j = n - 1, ans = 0;
    while (i <= j) {
        if (i == j) {
            ans++;
            break;
        }

        if (a[i] + a[j] <= x) {
            i++;
            j--;
        } else {
            j--;
        }
        ans++;
    }
    cout << ans << endl;
}

signed main() {
    ios_base::sync_with_stdio(0);cin.tie(0);
    int tc = 1;
    // cin >> tc;
    while(tc--) solve();
}
```

12.7 maximum subarray sum

```
#include <bits/stdc++.h>

using namespace std;

#define _                                                       \
  ios_base::sync_with_stdio(0);                                 \
  cin.tie(0);
#define endl '\n'
#define f first
#define s second

typedef long long ll;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

int main() {

  _
    int n; cin >> n;
  vector<long long> a(n);
  for(auto &x:a) cin >> x;

  long long ans=INT_MIN,sum=INT_MIN;
  for(int i=0;i<n;i++){
    sum = max(a[i], sum+a[i]);
    ans = max(ans, sum);
  }
  cout << ans << endl;

      return 0;
}
```

12.8 missing coin sum

```
#include <iostream>
#include <set>
#include <iterator>
#include <vector>
#include <algorithm>

using namespace std;
using ll = long long;

int main() {
    cin.tie(NULL);
    ios_base::sync_with_stdio(false);

    ll n;
    cin >> n;

    vector<ll> coins(n);
    for (auto &e : coins) cin >> e;

    sort(coins.begin(), coins.end());

    ll res = 1;
    for (auto c : coins) {
        if (c > res) break;
        res += c;
    }

    cout << res << endl;

    return 0;
}
```

12.9 movie festival

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define ff first
#define ss second
```

```
int main() {
    ios_base::sync_with_stdio(0);cin.tie(0);

        int n; cin >> n;
        vector<pair<int, int>> ev(n);
        for (int i = 0; i < n; i++) {
            cin >> ev[i].ff >> ev[i].ss;
        }
    sort(ev.begin(), ev.end(), [](pair<int, int> a, pair<int, int> b) {
        return a.ss < b.ss; // Comparando com o tempo de encerramento do evento
    });

    int t = 0, ans = 0;
    for (int i = 0; i < n; i++) {
        if (ev[i].ff >= t) {
            ans++;
            t = ev[i].ss;
        }
    }

    cout << ans << endl;
}
```

12.10 nearest smaller values

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back
#define int long long

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;

signed main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n;
  cin >> n;
  vi a(n);
  for(auto &x:a)cin>>x;
  stack<pii> st;
  vi ans(n, 0);
  for (int i = 0; i <n; i++) {
    while (!st.empty() and st.top().f >= a[i])
      st.pop();
    if (!st.empty())
      ans[i] = st.top().sec;

    st.push({a[i],i+1});
  }

  for (auto i : ans)
    cout << i << " ";
  cout << endl;

  return 0;
}
```

12.11 playlist

```
#include <iostream>
#include <stdlib.h>
#include <vector>
#include <map>
#include <algorithm>
#include <set>
#include <iterator>

using namespace std;
using ll = long long;

vector<ll> songs;
vector<ll> prev_ind;
map<ll, ll> ind;

int main() {
    ll n;
    cin >> n;
    songs.resize(n);
    prev_ind.resize(n, -1);

    for (ll i = 0; i < n; i++) {
        cin >> songs[i];
        if (ind.count(songs[i])) prev_ind[i] = ind[songs[i]];
        ind[songs[i]] = i;
    }

    ll i = 0;
    ll j = 0;
    ll ans = 0;

    while (i < n && j < n) {
        if (prev_ind[j] >= i) {
            i = prev_ind[j] + 1;
        } else {
            ans = max(ans, j - i + 1);
            j++;
        }
    }

    cout << ans << endl;

    return 0;
}
```

12.12 restaurant customers

```
#include <bits/stdc++.h>

using namespace std;

#define _                                                \
  ios_base::sync_with_stdio(0);                          \
  cin.tie(0);
#define endl '\n'
#define f first
#define s second

typedef long long ll;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

int main() {
  _

      int n;
  cin >> n;

  vector<pair<int, int>> a;
  for (int i = 0; i < n; i++) {
```

```
    ll l, r;
    cin >> l >> r;

    a.push_back({l, 1});
    a.push_back({r, -1});
  }

  // definir um evento, nesse caso, entrar e sair do restaurante
  //
  sort(a.begin(), a.end());

  int ans = 0;
  int persons = 0;
  // for(const pair<int, int> &t : a) ou do jeito abaixo
  for (auto [i, j] : a) {
    persons += j;
    ans = max(ans, persons);
  }

  cout << ans << endl;
  return 0;
}
```

12.13 room allocation

```
#include <bits/stdc++.h>
using namespace std;

#define _ ios_base::sync_with_stdio(0);cin.tie(0);
#define endl '\n'
#define pb push_back
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()

#define dbg(x) cout << #x << " = " << x << endl

typedef long long ll;
typedef pair<int, int> pi;
typedef tuple<int, int, int> ti;

const ll MOD = 1000000007;
const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;

#define ENTRADA 0
#define SAIDA 1

int main(){ _

    int n; cin >> n;
    vector<ti> ev;
    for (int i = 0; i < n; i++) {
        int a, b; cin >> a >> b;
        ev.emplace_back(a, ENTRADA, i);
        ev.emplace_back(b, SAIDA, i);
    }

    sort(all(ev));
    int acc = 0, ans = 0;
    for (auto [tempo, tipo, id] : ev) {
        if (tipo == ENTRADA) acc++;
        else acc--;
        ans = max(ans, acc);
    }

    cout << ans << endl;

    vector<int> rooms(n);
    queue<int> pq;
    for (int i = 0; i < ans; i++) pq.push(i+1);
    for (auto [tempo, tipo, id] : ev) {
        if (tipo == ENTRADA) {
            int room = pq.front(); pq.pop();
            rooms[id] = room;
        } else {
            pq.push(rooms[id]);
        }
    }

    for (auto x : rooms) {
        cout << x << " ";
    }
    cout << endl;

    return 0;
}
```

12.14 stick lengths

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
typedef long long ll;


signed main() {
    int n; cin >> n;
    vector<ll> v(n);
    for (int i = 0; i < n; i++) cin >> v[i];
    sort(v.begin(), v.end());
    ll mn = LLONG_MAX;
    if (n % 2 == 1) {
        ll md = v[n/2], acc = 0;
        for (int i = 0; i < n; i++) {
            acc += llabs(v[i] - md);
        };
        mn = min(acc, mn);
    } else {
        ll md1 = v[n/2], md2 = v[(n/2)-1], acc = 0;
        for (int i = 0; i < n; i++) acc += llabs(v[i] - md1);
        mn = min(acc, mn);
        acc = 0;
        for (int i = 0; i < n; i++) acc += llabs(v[i] - md2);
        mn = min(acc, mn);
    }
    cout << mn << endl;
}
```

12.15 subarray sums ii

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back
#define int long long

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;

signed main() {
```

```
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n,x; cin >> n >> x;
  vi a(n);
  for(int i=0;i<n;i++) cin >> a[i];

  vi p(n+1);
  p[0]=0;
  for(int i=1;i<=n;i++){
    p[i] = p[i-1]+a[i-1];
  }

    map<int,int> mp;
    int sum=0;
    int ans=0;
    for(int i=1;i<=n;i++){
        sum += a[i-1];
        if(p[i]==x)ans++;
        int need=p[i]-x;
        ans += mp[need];
        mp[sum]++;
    }


  cout << ans << endl;

  return 0;
}
```

12.16 sum of three values

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back
#define int long long

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;

signed main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n,x; cin >> n >> x;

  vpii a(n);
  for(int i=0;i<n;i++){
    cin >> a[i].f;
    a[i].sec=i+1;
  }


  sort(all(a));
  for(int i=0;i<n;i++){
    int need = x-a[i].f;
    int l=0,r=n-1;
    while(l<r){
        if(a[r].f + a[l].f == need && i!= l && i!=r){
            cout << a[i].sec << " " << a[l].sec << " " << a[r].sec << endl;
            return 0;
        }
        if(a[r].f + a[l].f > need){
            r--;
        }else l++;
    }
 }
    cout << "IMPOSSIBLE" << endl;

  return 0;
}
```

12.17 sum of two values

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back
#define int long long

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;


signed main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);

  map<int,int> mp;
  int n,x; cin >>n>>x;
  vi a(n);
  for(int i=0;i<n;i++){
    cin >> a[i];
  }

  for(int i=0;i<n;i++){
    int cur = x-a[i];
    if(mp[cur]){
        cout << i+1 << " "<< mp[cur] << endl;
        return 0;
    }
    mp[a[i]]=i+1;
  }

  cout << "IMPOSSIBLE" << endl;


  return 0;
}
```

12.18 tasks and deadlines

```
#include <bits/stdc++.h>
using namespace std;
#define ff first
```

```
#define ss second
#define pb push_back
#define int long long
#define endl '\n' //<< flush
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cout << #x << " = " << x << endl
#define uniq(v) sort(all(v)); v.erase(unique(all(v)), v.end())
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
typedef long long ll;
typedef pair<int, int> pi;
// const int MOD = 1e9 + 7; // 998244353;
// const int INF = 0x3f3f3f3f;
// const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const int MAXN = 2e5+5;

void solve() {
    int n; cin >> n;
    vector<pi> v(n);
    for (auto &[dur, dl] : v) {
        cin >> dur >> dl;
    }
    sort(all(v), [](pi a, pi b) {
        if (a.ff != b.ff) return a.ff < b.ff;
        return a.ss < b.ss;
    });
    int cur = 0, ans = 0;
    for (int i = 0; i < n; i++) {
        auto [dur, dl] = v[i];
        cur += dur;
        ans += dl - cur;
    }
    cout << ans << endl;
}

signed main() {
    ios_base::sync_with_stdio(0);cin.tie(0);
    int tc = 1;
    // cin >> tc;
    while(tc--) solve();
}
```

12.19 towers

```
#include <iostream>
#include <stdlib.h>
#include <vector>
#include <map>
#include <algorithm>
#include <set>
#include <iterator>

using namespace std;
using ll = long long;

ll ans = 0;
multiset<ll> m;

int main() {
    ll n;
    cin >> n;

    for (ll i = 0; i < n; i++) {
        ll x; cin >> x;
        auto it = m.upper_bound(x);
        // cout << i << endl;
        if (it != m.end()) {
            // cout << "ERASE: " << *(it) << endl;
            m.erase(it);
        } else {
            ans++;
        }
        // cout << x << endl;
        m.insert(x);
    }
    cout << ans << endl;
    return 0;
}
```

12.20 traffic lights

```
#include <iostream>
#include <set>
#include <iterator>

using namespace std;
using ll = long long;

set<int> positions;
multiset<int> distances;

int main() {
    cin.tie(NULL);
    ios_base::sync_with_stdio(false);

    int x, n;
    cin >> x >> n;

    positions.insert(0);
    positions.insert(x);
    distances.insert(x);

    for (int i = 0; i < n; ++i) {
        int p;
        cin >> p;

        auto it = positions.insert(p).first;
        int prevValue = *(prev(it));
        int nextValue = *(next(it));

        distances.erase(distances.find(nextValue - prevValue));
        distances.insert(p - prevValue);
        distances.insert(nextValue - p);

        cout << *distances.rbegin() << endl;
    }

    return 0;
}
```

13 Classics - String Algorithms

13.1 all palindromes

```
#include <bits/stdc++.h>
#define ll long long
#define nl '\n'

using namespace std;

const ll INF = 1e9;

template<typename T> vector<int> manacher(const T& s) {
        int l = 0, r = -1, n = s.size();
        vector<int> d1(n), d2(n);
        for (int i = 0; i < n; i++) {
                int k = i > r ? 1 : min(d1[l+r-i], r-i);
                while (i+k < n && i-k >= 0 && s[i+k] == s[i-k]) k++;
                d1[i] = k--;
                if (i+k > r) l = i-k, r = i+k;
        }
        l = 0, r = -1;
        for (int i = 0; i < n; i++) {
                int k = i > r ? 0 : min(d2[l+r-i+1], r-i+1); k++;
                while (i+k <= n && i-k >= 0 && s[i+k-1] == s[i-k]) k++;
```

```
                d2[i] = --k;
                if (i+k-1 > r) l = i-k, r = i+k-1;
        }
        vector<int> ret(2*n-1);
        for (int i = 0; i < n; i++) ret[2*i] = 2*d1[i]-1;
        for (int i = 0; i < n-1; i++) ret[2*i+1] = 2*d2[i+1];
        return ret;
}

// verifica se a string s[i..j] eh palindromo
template<typename T> struct palindrome {
        vector<int> man;

        palindrome(const T& s) : man(manacher(s)) {}
        bool query(int i, int j) {
                return man[i+j] >= j-i+1;
        }
};

// tamanho do maior palindromo que termina em cada posicao
template<typename T> vector<int> pal_end(const T& s) {
        vector<int> ret(s.size());
        palindrome<T> p(s);
        ret[0] = 1;
        for (int i = 1; i < s.size(); i++) {
                ret[i] = min(ret[i-1]+2, i+1);
                while (!p.query(i-ret[i]+1, i)) ret[i]--;
        }
        return ret;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    cin >> s;
    int n = (int)s.size();
    vector<int> man = pal_end(s);
    for (ll x : man) cout << x << " ";
    cout << nl;
    return 0;
}
```

13.2 counting patterns

```
#include <bits/stdc++.h>

#define nl '\n'
#define ll long long
#define ull unsigned long long
#define ld long double
#define pii pair<int, int>
#define pll pair<ll, ll>
#define vl vector<ll>
#define vvl vector<vl>
#define vll vector<pll>
#define pb push_back
#define eb emplace_back
#define ff first
#define ss second
#define all(x) (x).begin(), (x).end()
#define allr(x) (x).rbegin(), (x).rend()
#define sz(x) ((int) x.size())

using namespace std;

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

typedef __gnu_pbds::tree<int, __gnu_pbds::null_type, less<int>, __gnu_pbds::
     rb_tree_tag, __gnu_pbds::tree_order_statistics_node_update> ordered_set;

typedef __gnu_pbds::tree<int, __gnu_pbds::null_type, less_equal<int>, __gnu_pbds
     ::rb_tree_tag, __gnu_pbds::tree_order_statistics_node_update>
     ordered_multiset;

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

const ll INF = 1e16;
const ll P = 13;
const ll M = 1e9 + 7;
const ll MAX = 5e3 + 5;
const ll MAXN = 3e5 + 5;
const ll MAXT = 1e2 + 5;
const ll MAXK = 1e6 + 5;
const ll K = 26;

ll n, m, k, q, x, y, s;

vector<vl> g;
//vl vis;
//vector<vector<pll>> g;

//vector<ll> primes;
//ll is_prime[MAXN];
vector<ll> ans;

struct Vertex {
    int next[K];
    int go[K];
    int link = -1;
    int p = -1;
    char pch;
    vector<int> output;
    int cnt = 0;

    Vertex(int p = -1, char ch = '$') : p(p), pch(ch) {
        fill(begin(next), end(next), -1);
        fill(begin(go), end(go), -1);
    }
};

vector<Vertex> t(1);

void add_string(string const &s, int id) {
    int v = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (t[v].next[c] == -1) {
            t[v].next[c] = t.size();
            t.emplace_back(v, ch);
        }
        v = t[v].next[c];
    }
    t[v].output.pb(id);
}

int go(int v, char ch);

int get_link(int v) {
    if (t[v].link == -1) {
        if (v == 0 || t[v].p == 0) {
            t[v].link = 0;
        } else {
            t[v].link = go(get_link(t[v].p), t[v].pch);
        }
    }
    return t[v].link;
}

int go(int v, char ch) {
    int c = ch-'a';
    if (t[v].go[c] == -1) {
        if (t[v].next[c] == -1) {
            t[v].go[c] = v == 0 ? 0 : go(get_link(v), ch);
        } else {
            t[v].go[c] = t[v].next[c];
        }
    }
    return t[v].go[c];
}

void dfs(ll u, ll p = -1) {
```

```
    for (ll v : g[u]) {
        dfs(v);
        t[u].cnt += t[v].cnt;
    }
    for (ll i : t[u].output) {
        ans[i] = t[u].cnt;
    }
}

void solve(ll tt, ll ti) {
    string s; cin >> s;
    n = s.size();
    cin >> k;

    vector<string> pat(k);
    for (ll i = 0; i < k; i++) {
        string t; cin >> t;
        pat[i] = t;
        add_string(t, i);
    }

    queue<ll> q;
    q.push(0);
    t[0].link = 0;

    while (!q.empty()) {
        ll v = q.front();
        q.pop();
        for (ll c = 0; c < K; c++) {
            if (t[v].next[c] != -1) {
                t[t[v].next[c]].link = v == 0 ? 0 : go(t[v].link, (char) (c + 'a
                    '));
                q.push(t[v].next[c]);
            }
        }
    }

    g.assign(t.size(), vl());
    for (ll i = 1; i < t.size(); i++) {
        g[t[i].link].pb(i);
    }

    int v = 0;
    for (auto ch : s) {
        v = go(v, ch);
        t[v].cnt++;
    }

    ans.assign(k, 0LL);
    dfs(0);

    for (ll i = 0; i < k; i++) {
        cout << ans[i] << nl;
    }
}

int main() {
        ios::sync_with_stdio(0), cin.tie(0);
        int t = 1;
        //cin >> t;

        for (int i = 1; i <= t; i++) {
                solve(t, i);
        }

        return 0;
}
```

13.3 distinct subsequences

```
#include <bits/stdc++.h>

#define nl '\n'
#define ll long long
#define ull unsigned long long
#define ld long double
#define pii pair<int, int>
#define pll pair<ll, ll>
#define vl vector<ll>
#define vvl vector<vl>
#define vll vector<pll>
#define pb push_back
#define eb emplace_back
#define ff first
#define ss second
#define all(x) (x).begin(), (x).end()
#define allr(x) (x).rbegin(), (x).rend()
#define sz(x) ((int) x.size())

using namespace std;

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

typedef __gnu_pbds::tree<int, __gnu_pbds::null_type, less<int>, __gnu_pbds::
    rb_tree_tag, __gnu_pbds::tree_order_statistics_node_update> ordered_set;

typedef __gnu_pbds::tree<int, __gnu_pbds::null_type, less_equal<int>, __gnu_pbds
    ::rb_tree_tag, __gnu_pbds::tree_order_statistics_node_update>
    ordered_multiset;

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

const ll INF = 1e9;
const ll P = 13;
const ll M = 1e9 + 7;
const ll MAX = 2e5 + 5;
const ll MAXN = 3e5 + 5;
const ll MAXT = 1e2 + 5;
const ll MAXK = 1e6 + 5;
const ll K = 26;

ll n, m, k, q, x, y, s;

//vector<vl> g;
//vl vis;
//vector<vector<pll>> g;

//vector<ll> primes;
//ll is_prime[MAXN];

void solve(ll tt, ll ti) {
    string s; cin >> s;
    n = s.size();

    vl dp(n+1), pref(n+1);
    dp[0] = pref[0] = 1;

    vl lst(26);
    for (ll i = 1; i <= n; i++) {
        if (lst[s[i-1]-'a'] == 0) {
            dp[i] = pref[i-1];
        } else {
            dp[i] = (pref[i-1] - pref[lst[s[i-1]-'a'] - 1] + M) % M;
        }
        pref[i] = (pref[i-1] + dp[i]) % M;
        lst[s[i-1]-'a'] = i;
    }
    cout << (pref[n]-1+M)%M << nl;
}

int main() {
        ios::sync_with_stdio(0), cin.tie(0);
        int t = 1;
        //cin >> t;

        for (int i = 1; i <= t; i++) {
                solve(t, i);
        }

        return 0;
}
```

13.4 distinct substrings

```
#include <bits/stdc++.h>
#define ll long long
#define pb push_back
#define eb emplace_back
#define all(x) (x).begin(), (x).end()
#define ff first
#define ss second
#define pll pair<ll, ll>
#define vl vector<ll>
#define vll vector<pll>
#define vvl vector<vl>
#define nl '\n'

using namespace std;

const ll M = 1e9 + 7;

ll n, m;

vector<int> suffix_array(string s) {
        s += "$";
        int n = s.size(), N = max(n, 260);
        vector<int> sa(n), ra(n);
        for(int i = 0; i < n; i++) sa[i] = i, ra[i] = s[i];

        for(int k = 0; k < n; k ? k *= 2 : k++) {
                vector<int> nsa(sa), nra(n), cnt(N);

                for(int i = 0; i < n; i++) nsa[i] = (nsa[i]-k+n)%n, cnt[ra[i
                    ]]++;
                for(int i = 1; i < N; i++) cnt[i] += cnt[i-1];
                for(int i = n-1; i+1; i--) sa[--cnt[ra[nsa[i]]]] = nsa[i];

                for(int i = 1, r = 0; i < n; i++) nra[sa[i]] = r += ra[sa[i]] !=
                        ra[sa[i-1]] or ra[(sa[i]+k)%n] != ra[(sa[i-1]+k)%n];
                ra = nra;
                if (ra[sa[n-1]] == n-1) break;
        }
        return vector<int>(sa.begin()+1, sa.end());
}

vector<int> kasai(string s, vector<int> sa) {
        int n = s.size(), k = 0;
        vector<int> ra(n), lcp(n);
        for (int i = 0; i < n; i++) ra[sa[i]] = i;

        for (int i = 0; i < n; i++, k -= !!k) {
                if (ra[i] == n-1) { k = 0; continue; }
                int j = sa[ra[i]+1];
                while (i+k < n and j+k < n and s[i+k] == s[j+k]) k++;
                lcp[ra[i]] = k;
        }
        return lcp;
}

void solve() {
    string s; cin >> s;
    n = s.size();
    vector<int> sa = suffix_array(s);
    vector<int> lcp = kasai(s, sa);
    ll ans = n-sa[0];
    for (ll i = 0; i < n-1; i++) {
        ans += n-(sa[i+1]+lcp[i]);
    }
    cout << ans << nl;
}

int main() {
        // your code goes here
        ios_base::sync_with_stdio(0), cin.tie(0);
        solve();
    return 0;
}
```

13.5 finding borders

```
#include <bits/stdc++.h>
#define ll long long
#define pb push_back
#define nl '\n'
using namespace std;

const ll M = 1e9 + 7;
const ll MAXN = 1e6 + 4;

vector<int> z_function(string &s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for(int i = 1; i < n; i++) {
        if(i < r) {
            z[i] = min(r - i, z[i - l]);
        }
        while(i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
        if(i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}

int main() {
        // your code goes here
        ios_base::sync_with_stdio(0), cin.tie(0);
    string s; cin >> s;
    ll n = s.size();

    vector<int> z = z_function(s);
    for (ll i = n-1; i >= 1; i--) {
        if (z[i] == n-i) cout << z[i] << " ";
    }
    cout << nl;

    return 0;
}
```

13.6 finding patterns

```
#include <bits/stdc++.h>

#define nl '\n'
#define ll long long
#define ull unsigned long long
#define ld long double
#define pii pair<int, int>
#define pll pair<ll, ll>
#define vl vector<ll>
#define vvl vector<vl>
#define vll vector<pll>
#define pb push_back
#define eb emplace_back
#define ff first
#define ss second
#define all(x) (x).begin(), (x).end()
#define allr(x) (x).rbegin(), (x).rend()
#define sz(x) ((int) x.size())

using namespace std;

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

typedef __gnu_pbds::tree<int, __gnu_pbds::null_type, less<int>, __gnu_pbds::
    rb_tree_tag, __gnu_pbds::tree_order_statistics_node_update> ordered_set;
```

```
typedef __gnu_pbds::tree<int, __gnu_pbds::null_type, less_equal<int>, __gnu_pbds
    ::rb_tree_tag, __gnu_pbds::tree_order_statistics_node_update>
    ordered_multiset;

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

const ll INF = 1e16;
const ll P = 13;
const ll M = 1e9 + 7;
const ll MAX = 5e3 + 5;
const ll MAXN = 3e5 + 5;
const ll MAXT = 1e2 + 5;
const ll MAXK = 1e6 + 5;
const ll K = 26;

ll n, m, k, q, x, y, s;

//vector<vl> g;
//vl vis;
//vector<vector<pll>> g;

//vector<ll> primes;
//ll is_prime[MAXN];

struct Vertex {
    int next[K];
    int go[K];
    int link = -1;
    int p = -1;
    char pch;
    int output = -1;

    Vertex(int p = -1, char ch = '$') : p(p), pch(ch) {
        fill(begin(next), end(next), -1);
        fill(begin(go), end(go), -1);
    }
};

vector<Vertex> t(1);

void add_string(string const &s, int id) {
    int v = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (t[v].next[c] == -1) {
            t[v].next[c] = t.size();
            t.emplace_back(v, ch);
        }
        v = t[v].next[c];
    }
    t[v].output = id;
}

int go(int v, char ch);

int get_link(int v) {
    if (t[v].link == -1) {
        if (v == 0 || t[v].p == 0) {
            t[v].link = 0;
        } else {
            t[v].link = go(get_link(t[v].p), t[v].pch);
        }
    }
    return t[v].link;
}

int go(int v, char ch) {
    int c = ch-'a';
    if (t[v].go[c] == -1) {
        if (t[v].next[c] == -1) {
            t[v].go[c] = v == 0 ? 0 : go(get_link(v), ch);
        } else {
            t[v].go[c] = t[v].next[c];
        }
    }
    return t[v].go[c];
}

void solve(ll tt, ll ti) {
    string s; cin >> s;
    n = s.size();
    cin >> k;

    map<string, ll> mp;
    vl get_id(k);
    for (ll i = 0; i < k; i++) {
        string t; cin >> t;
        if (mp.count(t)) {
            get_id[i] = mp[t];
            continue;
        }
        mp[t] = i;
        get_id[i] = i;
        add_string(t, i);
    }

    int v = 0;
    vl ans(k);
    vl vis(t.size());
    for (char ch : s) {
        int c = ch-'a';
        if (t[v].next[c] == -1) {
            v = go(v, ch);
        } else {
            v = t[v].next[c];
        }
        ll cur = v;
        while (cur != 0) {
            if (vis[cur]) break;
            vis[cur] = 1;
            if (t[cur].output != -1) ans[t[cur].output] = 1;
            cur = get_link(cur);
        }
    }

    for (ll i = 0; i < k; i++) cout << (ans[get_id[i]] ? "YES" : "NO") << nl;
}

int main() {
        ios::sync_with_stdio(0), cin.tie(0);
        int t = 1;
        //cin >> t;

        for (int i = 1; i <= t; i++) {
                solve(t, i);
        }

        return 0;
}
```

13.7 finding periods

```
#include <bits/stdc++.h>
#define ll long long
#define pb push_back
#define nl '\n'
using namespace std;

const ll M = 1e9 + 7;
const ll MAXN = 1e6 + 4;
const ll INF = 1e9;

vector<int> z_function(string &s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for(int i = 1; i < n; i++) {
        if(i < r) {
            z[i] = min(r - i, z[i - l]);
        }
        while(i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
```

```
        }
        if(i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}

int main() {
        // your code goes here
        ios_base::sync_with_stdio(0), cin.tie(0);
    string s; cin >> s;
    ll n = s.size();

    vector<int> z = z_function(s);
    for (ll i = 1; i <= n; i++) {
        ll ok = 1;
        for (ll j = i; j < n; j += i) {
            if (n - j >= i) {
                ok &= z[j] >= i;
            } else {
                ok &= z[j] == n-j;
            }
        }
        if (ok) cout << i << " ";
    }
    cout << nl;

    return 0;
}
```

13.8 longest palindrome

```
#include <bits/stdc++.h>
#define ll long long
#define nl '\n'

using namespace std;

const ll INF = 1e9;

template<typename T> vector<int> manacher(const T& s) {
        int l = 0, r = -1, n = s.size();
        vector<int> d1(n), d2(n);
        for (int i = 0; i < n; i++) {
                int k = i > r ? 1 : min(d1[l+r-i], r-i);
                while (i+k < n && i-k >= 0 && s[i+k] == s[i-k]) k++;
                d1[i] = k--;
                if (i+k > r) l = i-k, r = i+k;
        }
        l = 0, r = -1;
        for (int i = 0; i < n; i++) {
                int k = i > r ? 0 : min(d2[l+r-i+1], r-i+1); k++;
                while (i+k <= n && i-k >= 0 && s[i+k-1] == s[i-k]) k++;
                d2[i] = --k;
                if (i+k-1 > r) l = i-k, r = i+k-1;
        }
        vector<int> ret(2*n-1);
        for (int i = 0; i < n; i++) ret[2*i] = 2*d1[i]-1;
        for (int i = 0; i < n-1; i++) ret[2*i+1] = 2*d2[i+1];
        return ret;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    cin >> s;
    int n = (int)s.size();
    vector<int> man = manacher(s);
    ll ans = 0;
    ll start = 0;
    for (ll i = 0; i < n; i++) {
        if (man[2*i] > ans) {
            ans = man[2*i];
            start = i - ans / 2;
        }

        if (man[2*i+1] > ans) {
            ans = man[2*i+1];
            start = i - ans / 2 + 1;
        }
    }
    cout << s.substr(start, ans) << nl;
    return 0;
}
```

13.9 minimal rotation

```
input/code.cpp: In function 'int main()':
input/code.cpp:34:9: warning: unused variable 'n' [-Wunused-variable]
   34 |     int n = (int)s.size();
      |         ^
input/code.cpp: In instantiation of 'int max_suffix(T, bool) [with T = std::
    __cxx11::basic_string<char>]':
input/code.cpp:21:19:   required from 'int max_cyclic_shift(T) [with T = std::
    __cxx11::basic_string<char>]'
input/code.cpp:26:25:   required from 'int min_cyclic_shift(T) [with T = std::
    __cxx11::basic_string<char>]'
input/code.cpp:35:33:   required from here
input/code.cpp:8:27: warning: comparison of integer expressions of different
    signedness: 'int' and 'std::__cxx11::basic_string<char>::size_type' {aka '
    long unsigned int'} [-Wsign-compare]
    8 |         for (int i = 1; i < s.size(); i++) {
      |                         ~~^~~~~~~~~~~
input/code.cpp:12:38: warning: comparison of integer expressions of different
    signedness: 'int' and 'std::__cxx11::basic_string<char>::size_type' {aka '
    long unsigned int'} [-Wsign-com...
```

13.10 palindrome queries

```
#include <bits/stdc++.h>

#define nl '\n'
#define ll long long
#define ull unsigned long long
#define ld long double
#define pii pair<int, int>
#define pll pair<ll, ll>
#define vl vector<ll>
#define vvl vector<vl>
#define vll vector<pll>
#define pb push_back
#define eb emplace_back
#define ff first
#define ss second
#define all(x) (x).begin(), (x).end()
#define allr(x) (x).rbegin(), (x).rend()
#define sz(x) ((int) x.size())

using namespace std;

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

typedef __gnu_pbds::tree<int, __gnu_pbds::null_type, less<int>, __gnu_pbds::
    rb_tree_tag, __gnu_pbds::tree_order_statistics_node_update> ordered_set;

typedef __gnu_pbds::tree<int, __gnu_pbds::null_type, less_equal<int>, __gnu_pbds
    ::rb_tree_tag, __gnu_pbds::tree_order_statistics_node_update>
    ordered_multiset;

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
```

```
const ll INF = 1e9;
const ll P = 13;
const ll MOD = 1e9 + 7;
const ll MAX = 2e5 + 5;
const ll MAXN = 3e5 + 5;
const ll MAXT = 1e2 + 5;
const ll MAXK = 1e6 + 5;
const ll K = 26;

ll n, m, k, q, x, y, s;

//vector<vl> g;
//vl vis;
//vector<vector<pll>> g;

//vector<ll> primes;
//ll is_prime[MAXN];

ll bp(ll a, ll b, ll MOD) {
    ll res = 1;
    while (b > 0) {
        if (b & 1) res = res*a%MOD;
        b /= 2;
        a = a*a%MOD;
    }
    return res;
}

ll inverse(ll x, ll MOD) {
    return bp(x, MOD-2, MOD);
}

template<int MOD>
struct Bit {
        int n;
        vector<ll> bit;
        Bit(int _n=0) : n(_n), bit(n + 1) {}
        Bit(vector<ll> v) : n(v.size()), bit(n + 1) {
                for (int i = 1; i <= n; i++) {
                        bit[i] = (bit[i] + MOD + v[i - 1])%MOD;
                        int j = i + (i & -i);
                        if (j <= n) bit[j] = (bit[j]+MOD+bit[i])%MOD;
                }
        }
        void update(int i, ll x) { // soma x na posicao i
                for (i++; i <= n; i += i & -i) bit[i] = (bit[i] + x + MOD)%MOD;
        }
        ll pref(int i) { // soma [0, i]
                ll ret = 0;
                for (i++; i; i -= i & -i) ret = (ret+bit[i]+MOD)%MOD;
                return ret;
        }
        ll query(int l, int r) {  // soma [l, r]
                return (pref(r) - pref(l - 1)+MOD)%MOD;
        }
};

int uniform(int l, int r) {
        uniform_int_distribution<int> uid(l, r);
        return uid(rng);
}

template<int MOD> struct str_hash {
        static int P;
        vector<ll> h, p, inv;
        string s;
        int n;
        Bit<MOD> *bit = nullptr;
        str_hash(string s) : h(s.size()), p(s.size()), s(s) {
            n = s.size();
                p[0] = 1, h[0] = s[0];
                inv.resize(n);

                for (int i = 1; i < s.size(); i++) {
                    p[i] = p[i - 1]*P%MOD, h[i] = p[i]*s[i] % MOD;
                }

                for (ll i = 0; i < n; i++) inv[i] = inverse(p[i], MOD);
                bit = new Bit<MOD>(h);
        }
        ll operator()(int l, int r) { // retorna hash s[l...r]
                return bit->query(l, r) * inv[l] % MOD;
        }
        void update(ll i, ll ch) {
            bit->update(i, -(p[i]*s[i]%MOD));
            bit->update(i, p[i]*ch % MOD);
            s[i] = ch;
        }
};
template<int MOD> int str_hash<MOD>::P = 73; // l > |sigma|

void solve(ll tt, ll ti) {
    cin >> n >> m;
    string s; cin >> s;
    string t = s;
    reverse(all(t));
    str_hash<1000000007> h1(s);
    str_hash<1000000007> h2(t);
    for (ll i = 0; i < m; i++) {
        ll type; cin >> type;
        if (type == 1) {
            ll k; cin >> k;
            char ch; cin >> ch;
            k--;
            h1.update(k, ch);
            h2.update(n-k-1, ch);
        } else {
            ll a, b; cin >> a >> b;
            a--, b--;
            ll q1 = h1(a, b);
            ll q2 = h2(n-b-1, n-a-1);
            cout << (q1 == q2 ? "YES" : "NO") << nl;
        }
    }
}

int main() {
        ios::sync_with_stdio(0), cin.tie(0);
        int t = 1;
        //cin >> t;

        for (int i = 1; i <= t; i++) {
                solve(t, i);
        }

        return 0;
}
```

13.11 pattern positions

```
#include <bits/stdc++.h>

#define nl '\n'
#define ll long long
#define ull unsigned long long
#define ld long double
#define pii pair<int, int>
#define pll pair<ll, ll>
#define vl vector<ll>
#define vvl vector<vl>
#define vll vector<pll>
#define pb push_back
#define eb emplace_back
#define ff first
#define ss second
#define all(x) (x).begin(), (x).end()
#define allr(x) (x).rbegin(), (x).rend()
#define sz(x) ((int) x.size())

using namespace std;

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
```

```
typedef __gnu_pbds::tree<int, __gnu_pbds::null_type, less<int>, __gnu_pbds::
    rb_tree_tag, __gnu_pbds::tree_order_statistics_node_update> ordered_set;

typedef __gnu_pbds::tree<int, __gnu_pbds::null_type, less_equal<int>, __gnu_pbds
    ::rb_tree_tag, __gnu_pbds::tree_order_statistics_node_update>
    ordered_multiset;

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

const ll INF = 1e9;
const ll P = 13;
const ll M = 1e9 + 7;
const ll MAX = 5e3 + 5;
const ll MAXN = 3e5 + 5;
const ll MAXT = 1e2 + 5;
const ll MAXK = 1e6 + 5;
const ll K = 26;

ll n, m, k, q, x, y, s;

vector<vl> g;
//vl vis;
//vector<vector<pll>> g;

//vector<ll> primes;
//ll is_prime[MAXN];
vector<ll> ans, mn;

struct Vertex {
    int next[K];
    int go[K];
    int link = -1;
    int p = -1;
    char pch;
    vector<int> output;
    int len = 0;
    int mn = INF;

    Vertex(int p = -1, char ch = '$', int len = 0) : p(p), pch(ch), len(len) {
        fill(begin(next), end(next), -1);
        fill(begin(go), end(go), -1);
    }
};

vector<Vertex> t(1);

void add_string(string const &s, int id) {
    int v = 0;
    int len = 1;
    for (char ch : s) {
        int c = ch - 'a';
        if (t[v].next[c] == -1) {
            t[v].next[c] = t.size();
            t.emplace_back(v, ch, len);
        }
        v = t[v].next[c];
        len++;
    }
    t[v].output.pb(id);
}

int go(int v, char ch);

int get_link(int v) {
    if (t[v].link == -1) {
        if (v == 0 || t[v].p == 0) {
            t[v].link = 0;
        } else {
            t[v].link = go(get_link(t[v].p), t[v].pch);
        }
    }
    return t[v].link;
}

int go(int v, char ch) {
    int c = ch-'a';
    if (t[v].go[c] == -1) {
        if (t[v].next[c] == -1) {
            t[v].go[c] = v == 0 ? 0 : go(get_link(v), ch);
        } else {
            t[v].go[c] = t[v].next[c];
        }
    }
    return t[v].go[c];
}

void dfs(ll u) {
    for (ll v : g[u]) {
        dfs(v);
        t[u].mn = min(t[u].mn, t[v].mn + t[v].len - t[u].len);
    }
    for (ll i : t[u].output) {
        ans[i] = t[u].mn;
    }
}

void solve(ll tt, ll ti) {
    string s; cin >> s;
    n = s.size();
    cin >> k;

    vector<string> pat(k);
    for (ll i = 0; i < k; i++) {
        string t; cin >> t;
        pat[i] = t;
        add_string(t, i);
    }

    mn.assign(t.size(), INF);

    queue<ll> q;
    q.push(0);
    t[0].link = 0;

    while (!q.empty()) {
        ll v = q.front();
        q.pop();
        for (ll c = 0; c < K; c++) {
            if (t[v].next[c] != -1) {
                t[t[v].next[c]].link = v == 0 ? 0 : go(t[v].link, (char) (c + 'a
                    '));
                q.push(t[v].next[c]);
            }
        }
    }

    g.assign(t.size(), vl());
    for (ll i = 1; i < t.size(); i++) {
        g[t[i].link].pb(i);
    }

    int v = 0;
    for (ll i = 0; i < n; i++) {
        v = go(v, s[i]);
        t[v].mn = min((ll) t[v].mn, i - t[v].len + 1);
    }

    ans.assign(k, 0LL);
    dfs(0);

    for (ll i = 0; i < k; i++) {
        cout << (ans[i] == INF ? -1 : ans[i] + 1) << nl;
    }
}

int main() {
        ios::sync_with_stdio(0), cin.tie(0);
        int t = 1;
        //cin >> t;

        for (int i = 1; i <= t; i++) {
                solve(t, i);
        }

        return 0;
}
```

13.12 repeating substring

```
#include <bits/stdc++.h>
#define ll long long
#define pb push_back
#define eb emplace_back
#define all(x) (x).begin(), (x).end()
#define ff first
#define ss second
#define pll pair<ll, ll>
#define vl vector<ll>
#define vll vector<pll>
#define vvl vector<vl>
#define nl '\n'

using namespace std;

const ll M = 1e9 + 7;

ll n, m;

vector<int> suffix_array(string s) {
        s += "$";
        int n = s.size(), N = max(n, 260);
        vector<int> sa(n), ra(n);
        for(int i = 0; i < n; i++) sa[i] = i, ra[i] = s[i];

        for(int k = 0; k < n; k ? k *= 2 : k++) {
                vector<int> nsa(sa), nra(n), cnt(N);

                for(int i = 0; i < n; i++) nsa[i] = (nsa[i]-k+n)%n, cnt[ra[i
                    ]]++;
                for(int i = 1; i < N; i++) cnt[i] += cnt[i-1];
                for(int i = n-1; i+1; i--) sa[--cnt[ra[nsa[i]]]] = nsa[i];

                for(int i = 1, r = 0; i < n; i++) nra[sa[i]] = r += ra[sa[i]] !=
                        ra[sa[i-1]] or ra[(sa[i]+k)%n] != ra[(sa[i-1]+k)%n];
                ra = nra;
                if (ra[sa[n-1]] == n-1) break;
        }
        return vector<int>(sa.begin()+1, sa.end());
}

vector<int> kasai(string s, vector<int> sa) {
        int n = s.size(), k = 0;
        vector<int> ra(n), lcp(n);
        for (int i = 0; i < n; i++) ra[sa[i]] = i;

        for (int i = 0; i < n; i++, k -= !!k) {
                if (ra[i] == n-1) { k = 0; continue; }
                int j = sa[ra[i]+1];
                while (i+k < n and j+k < n and s[i+k] == s[j+k]) k++;
                lcp[ra[i]] = k;
        }
        return lcp;
}

void solve() {
    string s; cin >> s;
    n = s.size();
    vector<int> sa = suffix_array(s);
    vector<int> lcp = kasai(s, sa);
    ll ans = n-sa[0];
    ll mx = 0;
    ll start = -1;
    for (ll i = 0; i < n-1; i++) {
        if (lcp[i] > mx) {
            mx = lcp[i];
            start = sa[i];
        }
    }
    if (start == -1) {
        cout << -1 << nl;
    } else {
        cout << s.substr(start, mx) << nl;
    }
}

int main() {
        // your code goes here
        ios_base::sync_with_stdio(0), cin.tie(0);
        solve();
    return 0;
}
```

13.13 required substring

```
#include <bits/stdc++.h>
#define ll long long
#define pb push_back
#define eb emplace_back
#define all(x) (x).begin(), (x).end()
#define ff first
#define ss second
#define pll pair<ll, ll>
#define vl vector<ll>
#define vll vector<pll>
#define vvl vector<vl>
#define nl '\n'

using namespace std;

const ll M = 1e9 + 7;

ll n, m;

ll len[101][26];

vector<int> prefix_function(string s) {
    int n = (int)s.length();
    vector<int> pi(n);
    memset(len, 0, sizeof(len));
    len[0][s[0]-'A'] = 1;
    for (int i = 1; i < n; i++) {
        char old = s[i];
        for (ll k = 0; k < 26; k++) {
            s[i] = old;
            if (k == (s[i] - 'A')) {
                len[i][k] = i+1;
                continue;
            }
            s[i] = (char) (k + 'A');
            int j = pi[i-1];
            while (j > 0 && s[i] != s[j])
                j = pi[j-1];
            if (s[i] == s[j])
                j++;
            len[i][k] = j;
        }
        s[i] = old;
        int j = pi[i-1];
        while (j > 0 && s[i] != s[j])
            j = pi[j-1];
        if (s[i] == s[j])
            j++;
        pi[i] = j;
    }
    return pi;
}

void solve() {
    cin >> n;
    string s; cin >> s;
    m = s.size();

    auto p = prefix_function(s);

    ll dp[n+1][m+1];
    memset(dp, 0, sizeof(dp));
    dp[0][0] = 1;
    for (ll i = 1; i <= n; i++) {
```

```
        for (ll j = 0; j < 26; j++) {
            for (ll k = 0; k < m; k++) {
                dp[i][len[k][j]] += dp[i-1][k];
                dp[i][len[k][j]] %= M;
            }
        }
    }

    ll ans = 1;
    for (ll i = 0; i < n; i++) {
        ans = ans * 26 % M;
    }

    for (ll k = 0; k < m; k++) {
        ans = (ans - dp[n][k] + M) % M;
    }

    cout << ans << nl;
}

int main() {
        // your code goes here
        ios_base::sync_with_stdio(0), cin.tie(0);
        solve();
    return 0;
}
```

13.14 string matching

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define pb push_back
#define eb emplace_back
#define ff first
#define ss second
#define sz(x) (int) (x).size()
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cout << #x << " = " << x << endl

vector<int> pi(string s) {
    vector<int> p(s.size());
    for (int i = 1, j = 0; i < (int) s.size(); i++) {
        while (j > 0 and s[i] != s[j]) j = p[j-1];
        if (s[i] == s[j]) j++;
        p[i] = j;
    }
    return p;
}

int matching(string &t, string& s) {
    vector<int> p = pi(s+'$');
    int matchs = 0;
    for (int i = 0, j = 0; i < (int) t.size(); i++) {
        while (j > 0 and t[i] != s[j]) j = p[j-1];
        if (t[i] == s[j]) j++;
        if (j == (int) s.size()) matchs++;
    }
    return matchs;
}

signed main() {
    ios_base::sync_with_stdio(0);cin.tie(0);
    string t, s; cin >> t >> s;
    int ans = matching(t, s);
    cout << ans << endl;
}
```

13.15 substring distribution

```
#include <bits/stdc++.h>

#define nl '\n'
#define ll long long
#define ull unsigned long long
#define ld long double
#define pii pair<int, int>
#define pll pair<ll, ll>
#define vl vector<ll>
#define vvl vector<vl>
#define vll vector<pll>
#define pb push_back
#define eb emplace_back
#define ff first
#define ss second
#define all(x) (x).begin(), (x).end()
#define allr(x) (x).rbegin(), (x).rend()
#define sz(x) ((int) x.size())

using namespace std;

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

typedef __gnu_pbds::tree<int, __gnu_pbds::null_type, less<int>, __gnu_pbds::
    rb_tree_tag, __gnu_pbds::tree_order_statistics_node_update> ordered_set;

typedef __gnu_pbds::tree<int, __gnu_pbds::null_type, less_equal<int>, __gnu_pbds
    ::rb_tree_tag, __gnu_pbds::tree_order_statistics_node_update>
    ordered_multiset;

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

const ll INF = 1e9;
const ll P = 13;
const ll M = 1e9 + 7;
const ll MAX = 2e5 + 5;
const ll MAXN = 3e5 + 5;
const ll MAXT = 1e2 + 5;
const ll MAXK = 1e6 + 5;
const ll K = 26;

ll n, m, k, q, x, y, s;

//vector<vl> g;
//vl vis;
//vector<vector<pll>> g;

//vector<ll> primes;
//ll is_prime[MAXN];

vector<int> suffix_array(string s) {
        s += "$";
        int n = s.size(), N = max(n, 260);
        vector<int> sa(n), ra(n);
        for(int i = 0; i < n; i++) sa[i] = i, ra[i] = s[i];

        for(int k = 0; k < n; k ? k *= 2 : k++) {
                vector<int> nsa(sa), nra(n), cnt(N);

                for(int i = 0; i < n; i++) nsa[i] = (nsa[i]-k+n)%n, cnt[ra[i
                    ]]++;
                for(int i = 1; i < N; i++) cnt[i] += cnt[i-1];
                for(int i = n-1; i+1; i--) sa[--cnt[ra[nsa[i]]]] = nsa[i];

                for(int i = 1, r = 0; i < n; i++) nra[sa[i]] = r += ra[sa[i]] !=
                                ra[sa[i-1]] or ra[(sa[i]+k)%n] != ra[(sa[i-1]+k)
                                    %n];
                ra = nra;
                if (ra[sa[n-1]] == n-1) break;
        }
        return vector<int>(sa.begin()+1, sa.end());
}

vector<int> kasai(string s, vector<int> sa) {
        int n = s.size(), k = 0;
        vector<int> ra(n), lcp(n);
        for (int i = 0; i < n; i++) ra[sa[i]] = i;
```

```
        for (int i = 0; i < n; i++, k -= !!k) {
                if (ra[i] == n-1) {
                        k = 0;
                        continue;
                }
                int j = sa[ra[i]+1];
                while (i+k < n and j+k < n and s[i+k] == s[j+k]) k++;
                lcp[ra[i]] = k;
        }
        return lcp;
}

void solve(ll tt, ll ti) {
        string s;
        cin >> s;
        cin >> k;
        n = s.size();
        ll cnt = 0;

        vector<int> sa = suffix_array(s);
        vector<int> lcp = kasai(s, sa);

        vl pref(n+2);

        for (ll i = 0; i < n; i++) {
            if (i == 0) pref[1]++;
            else pref[lcp[i-1]+1]++;
            pref[n - sa[i] + 1]--;
        }

        for (ll i = 1; i <= n; i++) {
            pref[i] += pref[i-1];
            cout << pref[i] << " ";
        }
        cout << nl;
}

int main() {
        ios::sync_with_stdio(0), cin.tie(0);
        int t = 1;
        //cin >> t;

        for (int i = 1; i <= t; i++) {
                solve(t, i);
        }

        return 0;
}
```

13.16 substring order i

```
#include <bits/stdc++.h>

#define nl '\n'
#define ll long long
#define ull unsigned long long
#define ld long double
#define pii pair<int, int>
#define pll pair<ll, ll>
#define vl vector<ll>
#define vvl vector<vl>
#define vll vector<pll>
#define pb push_back
#define eb emplace_back
#define ff first
#define ss second
#define all(x) (x).begin(), (x).end()
#define allr(x) (x).rbegin(), (x).rend()
#define sz(x) ((int) x.size())

using namespace std;

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

typedef __gnu_pbds::tree<int, __gnu_pbds::null_type, less<int>, __gnu_pbds::
    rb_tree_tag, __gnu_pbds::tree_order_statistics_node_update> ordered_set;

typedef __gnu_pbds::tree<int, __gnu_pbds::null_type, less_equal<int>, __gnu_pbds
    ::rb_tree_tag, __gnu_pbds::tree_order_statistics_node_update>
    ordered_multiset;

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

const ll INF = 1e9;
const ll P = 13;
const ll M = 1e9 + 7;
const ll MAX = 2e5 + 5;
const ll MAXN = 3e5 + 5;
const ll MAXT = 1e2 + 5;
const ll MAXK = 1e6 + 5;
const ll K = 26;

ll n, m, k, q, x, y, s;

//vector<vl> g;
//vl vis;
//vector<vector<pll>> g;

//vector<ll> primes;
//ll is_prime[MAXN];

vector<int> suffix_array(string s) {
        s += "$";
        int n = s.size(), N = max(n, 260);
        vector<int> sa(n), ra(n);
        for(int i = 0; i < n; i++) sa[i] = i, ra[i] = s[i];

        for(int k = 0; k < n; k ? k *= 2 : k++) {
                vector<int> nsa(sa), nra(n), cnt(N);

                for(int i = 0; i < n; i++) nsa[i] = (nsa[i]-k+n)%n, cnt[ra[i
                    ]]++;
                for(int i = 1; i < N; i++) cnt[i] += cnt[i-1];
                for(int i = n-1; i+1; i--) sa[--cnt[ra[nsa[i]]]] = nsa[i];

                for(int i = 1, r = 0; i < n; i++) nra[sa[i]] = r += ra[sa[i]] !=
                                ra[sa[i-1]] or ra[(sa[i]+k)%n] != ra[(sa[i-1]+k)
                                    %n];
                ra = nra;
                if (ra[sa[n-1]] == n-1) break;
        }
        return vector<int>(sa.begin()+1, sa.end());
}

vector<int> kasai(string s, vector<int> sa) {
        int n = s.size(), k = 0;
        vector<int> ra(n), lcp(n);
        for (int i = 0; i < n; i++) ra[sa[i]] = i;

        for (int i = 0; i < n; i++, k -= !!k) {
                if (ra[i] == n-1) {
                        k = 0;
                        continue;
                }
                int j = sa[ra[i]+1];
                while (i+k < n and j+k < n and s[i+k] == s[j+k]) k++;
                lcp[ra[i]] = k;
        }
        return lcp;
}

void solve(ll tt, ll ti) {
        string s;
        cin >> s;
        cin >> k;
        n = s.size();
        ll cnt = 0;

        vector<int> sa = suffix_array(s);
        vector<int> lcp = kasai(s, sa);

        for (ll i = 0; i < n; i++) {
                if (i == 0) {
```

```
                cnt += n - sa[0];
            } else {
                cnt += n - (sa[i] + lcp[i-1]);
            }

            if (cnt >= k) {
                ll len = k, start = sa[i];
                if (i != 0) {
                    ll end = (n-1) - (cnt - k);
                        len = end - start + 1;
                }
                cout << s.substr(start, len);
                break;
            }
        }
}

int main() {
        ios::sync_with_stdio(0), cin.tie(0);
        int t = 1;
        //cin >> t;

        for (int i = 1; i <= t; i++) {
            solve(t, i);
        }

        return 0;
}
```

13.17 word combinations

```
#include <bits/stdc++.h>
#define ll long long
#define pb push_back

using namespace std;

const ll M = 1e9 + 7;
const ll MAXN = 1e6 + 4;
ll is_end[MAXN];

vector<vector<ll>> trie(1e6+5, vector<ll>(26));
ll id = 0;

vector<int> z_function(string &s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for(int i = 1; i < n; i++) {
        if(i < r) {
            z[i] = min(r - i, z[i - l]);
        }
        while(i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
        if(i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}

void insert(string &s) {
    ll node = 0;
    for (auto ch : s) {
        if (!trie[node][ch-'a']) trie[node][ch-'a'] = ++id;
        node = trie[node][ch-'a'];
    }
    is_end[node] = 1;
}

int main() {
        // your code goes here
        ios_base::sync_with_stdio(0), cin.tie(0);
    string s; cin >> s;
    int k; cin >> k;
    int n = s.size();
    memset(is_end, 0, sizeof(is_end));

    for (int i = 0; i < k; i++) {
        string t; cin >> t;
        insert(t);
    }

    vector<int> dp(n+1);
    dp[n] = 1;
    for (int i = n-1; i >= 0; i--) {
        ll node = 0;
        for (ll j = i; j < n; j++) {
            if (!trie[node][s[j]-'a']) break;
            node = trie[node][s[j]-'a'];
            if (is_end[node]) {
                dp[i] += dp[j+1];
                dp[i] %= M;
            }
        }
    }

    cout << dp[0] << endl;

    return 0;
}
```

14 Classics - Tree Algorithms

14.1 company queries i

```
#include <bits/stdc++.h>
using namespace std;
#define ff first
#define ss second
#define pb push_back
// #define int long long
#define endl '\n' // << flush
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cerr << #x << " = " << x << endl
#define uniq(v) sort(all(v)); v.erase(unique(all(v)), v.end())
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
typedef long long ll;
typedef pair<int, int> pi;
// const int MOD = 1e9 + 7; // 998244353;
// const int INF = 0x3f3f3f3f;
// const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const int MAXN = 2e5+5;
const int LOG = 20;
int n, q, up[MAXN][LOG], depth[MAXN];
vector<int> adj[MAXN];
void dfs(int v, int p) { // O(nlog(n))
    for (auto u : adj[v]) if (u != p) {
        depth[u] = depth[v] + 1;
        up[u][0] = v;
        for (int j = 1; j < LOG; j++) {
            up[u][j] = up[ up[u][j-1] ][j-1];
        }
        dfs(u, v);
    }
}
// Binary Lifting -> Encontrar o kth ancestor O(logn)
int get_kth(int x, int k) {
    if (k > depth[x]) return -1;
    for (int i = 0; i < LOG; i++) {
        if (k & (1 << i))
            x = up[x][i];
    }
    return x;
}
// Encontrar o LCA O(log(n))
```

```
int get_lca(int a, int b) {
    if (depth[a] < depth[b]) {
        swap(a, b);
    }
    int k = depth[a] - depth[b];
    for (int j = LOG-1; j >= 0; j--) {
        if (k & (1 << j)) {
            a = up[a][j];
        }
    }
    if (a == b) {
        return a;
    }
    for (int j = LOG-1; j >= 0; j--) {
        if (up[a][j] != up[b][j]) {
            a = up[a][j];
            b = up[b][j];
        }
    }
    return up[a][0];
}
void solve() {
    cin >> n >> q;
    for (int v = 1; v < n; v++) {
        int u; cin >> u; u--;
        adj[v].pb(u);
        adj[u].pb(v);
    }
    dfs(0, -1);
    for (int i = 0; i < q; i++) {
        // int a, b; cin >> a >> b; a--, b--;
        // cout << (get_lca(a, b) + 1) << endl;
        int x, k; cin >> x >> k; x--;
        int ans = get_kth(x, k);
        if (ans == -1) cout << ans;
        else cout << ans+1;
        cout << endl;
    }
}
signed main() {
    ios_base::sync_with_stdio(0);cin.tie(0);
    int tc = 1;
    //cin >> tc;
    while(tc--) solve();
}
```

14.2 company queries ii

```
#include <bits/stdc++.h>
using namespace std;
#define ff first
#define ss second
#define pb push_back
// #define int long long
#define endl '\n' // << flush
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cerr << #x << " = " << x << endl
#define uniq(v) sort(all(v)); v.erase(unique(all(v)), v.end())
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
typedef long long ll;
typedef pair<int, int> pi;
// const int MOD = 1e9 + 7; // 998244353;
// const int INF = 0x3f3f3f3f;
// const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const int MAXN = 2e5+5;
const int LOG = 20;

int n, q, up[MAXN][LOG], depth[MAXN];
vector<int> adj[MAXN];

void dfs(int v, int p) {
    for (auto u : adj[v]) if (u != p) {
        depth[u] = depth[v] + 1;
        up[u][0] = v;
        for (int j = 1; j < LOG; j++) {
            up[u][j] = up[ up[u][j-1] ][j-1];
        }
        dfs(u, v);
    }
}

int get_lca(int a, int b) {
    if (depth[a] < depth[b]) {
        swap(a, b);
    }
    int k = depth[a] - depth[b];
    for (int j = LOG-1; j >= 0; j--) {
        if (k & (1 << j)) {
            a = up[a][j];
        }
    }
    if (a == b) {
        return a;
    }
    for (int j = LOG-1; j >= 0; j--) {
        if (up[a][j] != up[b][j]) {
            a = up[a][j];
            b = up[b][j];
        }
    }
    return up[a][0];
}

void solve() {
    cin >> n >> q;
    for (int v = 1; v < n; v++) {
        int u; cin >> u; u--;
        adj[v].pb(u);
        adj[u].pb(v);
    }
    dfs(0, -1);
    for (int i = 0; i < q; i++) {
        int a, b; cin >> a >> b; a--, b--;
        cout << (get_lca(a, b) + 1) << endl;
    }
}

signed main() {
    ios_base::sync_with_stdio(0);cin.tie(0);
    int tc = 1;
    //cin >> tc;
    while(tc--) solve();
}
```

14.3 distance queries

```
#include <bits/stdc++.h>
using namespace std;
#define ff first
#define ss second
#define pb push_back
// #define int long long
#define endl '\n' // << flush
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cerr << #x << " = " << x << endl
#define uniq(v) sort(all(v)); v.erase(unique(all(v)), v.end())
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
typedef long long ll;
typedef pair<int, int> pi;
// const int MOD = 1e9 + 7; // 998244353;
// const int INF = 0x3f3f3f3f;
// const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const int MAXN = 2e5+5;
const int LOG = 20;
int n, q, up[MAXN][LOG], depth[MAXN];
vector<int> adj[MAXN];
void dfs(int v, int p) { // O(nlog(n))
    for (auto u : adj[v]) if (u != p) {
```

```
        depth[u] = depth[v] + 1;
        up[u][0] = v;
        for (int j = 1; j < LOG; j++) {
            up[u][j] = up[ up[u][j-1] ][j-1];
        }
        dfs(u, v);
    }
}
// Binary Lifting -> Encontrar o kth ancestor O(logn)
int get_kth(int x, int k) {
    if (k > depth[x]) return -1;
    for (int i = 0; i < LOG; i++) {
        if (k & (1 << i))
            x = up[x][i];
    }
    return x;
}
// Encontrar o LCA O(log(n))
int get_lca(int a, int b) {
    if (depth[a] < depth[b]) {
        swap(a, b);
    }
    int k = depth[a] - depth[b];
    for (int j = LOG-1; j >= 0; j--) {
        if (k & (1 << j)) {
            a = up[a][j];
        }
    }
    if (a == b) {
        return a;
    }
    for (int j = LOG-1; j >= 0; j--) {
        if (up[a][j] != up[b][j]) {
            a = up[a][j];
            b = up[b][j];
        }
    }
    return up[a][0];
}
int dist(int u, int v) {
    int l = get_lca(u, v);
    return depth[u] + depth[v] - 2*depth[l];
}
void solve() {
    cin >> n >> q;
    for (int i = 0; i < n-1; i++) {
        int v, u; cin >> v >> u; v--, u--;
        adj[v].pb(u);
        adj[u].pb(v);
    }
    dfs(0, -1);
    for (int i = 0; i < q; i++) {
        int a, b; cin >> a >> b; a--, b--;
        cout << dist(a, b) << endl;
    }
}
signed main() {
    ios_base::sync_with_stdio(0);cin.tie(0);
    int tc = 1;
    //cin >> tc;
    while(tc--) solve();
}
```

14.4 distinct colors

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back
#define int long long

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;

const int N = 2e5 + 10;
vi adj[N];
int ans[N], sz[N], cnt[N], cor[N];
bool big[N];
int resp=0;

void dfs_size(int u, int p) {
  sz[u] = 1;
  for (auto v : adj[u])
    if (v != p) {
      dfs_size(v, u);
      sz[u] += sz[v];
    }
}

void add(int u, int p, int val) {
  if(cnt[cor[u]]==0) resp+=val;
  cnt[cor[u]] += val;
  if(cnt[cor[u]]==0) resp+=val;
  for (auto v : adj[u]) {
    if (v != p and !big[v]) {
      add(v,u,val);
    }
  }
}

void dfs_dsu(int u, int p, bool keep) {
  int bigchild = -1, mx = -1;
  for (auto v : adj[u]) {
    if (v == p)
      continue;
    if (sz[v] > mx)
      mx = sz[v], bigchild = v;
  }
  for (auto v : adj[u]) {
    if (v != p and v!=bigchild) {
      dfs_dsu(v, u, false);
    }
  }

  if (bigchild != -1) {
    dfs_dsu(bigchild, u, true);
    big[bigchild]=1;
  }

  add(u, p, 1);
  ans[u]=resp;
  if (bigchild!=-1) {
    big[bigchild] = 0;
  }
  if (!keep) {
    add(u, p, -1);
  }
}

signed main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n;
  cin >> n;
  vi vals;
  for (int i = 1; i <= n; i++){
    cin >> cor[i];
    vals.pb(cor[i]);
```

```
  }

  sort(all(vals));
  vals.erase(unique(all(vals)),vals.end());

  for(int i=1;i<=n;i++){
    cor[i] = lower_bound(all(vals), cor[i])-vals.begin();
  }

  for (int i = 0; i < n - 1; i++) {
    int u, v;
    cin >> u >> v;
    adj[u].pb(v);
    adj[v].pb(u);
  }

  dfs_size(1, 0);
  dfs_dsu(1, 0, false);

  for (int i = 1; i <= n; i++) {
    cout << ans[i] << " ";
  }
  cout << endl;

  return 0;
}
```

14.5 finding a centroid

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back
#define int long long

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;

const int N = 2e5+10;

vi adj[N];
int sz[N];
int tot=0;

void dfs(int u, int p){
  tot++;
  sz[u]=1;
  for(auto v : adj[u]) if(v!=p){
    dfs(v,u);
    sz[u]+=sz[v];
  }
}

int cent(int u, int p){
  for(auto v : adj[u]){
    if(v==p) continue;
    if(sz[v] > tot/2) return cent(v,u);
  }
  return u;
}

int find_centroid(){
  dfs(1,0);
  return cent(1,0);
}

signed main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n; cin >> n;
  for(int i=0;i<n-1;i++){
    int u, v; cin >> u >> v;
    adj[u].pb(v);
    adj[v].pb(u);
  }

  int ans = find_centroid();

  cout << ans << endl;

  return 0;
}
```

14.6 fixed length paths i

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back
#define int long long

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;

const int N = 2e5 + 10;

vi adj[N];
int sz[N], cnt[N]{1};
int tot = 0, ans = 0, mx_depth=-1;
bool done[N];
int k;

void calc_size(int u, int p) {
  sz[u] = 1;
  tot++;
  for (auto v : adj[u]) {
    if (v == p || done[v])
      continue;
    calc_size(v, u);
    sz[u] += sz[v];
  }
}

int find_cent(int u, int p) {
  for (auto v : adj[u]) {
```

```
    if (v == p || done[v])
      continue;
    if (sz[v] > tot / 2)
      return find_cent(v, u);
  }
  return u;
}

void add_cnt(int u, int p, bool filling, int depth=1) {
  if(depth >k) return;
  mx_depth = max(mx_depth, depth);
  if(filling) cnt[depth]++;
  else ans += cnt[k-depth];
  for(auto v : adj[u]){
    if(done[v] or v==p)continue;
    add_cnt(v,u,filling,depth+1);
  }

}

void centroid_tree(int u, int p) {
  tot = 0;
  calc_size(u, p);
  int cen = find_cent(u, p);
  done[cen]=1;
  mx_depth=0;
  for(auto v : adj[cen]){
    if(done[v]) continue;
    add_cnt(v,cen,false);
    add_cnt(v,cen,true);
  }
  fill(cnt+1, cnt+mx_depth+1,0);
  for(auto v : adj[cen]){
    if(!done[v]) centroid_tree(v,u);
  }
}

signed main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);

  int n;
  cin >> n >> k;
  for (int i = 0; i < n - 1; i++) {
    int u, v;
    cin >> u >> v;
    adj[u].pb(v);
    adj[v].pb(u);
  }

  centroid_tree(1, 0);

  cout << ans << endl;
  return 0;
}
```

14.7 fixed length paths ii

```
#include <bits/stdc++.h>
using namespace std;
#define all(a) a.begin(), a.end()
struct FenwickTree
{
    vector<int> BIT;
    int n;
    FenwickTree(int n) : n(n + 1), BIT(n + 5) {}

    void add(int i, int val)
    {
        for (++i; i <= n; i += i & -i)
            BIT[i] += val;
    }
    void set(int i)
    {
        for (++i; i <= n; i += i & -i)
            BIT[i] = 0;
    }
    int pre(int i)
    {
        int sum = 0;
        for (++i; i; i -= i & -i)
            sum += BIT[i];
        return sum;
    }
    int query(int l, int r)
    {
        return pre(r) - pre(l - 1);
    }
};
const int N = 2e5 + 5;
vector<int> adj[N];
long long ans;
int sz[N];
FenwickTree cnt(N);
bool removed[N];
int k1, k2, mx;
long long depth[N];
void dfsSz(int u, int p)
{
    sz[u] = 1;
    for (auto v : adj[u])
    {
        if (v == p || removed[v])
            continue;
        dfsSz(v, u);
        sz[u] += sz[v];
    }
};
int get(int u, int p, int n)
{
    for (auto v : adj[u])
    {
        if (v == p || removed[v])
            continue;
        if (2 * sz[v] > n)
            return get(v, u, n);
    }
    return u;
}
int dfs(int u, int p, int d)
{
    depth[d]++;
    int ret = d;
    for (auto v : adj[u])
    {
        if (v == p || removed[v] || k2 < d + 1)
            continue;
        ret = max(ret, dfs(v, u, d + 1));
    }
    return ret;
}
void decomp(int u)
{
    dfsSz(u, -1);
    int root = get(u, -1, sz[u]);
    removed[root] = 1;
    cnt.add(0, 1);
    mx = 0;
    for (auto v : adj[root])
    {
        if (!removed[v])
        {
            int tot = dfs(v, root, 1);
            for (int i = tot; i >= 1; i--)
            {
                if (depth[i] == 0)
                    continue;
                int l = max(0, k1 - i);
                int r = k2 - i;
                ans += depth[i] * cnt.query(l, r);
            }
            for (int i = tot; i >= 1; i--)
            {
```

```
                cnt.add(i, depth[i]);
                depth[i] = 0;
            }
        }
    }
    for (int i = 0; i <= sz[u]; i++)
        cnt.set(i), depth[i] = 0;
    for (auto v : adj[root])
    {
        if (!removed[v])
            decomp(v);
    }
}
void Solve()
{
    int n;
    cin >> n >> k1 >> k2;
    for (int i = 0; i < n - 1; i++)
    {
        int u, v;
        cin >> u >> v, u--, v--;
        adj[u].emplace_back(v);
        adj[v].emplace_back(u);
    }
    decomp(0);
    cout << ans;
}

int32_t main()
{
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);
#ifdef CPH
#elif defined(ONLINE_JUDGE)
    // freopen("IN.in", "r", stdin);
    // freopen("OUT.out", "w", stdout);
#else
    freopen("IN.txt", "r", stdin);
    freopen("OUT.txt", "w", stdout);
#endif
    int t = 1;
    // cin >> t;
    while (t--)
    {
        // cout << "Case #" << TC++ << ": ";
        Solve();
    }
    return 0;
}
```

14.8 path queries ii

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;

const int N = 2e5 + 10;

struct SegTree {
    int n;
    int tree[2 * N];

    SegTree() {}

    void build(int n_nodes, const vector<int> &arr) {
        n = n_nodes;
        for (int i = 0; i < n; i++) tree[n + i] = arr[i + 1];
        for (int i = n - 1; i > 0; --i) tree[i] = max(tree[i << 1], tree[i << 1
             | 1]);
    }

    void update(int pos, int val) {
        pos--;
        for (tree[pos += n] = val; pos > 1; pos >>= 1)
            tree[pos >> 1] = max(tree[pos], tree[pos ^ 1]);
    }

    int query(int l, int r) {
        l--; r--;
        int ans = 0;
        for (l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
            if (l & 1) ans = max(ans, tree[l++]);
            if (r & 1) ans = max(ans, tree[--r]);
        }
        return ans;
    }
};



SegTree tree;
int par[N];
int sz[N];
int depth[N];
vi adj[N];

int head[N];
int in[N];
int val[N];
int timer_hld = 0;

void dfs(int u, int p) {
  par[u] = p;
  sz[u] = 1;
  int max_sz = 0;
  int heavy_node = -1;
  depth[u] = depth[p] + 1;
  for (int i = 0; i < (int)adj[u].size(); i++) {
    int v = adj[u][i];
    if (v != p) {
      dfs(v, u);
      sz[u] += sz[v];
      if (sz[v] > max_sz) {
        max_sz = sz[v];
        heavy_node = i;
      }
    }
  }
  if (heavy_node != -1) {
    swap(adj[u][0], adj[u][heavy_node]);
  }
}

void dfs_solve(int u, int p, int h) {
  head[u] = h;
  in[u] = ++timer_hld;

  for (auto v : adj[u]) {
    if (v != p) {
      dfs_solve(v, u, v == adj[u][0] ? h : v);
    }
```

```
  }
}

int query_path(int l, int r) {
  int ans = 0;

  while (head[l] != head[r]) {
    if (depth[head[l]] < depth[head[r]])
      swap(l, r);

    ans = max(ans, tree.query(in[head[l]], in[l]));
    l = par[head[l]];
  }

  if (depth[l] > depth[r])
    swap(l, r);

  ans = max(ans, tree.query(in[l], in[r]));
  return ans;
}

signed main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n, q;
  cin >> n >> q;

  for (int i = 1; i <= n; i++)
    cin >> val[i];

  for (int i = 0; i < n - 1; i++) {
    int u, v;
    cin >> u >> v;
    adj[u].pb(v);
    adj[v].pb(u);
  }

  depth[0] = 0;
  dfs(1, 0);
  dfs_solve(1, 0, 1);

  vi vals2(n + 1, 0);
  for (int i = 1; i <= n; i++) {
    vals2[in[i]] = val[i];
  }

  tree.build(n,vals2);

  while (q--) {
    int t;
    cin >> t;
    if (t == 1) {
      int s, x;
      cin >> s >> x;
      val[s] = x;
      tree.update(in[s], x);
    } else {
      int l, r;
      cin >> l >> r;
      cout << query_path(l, r) << " ";
    }
  }
  cout << endl;

  return 0;
}
```

14.9 path queries

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define f first
#define sec second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define pb push_back
#define int long long

typedef long long ll;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vl;
typedef vector<bool> vb;
typedef vector<pii> vpii;
typedef vector<pll> vpll;
typedef vector<vi> vvi;

const int INF = 0x3f3f3f3f;
const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const double EPS = 1e-9;
const ll MOD = 1e9 + 7;

template<typename T>
struct LazySegTree {
    int n;
    vector<T> tree, lazy;
    T neutro = 0;

    LazySegTree(int n) : n(n) {
        tree.assign(4 * n, neutro);
        lazy.assign(4 * n, 0);
    }

    T combina(T a, T b) {
        return a + b;
    }

    void push(int node, int l, int r) {
        if (lazy[node] != 0) {
            tree[node] += lazy[node] * (r - l + 1);

            if (l != r) {
                lazy[2 * node] += lazy[node];
                lazy[2 * node + 1] += lazy[node];
            }

            lazy[node] = 0;
        }
    }

    void build(int node, int l, int r, const vector<T>& arr) {
        if (l == r) {
            tree[node] = arr[l];
            return;
        }
        int mid = l + (r - l) / 2;
        build(2 * node, l, mid, arr);
        build(2 * node + 1, mid + 1, r, arr);
        tree[node] = combina(tree[2 * node], tree[2 * node + 1]);
    }

    void update(int node, int l, int r, int ql, int qr, T val) {
        push(node, l, r);

        if (ql > r || qr < l) return;

        if (ql <= l && r <= qr) {
            lazy[node] += val;
            push(node, l, r);
            return;
        }

        int mid = l + (r - l) / 2;
        update(2 * node, l, mid, ql, qr, val);
        update(2 * node + 1, mid + 1, r, ql, qr, val);

        tree[node] = combina(tree[2 * node], tree[2 * node + 1]);
    }

    T query(int node, int l, int r, int ql, int qr) {
```

```
        push(node, l, r);

        if (ql > r || qr < l) return neutro;
        if (ql <= l && r <= qr) return tree[node];

        int mid = l + (r - l) / 2;
        return combina(query(2 * node, l, mid, ql, qr),
                       query(2 * node + 1, mid + 1, r, ql, qr));
    }

};

const int N = 2e5+10;

LazySegTree<int> tree(N);
int par[N];
int sz[N];
vi adj[N];

int head[N];
int in[N];
int val[N];
int timer_hld=0;

void dfs(int u, int p){
  par[u]=p;
  sz[u]=1;
  int max_sz=0;
  int heavy_node=-1;
  for(int i=0;i<(int)adj[u].size();i++){
    int v = adj[u][i];
    if(v!=p){
      dfs(v,u);
      sz[u]+=sz[v];
      if(sz[v] > max_sz){
        max_sz=sz[v];
        heavy_node=i;
      }
    }
  }
  if(heavy_node != -1){
    swap(adj[u][0], adj[u][heavy_node]);
  }
}

void dfs_solve(int u, int p, int h){
  head[u] = h;
  in[u] = ++timer_hld;

  for(auto v : adj[u]){
    if(v!=p){
      dfs_solve(v,u,v==adj[u][0]?h:v);
    }
  }
}

int query_path(int u, int n_nodes){
  int ans=0;

  while(head[u] != head[1]){
    ans += tree.query(1,1,n_nodes, in[head[u]],in[u]);
    u=par[head[u]];
  }

  ans += tree.query(1,1,n_nodes, in[1], in[u]);
  return ans;
}

signed main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n,q; cin >> n >> q;

  for(int i=1;i<=n;i++) cin >> val[i];

  for(int i=0;i<n-1;i++){
    int u,v; cin >> u >> v;
    adj[u].pb(v);
    adj[v].pb(u);
  }

  dfs(1,0);
  dfs_solve(1,0,1);

  vi vals2(n+1,0);
  for(int i=1;i<=n;i++){
    vals2[in[i]]=val[i];
  }

  tree.build(1,1,n,vals2);

  while(q--){
    int t; cin >> t;
    if(t==1){
      int s,x; cin >> s >> x;
      int dif = x-val[s];
      val[s]=x;
      tree.update(1,1,n,in[s],in[s],dif);
    }else{
      int s; cin >> s;
      cout << query_path(s,n) << endl;
    }
  }

  return 0;
}
```

14.10 subordinates

```
#include <bits/stdc++.h>
using namespace std;
#define ff first
#define ss second
#define pb push_back
#define int long long
#define endl '\n' // << flush
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cout << #x << " = " << x << endl
#define uniq(x) sort(all(x)); v.erase(unique(all(x)), x.end())
typedef pair<int, int> pi;
typedef long long ll;

// const int MOD = 1e9 + 7; // 998244353
// const int INF = 0x3f3f3f3f;
// const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const int MAXN = 2e5 + 5;

int n, sz[MAXN];
// bool vis[MAXN];
vector<int> adj[MAXN];

void dfs(int v, int p) {
    sz[v] = 1;
    // vis[v] = true;
    for (auto u : adj[v]) if (u != p) {
        dfs(u, v);
        sz[v] += sz[u];
    }
}

void solve() {
    cin >> n;
    for (int i = 1; i < n; i++) {
       int v; cin >> v; v--;
       adj[i].pb(v);
       adj[v].pb(i);
    }
    dfs(0, -1);
    for (int i = 0; i < n; i++) {
        cout << (sz[i]-1) << " ";
    }
    cout << endl;
}
```

```
signed main() {
    int tc = 1;
    // cin >> tc;
    while (tc--) solve();
}
```

14.11 subtree queries

```
#include <bits/stdc++.h>
using namespace std;
#define ff first
#define ss second
#define pb push_back
#define int long long
#define endl '\n' //<< flush
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cerr << #x << " = " << x << endl
#define uniq(v) sort(all(v)); v.erase(unique(all(v)), v.end())
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
typedef long long ll;
typedef pair<int, int> pi;
// const int MOD = 1e9 + 7; // 998244353;
// const int INF = 0x3f3f3f3f;
// const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const int MAXN = 2e5+5;

vector<int> adj[MAXN];
int n, q, timer = 0, dp[MAXN], tin[MAXN], tout[MAXN], a[MAXN];

struct fenw {
    int n;
    vector<int> bit;
    fenw() {}
    fenw(int size) {
        n = size;
        bit.assign(size + 1, 0);
    }
    // query do prefixo a[0] + a[1] + ... + a[r]
    int qry(int r) {
        int ans = 0;
        for (int i = r + 1; i > 0; i -= i & -i) // i & -i retorna os bits menos
             signativos de i
            ans += bit[i];
        return ans;
    }
    // atualiza o valor a[r] = x
    void upd(int r, int x) {
        for (int i = r + 1; i <= n; i += i & -i) bit[i] += x;
    }
};

void dfs(int v, int p) {
    tin[v] = timer++;
    for (auto u : adj[v]) if (u != p) {
        dfs(u, v);
    }
    tout[v] = timer - 1;
}

void solve() {
    cin >> n >> q;
    for (int i = 0; i < n; i++) cin >> a[i];
    for (int i = 0; i < n - 1; i++) {
        int u, v; cin >> u >> v; u--, v--;
        adj[u].pb(v);
        adj[v].pb(u);
    }
    // Podemos usar Euler Tour para processar uma arvore enraizada
    // como um array continua (Fenwick, Seg, ...) -> Podemos fazer upd e range
         queries
    dfs(0, -1);
    // Para cada intervalo [tin[i], tout[i]], temos que todo subintervalo [tin[j
         ], tout[j]]
    // temos que todos os vertices j pertencem a sub-arvore enraizada em i.
    fenw BIT(n);
    for (int i = 0; i < n; i++) BIT.upd(tin[i], a[i]);
    for (int i = 0; i < q; i++) {
        int op; cin >> op;
        if (op == 1) {
            // atualiza o valor do vertice v
            int v, x; cin >> v >> x; v--;
            BIT.upd(tin[v], x - a[v]);
            a[v] = x;
        } else {
            // query na soma da subarvore de v
            // = soma do intervalo [tin[v], tout[v]]
            int v; cin >> v; v--;
            int end = BIT.qry(tout[v]);
            int start = (tin[v] == 0) ? 0 : BIT.qry(tin[v]-1);
            cout << (end - start) << endl;
        }
    }
}

signed main() {
    ios_base::sync_with_stdio(0);cin.tie(0);
    int tc = 1;
    //cin >> tc;
    while(tc--) solve();
}
```

14.12 tree diameter

```
#include <bits/stdc++.h>
using namespace std;
#define ff first
#define ss second
#define pb push_back
#define int long long
#define endl '\n' // << flush
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cout << #x << " = " << x << endl
#define uniq(x) sort(all(x)); v.erase(unique(all(x)), x.end())
typedef pair<int, int> pi;
typedef long long ll;

// const int MOD = 1e9 + 7; // 998244353
// const int INF = 0x3f3f3f3f;
// const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const int MAXN = 2e5 + 5;

int n, dist[MAXN];
vector<int> adj[MAXN];

void dfs(int v, int p) {
    for (auto u : adj[v]) if (u != p) {
        dist[u] = dist[v] + 1;
        dfs(u, v);
    }
}

void solve() {
    cin >> n;
    for (int i = 0; i < n-1; i++) {
       int u, v; cin >> u >> v; u--,v--;
       adj[u].pb(v);
       adj[v].pb(u);
    }

    dfs(0, -1);
    int d = 0, a = 0, b = 0;
    for (int i = 0; i < n; i++) {
        if (dist[i] > d) {
            d = dist[i];
            a = i;
        }
    }
```

```
    memset(dist, 0, sizeof(dist));

    dfs(a, -1);
    d = 0;
    for (int i = 0; i < n; i++) {
        if (dist[i] > d) {
            d = dist[i];
            b = i;
        }
    }

    cout << d << endl;
}

signed main() {
    int tc = 1;
    // cin >> tc;
    while (tc--) solve();
}
```

14.13 tree distances i

```
#include <bits/stdc++.h>
using namespace std;
#define ff first
#define ss second
#define pb push_back
#define int long long
#define endl '\n' // << flush
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cout << #x << " = " << x << endl
#define uniq(x) sort(all(x)); v.erase(unique(all(x)), x.end())
typedef pair<int, int> pi;
typedef long long ll;

// const int MOD = 1e9 + 7; // 998244353
// const int INF = 0x3f3f3f3f;
// const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const int MAXN = 2e5 + 5;

int n;
vector<int> adj[MAXN];

void dfs(int v, int p, int dist[MAXN]) {
    for (auto u : adj[v]) if (u != p) {
        dist[u] = dist[v] + 1;
        dfs(u, v, dist);
    }
}

void solve() {
    int dist[MAXN] = {0}, dist_a[MAXN] = {0}, dist_b[MAXN] = {0};

    cin >> n;
    for (int i = 0; i < n-1; i++) {
       int u, v; cin >> u >> v; u--,v--;
       adj[u].pb(v);
       adj[v].pb(u);
    }
    // Procurando o "a"
    dfs(0, -1, dist);
    int d = 0, a = 0, b = 0;
    for (int i = 0; i < n; i++) {
        if (dist[i] > d) {
            d = dist[i];
            a = i;
        }
    }

    memset(dist, 0, sizeof(dist));
    // Procurando o "b"
    dfs(a, -1, dist);
    d = 0;
    for (int i = 0; i < n; i++) {
        if (dist[i] > d) {
            d = dist[i];
            b = i;
        }
    }
    // Calculando dist_a e dist_b
    dfs(a, -1, dist_a);
    dfs(b, -1, dist_b);
    for (int i = 0; i < n; i++) {
        cout << max(dist_a[i], dist_b[i]) << " ";
    }
    cout << endl;
}

signed main() {
    int tc = 1;
    // cin >> tc;
    while (tc--) solve();
}
```

14.14 tree distances ii

```
#include <bits/stdc++.h>
using namespace std;
#define ff first
#define ss second
#define pb push_back
#define int long long
#define endl '\n' //<< flush
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cerr << #x << " = " << x << endl
#define uniq(v) sort(all(v)); v.erase(unique(all(v)), v.end())
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
typedef long long ll;
typedef pair<int, int> pi;
// const int MOD = 1e9 + 7; // 998244353;
// const int INF = 0x3f3f3f3f;
// const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const int MAXN = 2e5+5;

vector<int> adj[MAXN];
int n, sub[MAXN],  ans[MAXN];

void dfs1(int v, int p, int depth) {
    sub[v] = 1;
    ans[0] += depth;
    for (auto u : adj[v]) if (u != p) {
        dfs1(u, v, depth+1);
        sub[v] += sub[u];
    }
}

void dfs2(int v, int p) {
    for (auto u : adj[v]) if (u != p) {
        ans[u] = ans[v] + (n - 2*sub[u]);
        dfs2(u, v);
    }
}

void solve() {
    cin >> n;
    for (int i = 0; i < n-1; i++) {
        int u, v; cin >> u >> v; u--, v--;
        adj[v].pb(u);
        adj[u].pb(v);
    }
    dfs1(0, -1, 0);
    dfs2(0, -1);
    for (int i = 0; i < n; i++) cout << ans[i] << " ";
    cout << endl;
}

signed main() {
    ios_base::sync_with_stdio(0);cin.tie(0);
    int tc = 1;
    // cin >> tc;
```

```
    while(tc--) solve();
}
```

14.15 tree matching

```
#include <bits/stdc++.h>
using namespace std;
#define ff first
#define ss second
#define pb push_back
#define int long long
#define endl '\n' // << flush
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cout << #x << " = " << x << endl
#define uniq(x) sort(all(x)); v.erase(unique(all(x)), x.end())
typedef pair<int, int> pi;
typedef long long ll;

// const int MOD = 1e9 + 7; // 998244353
// const int INF = 0x3f3f3f3f;
// const ll LINF = 0x3f3f3f3f3f3f3f3fll;
const int MAXN = 2e5 + 5;

int n, dp[2][MAXN];
// bool vis[MAXN];
vector<int> adj[MAXN];

void dfs(int v, int p) {
    dp[0][v] = dp[1][v] = 0;
    // prefixo e sufixo das respostas dos filhos
    vector<int> pref, suff;
    bool leaf = true;
    for (auto u : adj[v]) if (u != p) {
        leaf = false;
        dfs(u, v);
    }
    if (leaf) return;

    for (auto u : adj[v]) if (u != p) {
        pref.pb(max(dp[0][u], dp[1][u]));
        suff.pb(max(dp[0][u], dp[1][u]));
    }

    for (int i = 1; i < (int) pref.size(); i++) {
        pref[i] += pref[i-1];
    }
    for (int i = (int) suff.size() - 2; i >= 0; i--) {
        suff[i] += suff[i+1];
    }

    // Caso eu nao escolha a aresta (v, x), eu tenho dp[0][v] = SOMATORIO DAS
        RESPOSTAS DOS FILHOS
    dp[0][v] = suff[0];

    // Caso eu escolha formar uma aresta (v, x)
    int ci = 0;
    for (auto u : adj[v]) if (u != p) { // percorrer apenas para os filhos de v
        int left = (ci == 0) ? 0 : pref[ci-1];
        int right = (ci == (int) suff.size() - 1) ? 0 : suff[ci+1];
        dp[1][v] = max(dp[1][v], 1 + (left + dp[0][u] + right));
        ci++;
    }
}

void solve() {
    cin >> n;
    for (int i = 0; i < n-1; i++) {
       int u, v; cin >> u >> v; u--,v--;
       adj[u].pb(v);
       adj[v].pb(u);
    }
    dfs(0, -1);
    cout << max(dp[0][0], dp[1][0]) << endl;
}

signed main() {
    int tc = 1;
    // cin >> tc;
    while (tc--) solve();
}
```

15 Dynamic Programming

15.1 DigitDP (12daa6)

```
pair<ll, string> dp[25][2][2][2];
bool vis[25][2][2][2];

void reset() { memset(vis, 0, sizeof(vis)); }

pair<ll, string> rec(string& x, string& y, int idx, int tight_low,
                     int tight_high, int leading_zeros) {
  if (idx == sz(y)) return {leading_zeros ? 0 : 1, ""};
  if (vis[idx][tight_low][tight_high][leading_zeros])
    return dp[idx][tight_low][tight_high][leading_zeros];

  int l_sup = (tight_high == 1) ? y[idx] - '0' : 9;
  int l_inf = (tight_low == 1) ? x[idx] - '0' : 0;

  ll mx_prod = -1;
  string best = "";
  for (int d = l_inf; d <= l_sup; d++) {
    int new_low = (tight_low and d == l_inf);
    int new_high = (tight_high and d == l_sup);
    int new_lz = (leading_zeros and d == 0);

    pair<ll, string> next = rec(x, y, idx + 1, new_low, new_high, new_lz);
    ll prod;
    string num;
    if (new_lz) {
      prod = next.ff;
      num = next.ss;
    } else {
      prod = (ll)d * next.ff;
      num = to_string(d) + next.ss;
    }
    if (prod > mx_prod) {
      mx_prod = prod;
      best = num;
    }
  }
  vis[idx][tight_low][tight_high][leading_zeros] = 1;
  return dp[idx][tight_low][tight_high][leading_zeros] = {mx_prod, best};
}

int main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  string x, y;
  cin >> x >> y;
  string tmp = "";
  for (int i = 0; i < sz(y) - sz(x); i++) tmp += '0';
  x = tmp + x;

  reset();
  pair<ll, string> ans = rec(x, y, 0, 1, 1, 1);
  if (ans.ss == "") ans.ss = "0";

  cout << ans.ss << endl;
  return 0;
}
// https://codeforces.com/gym/100886/problem/G
```

15.2 Knapsack (635934)

```cpp
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 1e2 + 10, MAXW = 1e5 + 10;
int n, w_max, v[MAXN], w[MAXN], dp[MAXN][MAXW];
int solve(int i, int limit) { // O(nW) -> n: num de itens, W: peso maximo
     mochila
    // Chegou no fim do array de itens
    if (i == n) return dp[i][limit] = 0; // ou INF, para min()
    // Chegou no limite de peso
    if (!limit) return dp[i][limit] = 0;
    // Ja foi calculado?
    if (dp[i][limit] != -1) return dp[i][limit];
    // Possibilidade 1: Nao adicionar o elemento i
    dp[i][limit] = solve(i+1, limit);
    // Possibilidade 2: Adicionar o elemento i
    if (limit >= w[i]) {
        // Maximo entre o valor ja calculado e a segunda possibilidade
        dp[i][limit] = max(dp[i][limit], solve(i+1, limit - w[i]) + v[i]);
    }
    return dp[i][limit];
}
signed main(){
    ios_base::sync_with_stdio(0);cin.tie(0);
    memset(dp, -1, sizeof(dp));
    cin >> n >> w_max;
    for (int i = 0; i < n; i++) cin >> w[i] >> v[i];
    cout << solve(0, w_max) << endl;
}
```

15.3 LCS (6490f6)

```cpp
#include <bits/stdc++.h>
using namespace std;
#define endl '\n'
#define int long long
const int MAXN = 3e3 + 10;
string s, t;
int dp[MAXN][MAXN];
int lcs(int n, int m) { // n, m -> tamanho das strings
    if (n == 0 or m == 0) return dp[n][m] = 0;
    if (dp[n][m] != -1) return dp[n][m];
    if (s[n-1] == t[m-1]) { // Faz um match
        return dp[n][m] = 1 + lcs(n-1, m-1);
    } else { // s[n-1] != s[m-1] -> Nao fez um match
        return dp[n][m] = max(lcs(n-1, m), lcs(n, m-1));
    }
}
string get_lcs(int n, int m) {
    string str = "";
    int i = n, j = m;
    while (i > 0 and j > 0) {
        if (s[i-1] == t[j-1]) {
            str += s[i-1];
            i--, j--;
        } else {
            // Caminhamos para a celula de maior valor
            if (dp[i-1][j] > dp[i][j-1]) i--;
            else j--;
        }
    }
    reverse(str.begin(), str.end());
    return str;
}
signed main(){
    ios_base::sync_with_stdio(0);cin.tie(0);
    memset(dp, -1, sizeof(dp));
    cin >> s >> t;
    int n = (int) s.size(), m = (int) t.size();
    lcs(n, m);
    cout << get_lcs(n, m) << endl;
}
```

15.4 LIS BS (78d63a)

```cpp
#include <bits/stdc++.h>
using namespace std;

#define _ ios_base::sync_with_stdio(0);cin.tie(0);
#define endl '\n'
#define all(x) (x).begin(), (x).end()

// O(n*log(n)) --> Solucao TOP
// dp[l] --> e o menor a[i] que a maior LIS de tamanho l termina
// solucao e maior l t.q d[l] nao e INF
// Obs.: Essa solucao nao serve para contar o numero de LIS da a[], precisa usar
     lis-ds.

// int lis(vector<int> const& a) {
vector<int> lis(vector<int> const& a) {
    int n = a.size();
    const int INF = 1e9;
    vector<int> dp(n + 1, INF);
    vector<int> id_dp(n + 1, -1);       // Armazena o indice i do valor dp[l] =
         a[i]
    vector<int> previous(n + 1, -1);    // Armazena o indice do dp[l - 1]

    dp[0] = -INF;

    for (int i = 0; i < n; i++) {
        // Solucao trivial
        // for (int l = 1; l <= i; l++) {
        //     if (dp[l-1] < a[i] && a[i] < dp[l]) {
        //         dp[l] = a[i];
        //     }
        // }

        // dp e estritamente crescente e a[i] atualiza apenas um valor de dp[l]
        // dp[l - 1] < a[i] < dp[l] --> Podemos encontrar o l a partir da Busca
             Binaria
        int l = upper_bound(all(dp), a[i]) - dp.begin();
        if (dp[l-1] < a[i] && a[i] < dp[l]) {
            dp[l] = a[i];
            id_dp[l] = i;
            previous[i] = id_dp[l-1];
        }
    }

    int ans = 0;
    for (int l = 0; l <= n; l++) {
        if (dp[l] < INF) ans = l;
    }

    // Apenas retornar a resposta
    // return ans;

    // Reconstruir a subsequencia
    vector<int> subseq;

    int pos = id_dp[ans];
    while (pos != -1) {
        subseq.push_back(a[pos]);
        pos = previous[pos];
    }

    reverse(all(subseq));
    return subseq;
}

signed main(){ _
    int n; cin >> n;
    vector<int> a(n);
    for (auto &x : a) cin >> x;

    // int ans = lis(a);
    // cout << ans << endl;

    vector<int> ans = lis(a);
    for (auto x : ans) {
        cout << x << " ";
    }
```

```
    cout << endl;
}

/* https://cp-algorithms.com/sequences/longest_increasing_subsequence.html
Considere uma array a[n].

Problema: Nosso objetivo e encontrar a maior subsequencia estritamente crescente
      de a.

Subproblema: Vamos considerar que dp[l] e menor valor que um subsequencia
     estritamente crescente de tamanho l termina.

- dp[0] = -INF (Caso base)

Vamos iniciar dp[l] = INF e vamos processar cada a[i] com 0 <= i <= n

Exemplo: a = {8, 3, 4, 6, 5, 2, 0, 7, 9, 1}

prefix = {}                          --> d = {-INF, INF, INF, ..., INF}
prefix = {8}                         --> d = {-INF, 8, INF, ..., INF}
prefix = {8, 3}                      --> d = {-INF, 3, INF, ..., INF}
prefix = {8, 3, 4}                   --> d = {-INF, 3, 4, ..., INF}
prefix = {8, 3, 4, 6}                --> d = {-INF, 3, 4, 6, ..., INF}
prefix = {8, 3, 4, 6, 5}             --> d = {-INF, 3, 4, 5, ..., INF}
prefix = {8, 3, 4, 6, 5, 2}          --> d = {-INF, 2, 4, 5, ..., INF}
prefix = {8, 3, 4, 6, 5, 2, 0}       --> d = {-INF, 0, 4, 5, ..., INF}
prefix = {8, 3, 4, 6, 5, 2, 0, 7}    --> d = {-INF, 0, 4, 5, 7, ..., INF}
prefix = {8, 3, 4, 6, 5, 2, 0, 7, 9} --> d = {-INF, 0, 4, 5, 7, 9, ..., INF}
prefix = {8, 3, 4, 6, 5, 2, 0, 7, 9, 1} --> d = {-INF, 0, 1, 5, 7, 9, ..., INF}

Quando vamos fazer d[l] = a[i]?
- Quando tiver nenhuma lis de tamanho l que termine em a[i]
- E, se nao tiver nenhuma outra lis de tamanho l que termina em algum numero
     menor que a[i]

Note que ha uma sub-estrutura otima do problema, a solucao para l pode ser
     construida a partir de l - 1

dp[l] = -INF, se l = 0
        min(a[i], d[l]), se a[i] > d[l - 1]
        +INF, caso contrario.
*/
```

15.5 LIS DS (12492d)

```
#include <bits/stdc++.h>
using namespace std;

#define _ ios_base::sync_with_stdio(0);cin.tie(0);
#define endl '\n'
#define all(x) (x).begin(), (x).end()

typedef long long ll;

const ll MOD = 1e9 + 7;

// Coordinate Compreesion
void coordinate_compression(vector<int> &a) {
    set<int> s; // Conjunto para armazenar todos os numeros unicos
    for (auto x : a) s.insert(x);

    int index = 0;
    map<int, int> mp; // Map para armazenar os novos elementos

    set<int>::iterator itr;
    for (itr = s.begin(); itr != s.end(); itr++) {
        index++;
        mp[*itr] = index;
    }

    // Alterando o valor de a
    for (int i = 0; i < a.size(); i++) {
        a[i] = mp[a[i]];
    }
}

// BIT ou SegTree
struct fenw {
    int n;
    vector<int> bit;

    fenw() {}
    fenw(int size) {
        n = size;
        bit.assign(size + 1, 0);
    }
    // query do maior prefixo a[0...r]
    int qry(int r) {
        int ans = 0;
        for (int i = r + 1; i > 0; i -= i & -i) // i & -i retorna os bits menos
             signativos de i
            ans = max(ans, bit[i]);
        return ans;
    }

    // atualiza o valor a[r] = x
    void upd(int r, int x) {
        for (int i = r + 1; i <= n; i += i & -i)
            bit[i] = max(bit[i], x);
    }
};

// O(nlog(n))
int lis(vector<int> &a) {
    coordinate_compression(a);

    int n = a.size();

    fenw tree(n + 1);

    int ans = 0;
    for (int i = 0; i < n; i++) {
        int best = tree.qry(a[i] - 1);
        tree.upd(a[i], best + 1);
        ans = max(ans, best + 1);
    }

    return ans;
}

signed main(){ _

    int n; cin >> n;
    vector<int> a(n);
    for (auto &x : a) cin >> x;

    cout << lis(a) << endl;
}

// https://cp-algorithms.com/sequences/longest_increasing_subsequence.html#
     solution-in-on-log-n-with-data-structures
// Considere t[a[i]] = dp[i] # dp[i] e o maior l de LIS t.q termina com a[i]
/* Para calcular dp[], precisamos fazer:
ans = -INF;
Para i = 0 ate 0:
    dp[i] = max(t[0...a[i]] + 1) // Maior prefixo ate a[i]
    ans = max(ans, dp[i])
    # O problema e buscar o maior prefixo de a[0..i] com a[k]  mudando,
    sendo 0 <= k < i. --> SegTree ou BIT
    # Pontos Negativos da Solucao
        - Implementacao Complexa
        - a[i] for mt grande --> Coordinate Compression ou Dynamic SegTree
*/
```

15.6 SOS DP (d949b2)

```
#include<bits/stdc++.h>
using namespace std;

const int B = 20;
```

```
int a[1 << B], f[1 << B], g[1 << B];
int32_t main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  int n; cin >> n;
  for (int i = 1; i <= n; i++) {
    cin >> a[i];
    f[a[i]]++;
    g[a[i]]++;
  }

  // sum over subsets
  for (int i = 0; i < B; i++) {
    for (int mask = 0; mask < (1 << B); mask++) {
      if ((mask & (1 << i)) != 0) {
        f[mask] += f[mask ^ (1 << i)];
      }
    }
  }

  // sum over supersets
  for (int i = 0; i < B; i++) {
    for (int mask = (1 << B) - 1 ; mask >= 0 ; mask--) {
      if ((mask & (1 << i)) == 0) g[mask] += g[mask ^ (1 << i)] ;
    }
  }

  for (int i = 1; i <= n; i++) {
    cout << f[a[i]] << ' ' << g[a[i]] << ' ' << n - f[(1 << B) - 1 - a[i]] << '\
         n';
  }
  return 0;
}
// https://cses.fi/problemset/task/1654
```

15.7 TSP DP (b73d1f)

```
int main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n, m;
  cin >> n >> m;

  vvi adj(n);
  for (int i = 0; i < m; i++) {
    int u, v;
    cin >> u >> v;
    u--,v--;
    adj[v].pb(u);
  }


  int sz = (1 << n);
  vvi dp(sz, vi(n, 0));
  dp[1][0] = 1;
  for (int mask = 2; mask < sz; mask++) {
    if((mask & 1)==0) continue;
    if((mask & (1<<(n-1))) and mask != ((1<<n)-1)) continue;

    for(int i=1;i<=n;i++){
      if((mask & (1<<i))==0) continue;

      int prev = mask ^ (1<<i);
      for(auto v : adj[i]){
        if((mask & (1 << v))){
          dp[mask][i] += dp[prev][v];
          dp[mask][i] %= MOD;
        }
      }
    }
  }
  cout << dp[sz - 1][n-1] << endl;

  return 0;
}
// TSP
```

15.8 TreeDistances2 (fd0774)

```
const int N = 2e5+5;
vi adj[N];
ll dp[N];
ll sum[N];
ll ans[N];
int s;

// Objetive: compute for each node the sum of the distances from the other nodes
// Dfs1: compute depth (dp)
void dfs1(int u, int p){
    sum[u]=1;
    for(auto v : adj[u]) if(v!=p){
        dfs1(v,u);
        dp[u] += dp[v]+sum[v];
        sum[u] += sum[v];
    }
}

// DFS2: for each node compute the total sum
// f(x) = sum of parent - dp
void dfs2(int u, int p){
    ans[u] = dp[u];
    for(auto v : adj[u]){
        if(v!=p){
            dp[u] -= dp[v] + sum[v];
            sum[u] -= sum[v];

            dp[v] += dp[u] + sum[u];
            sum[v] += sum[u];
            dfs2(v,u);

            sum[v] -= sum[u];
            dp[v] -= dp[u] + sum[u];

            sum[u] += sum[v];
            dp[u] += dp[v] + sum[v];

        }
    }
}

int main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n; cin >> n;
  for(int i=0;i<n-1;i++){
      int u,v; cin >> u >> v;
      adj[u].pb(v);
      adj[v].pb(u);
  }

  dfs1(1,0);

  dfs2(1,0);
  for(int i=1;i<=n;i++) cout << ans[i] << " ";
  cout << endl;

  return 0;
}
```

16 Geometry

16.1 ConvexHull (e8ad2a)

```
#include <bits/stdc++.h>
using namespace std;
```

```
#define int long long int

struct pt {
    int x, y, id;
    pt(int x, int y, int id) : x(x), y(y), id(id) {}
    pt() {}
    pt operator-(pt const &o) const {
        return {x - o.x, y - o.y, -1};
    }
    bool operator<(pt const &o) const {
        if (x == o.x) {
            return y < o.y;
        }
        return x < o.x;
    }
    int operator^(pt const &o) const {
        return (int)x * o.y - (int)y * o.x;
    }
};
int ccw(pt const &a, pt const &b, pt const &x) {
    auto p = (b - a) ^ (x - a);
    return (p > 0) - (p < 0);
}
vector<pt> convex_hull(vector<pt> P, bool include_collinear) {
        // include_collinear: define se vai retornar os pontos colineares as
             bordas
        // ou seja, pontos da aresta do poligono
    sort(P.begin(), P.end());
    vector<pt> L, U;
    int pop_threshold = include_collinear ? -1 : 0;
    for (auto p : P) {
        while (L.size() >= 2 && ccw(L.end()[-2], L.end()[-1], p) <=
             pop_threshold) {
            L.pop_back();
        }
        L.push_back(p);
    }
    reverse(P.begin(), P.end());
    for (auto p : P) {
        while (U.size() >= 2 && ccw(U.end()[-2], U.end()[-1], p) <=
             pop_threshold) {
            U.pop_back();
        }
        U.push_back(p);
    }
    L.insert(L.end(), U.begin() + 1, U.end() - 1);
    return L;
}
signed main(){
    int n; cin >> n;
    vector<pt> arr;
    for (int i = 0; i < n; i++) {
                int x, y; cin >> x >> y;
                arr.emplace_back(x, y, i+1);
        }
        vector<pt> ans = convex_hull(arr, true);
        for (auto [x, y, id] : ans) {
                cout << x << " " << y << " " << id << '\n';
        }
}
```

16.2 GeometriaInt (e08340)

```
#include <bits/stdc++.h>
using namespace std;
/* PRIMITIVAS */
#define int long long
#define sq(x) ((x)*(int)(x))
struct pt {
    int x, y;
    pt(int a=0, int b=0) : x(a), y(b) {}
    friend istream &operator>>(istream &in, pt &p) {
                int x, y; in >> p.x >> p.y; return in;
        }
    bool operator < (const pt p) const {
                if (x != p.x) return x < p.x; return y < p.y;
        }
        bool operator == (const pt p) const {
                return x == p.x and y == p.y;
        }
        pt operator + (const pt p) const { return pt(x+p.x, y+p.y); }
        pt operator - (const pt p) const { return pt(x-p.x, y-p.y); }
    // multiplicar um escalar ao vetor
        pt operator * (const int c) const { return pt(x*c, y*c); }
    // produto escalar
        int operator * (const pt p) const { return x*(int)p.x + y*(int)p.y; }
    // produto vetorial (norma de u x v)
    int operator ^ (const pt p) const { return x*(int)p.y - y*(int)p.x; }
};
struct line {
    pt p, q;
    line() {}
    line(pt _p, pt _q) : p(_p), q(_q) {}
    friend istream &operator>>(istream &in, line &r) {
        int x, y; in >> r.p >> r.q; return in;
    }
};

/* PONTO e VETOR */
int dist2(pt p, pt q) { return sq(p.x - q.x) + sq(p.y - q.y); }
// area do paralelogramo formado pelos vetores p->q e p->r
// dividir por 2 retorna a area do triangulo formado pelos pontos
int area2(pt p, pt q, pt r) { return (q - p) ^ (r - q); }
// verifica se p, q, r sao colineares
bool col(pt p, pt q, pt r) { return area2(p, q, r) == 0; }
// verifica se p -> q -> r e no sentido anti-horario
bool ccw(pt p, pt q, pt r) { return area2(p, q, r) > 0; }
// quadrante de um ponto -> 1, 2, 3, 4
int quad(pt p) { return (p.x<0)^3*(p.y<0); }
// retorna se ang(p) < ang(q) -> Radial Sort (ordena pelos angulos)
bool compare_angle(pt p, pt q) {
        if (quad(p) != quad(q)) return quad(p) < quad(q);
        return ccw(q, pt(0, 0), p);
}
// rotaciona 90 graus o vetor
// vetor resultante e o normal ao original
pt rotate90(pt p) { return pt(-p.y, p.x); }

/* RETA */
// verifica se P pertence ao segmento R
bool in_seg(pt p, line r) {
    pt a = r.p - p, b = r.q - p;
    // (a * b) <= garante que p esta entre [r.p, r.q]
    return (a ^ b) == 0 and (a * b) <= 0;
}
// se o seg de r intersecta o seg de s
bool inter_seg(line r, line s) {
        if (in_seg(r.p, s) or in_seg(r.q, s)
                or in_seg(s.p, r) or in_seg(s.q, r)) return 1;
        return ccw(r.p, r.q, s.p) != ccw(r.p, r.q, s.q) and
                        ccw(s.p, s.q, r.p) != ccw(s.p, s.q, r.q);
}
// numero de pontos inteiros no segmento
int segpoints(line r) {
        return 1 + __gcd(abs(r.p.x - r.q.x), abs(r.p.y - r.q.y));
}
// retorna t tal que t*v pertence a reta r
double get_t(pt v, line r) {
        return (r.p^r.q) / (double) ((r.p-r.q)^v);
}

/* POLIGONOS */
// Shoelace Theorem: 2*area
int polarea2(vector<pt> v) {
    int ret = 0;
    for (int i = 0; i < v.size(); i++)
        ret += area2(pt(0, 0), v[i], v[(i+1) % v.size()]);
    return abs(ret);
}
// Verifica se um ponto esta dentro, fora ou na borda de um poligono
// 0 -> fora, 1 -> dentro e 2 -> na borda
int inpol(vector<pt>& v, pt p) { // O(|v|) -> raycasting
```

```
    int qt = 0;
    for (int i = 0; i < v.size(); i++) {
        if (p == v[i]) return 2;
        int j = (i + 1) % v.size();
        if (p.y == v[i].y and p.y == v[j].y) {
                    if ((v[i]-p)*(v[j]-p) <= 0) return 2;
            }
        bool baixo = v[i].y < p.y;
                if (baixo == (v[j].y < p.y)) continue;
                auto t = (p-v[i])^(v[j]-v[i]);
                if (!t) return 2;
                if (baixo == (t > 0)) qt += (baixo ? 1 : -1);
    }
    return qt != 0;
}
```

16.3 SweepLine (9d88c7)

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'
#define ENTRADA 0
#define SAIDA 1

typedef struct Evento {
    int tempo, tipo, id;
    Evento(int _tempo, int _tipo, int _id) : tempo(_tempo), tipo(_tipo), id(_id)
          {}
    // O elemento id e util quando temos que responder queries
    bool operator < (Evento outro) {
        if(tempo == outro.tempo) return tipo < outro.tempo; // Definir o
            criterio de desempate
        return tempo < outro.tempo;
    }
} evento_t;

int main() {
    int n; cin >> n;
    vector<evento_t> ev;
    // Leitura dos eventos
    for (int i = 0; i < n; i++) {
        int e, s; cin >> e >> s;
        ev.push_back({e, ENTRADA, i}); // Evento de Entrada
        ev.push_back({s, SAIDA, i});   // Evento de Saida
    }
    // Pre-ordenacao para Varredura
    sort(ev.begin(), ev.end());
    // Varredura --> Verificar qual o maximo de pessoas
    int acc = 0, ans = -1;
    for (auto [tempo, tipo, id] : ev) {
        if (tipo == ENTRADA) acc++;
        else acc--;
        ans = max(ans, acc);
    }
    cout << ans << endl;
    return 0;
}

/*
Problemas de sweep line sao descritas por duas variaveis:
- Localizacao temporal ou espacial;
- Tipo do evento

O problema esta em saber qual a melhor estrutura para armazenar a resposta (um
     inteiro, um set, uma Segment Tree, ...)

https://noic.com.br/materiais-informatica/curso/line-sweep/
*/
```

17 Graph

17.1 ArticulationPoints (4eb4d2)

```
int T, low[N], dis[N], art[N];
vector<int> g[N];
void dfs(int u, int pre = 0) {
  low[u] = dis[u] = ++T;
  int child = 0;
  for(auto v: g[u]) {
    if(!dis[v]) {
      dfs(v, u);
      low[u] = min(low[u], low[v]);
      if(low[v] >= dis[u] && pre != 0) art[u] = 1;
      ++child;
    }
    else if(v != pre) low[u] = min(low[u], dis[v]);
  }
  if(pre == 0 && child > 1) art[u] = 1;
}
```

17.2 BFS (ecec15)

```
#include <bits/stdc++.h>
using namespace std;
#define pb push_back
const int MAXN = 1e5 + 10;
int n, m;
vector<int> adj[MAXN], dist(MAXN, -1), parent(MAXN, -1);
bool vis[MAXN];
// BFS Multisource -> bfs(vector<pi> ms): permite fazer BFS em varias fontes
// Encontrar o menor ciclo de um grafo ou menor ciclo que possui um vertice
// Encontrar a menor distancia de s para todos os outros vertices
void bfs(int s) { // O(V + E)
    queue<int> q;
    q.push(s); vis[s] = true;
    dist[s] = 0;
    parent[s] = -1;
    while (!q.empty()) {
        int v = q.front(); q.pop();
        for (auto u : adj[v]) if (!vis[u]) {
            q.push(u); vis[u] = true;
            dist[u] = dist[v] + 1;
            parent[u] = v;
        }
    }
}
// https://cp-algorithms.com/graph/breadth-first-search.html
signed main() {
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int u, v; cin >> u >> v;
        adj[u].pb(v);
        adj[v].pb(u);
    }
    bfs(0);
    // Mostrar o caminho de s ate u
    int u; cin >> u;
    if (!vis[u]) {
        cout << "No path" << endl;
    } else {
        stack<int> path;
        for (int v = u; v != -1; v = parent[v]) {
            path.push(v);
        }
        // Exibir caminho
        cout << "Path: ";
        while (!path.empty()) {
            cout << path.top() + 1 << " ";
        }
        cout << endl;
    }
}
```

17.3 BellmanFord (03059b)

```
int n, m;
int d[MAX];
vector<pair<int, int>> ar; // vetor de arestas
vector<int> w;             // peso das arestas

bool bellman_ford(int a) {
        for (int i = 0; i < n; i++) d[i] = INF;
        d[a] = 0;

        for (int i = 0; i <= n; i++)
                for (int j = 0; j < m; j++) {
                        if (d[ar[j].second] > d[ar[j].first] + w[j]) {
                                if (i == n) return 1;

                                d[ar[j].second] = d[ar[j].first] + w[j];
                        }
                }

        return 0;
}
```

17.4 BinaryLifting (b9bccc)

```
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 2e5 + 10;
const int LOG = 20; // 2^20 ~= 1e6
int n, q, parent[MAXN], up[MAXN][LOG];
vector<int> adj[MAXN];
void binary_lifting() { // O(nlog(n))
    // Calcula o primeiro ancestral
    for (int v = 0; v < n; v++)
        up[v][0] = parent[v];
    // Calcular o 2 ate k-th ancestral
    for (int j = 1; j < LOG; j++)
        for (int v = 0; v < n; v++)
            if(up[v][j-1] == -1)
                up[v][j] = -1;
            else
                up[v][j] = up[ up[v][j-1] ][j-1];
}
int kth_ancestor(int v, int k) { // O(log(n))
    for (int j = 0; j < LOG; j++) {
        if (v == -1) break;
        if (k & (1LL << j)) v = up[v][j];
    }
    return v;
}
void solve() {
    memset(parent, -1, sizeof(parent));
    cin >> n >> q;
    for (int v = 1; v < n; v++) {
        int u; cin >> u; u--;
        adj[v].pb(u);
        adj[u].pb(v);
        parent[v] = u;
    }
    binary_lifting();
    // Responder as queries
    for (int i = 0; i < q; i++) {
        int x, k; cin >> x >> k; x--;
        int ans = kth_ancestor(x, k);
        if (ans == -1) cout << ans;
        else cout << ans+1;
        cout << endl;
    }

}
```

17.5 Bipartide (3e995c)

```
#include <bits/stdc++.h>
using namespace std;
#define pb push_back
const int MAXN = 2e5+5;
bool vis[MAXN];
int n, m, color[MAXN];
vector<int> adj[MAXN];
// Teorema: um grafo e bipartido sse todos os seus ciclos tem tamanho par.
// um grafo e bipartido sse ele e two-colorable.
bool dfs(int v) {
    for (auto u : adj[v]) {
        if (color[u] == -1) {
            color[u] = (color[v] == 1) ? 2 : 1;
            if (!dfs(u)) return false;
        } else if (color[u] == color[v]) {
            return false; // conflito -> nao bipartido
        }
    }
    return true;
}
signed main() {
    memset(color, -1, sizeof(color));
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int u, v; cin >> u >> v; u--, v--;
        adj[u].pb(v);
        adj[v].pb(u);
    }
    bool valid = true;
    for (int i = 0; i < n; i++) {
        if (color[i] == -1) {
            color[i] = 1;
            if (!dfs(i)) {
                valid = false;
                break;
            }
        }
    }
    if (valid) {
        for (int v = 0; v < n; v++) cout << color[v] << " ";
        cout << endl;
    } else {
        cout << "IMPOSSIBLE" << endl;
    }
}
```

17.6 Bridges (d768b0)

```
#include <bits/stdc++.h>
using namespace std;
#define pb push_back
const int MAXN = 2e5 + 5;
vector<int> adj[MAXN];
vector<pair<int, int>> ans;
// dp[u] -> qtd de back-edges que passam por cima da aresta (parent[u], u)
// lvl[u] -> nivel do vertice u
int n, dp[MAXN], lvl[MAXN], parent[MAXN];
void dfs(int v) {
    dp[v] = 0;
    for (auto u : adj[v]) {
        if (u == parent[v]) continue;
        if (lvl[u] == 0) {
            parent[u] = v;
            lvl[u] = lvl[v]+1;
            dfs(u);
            dp[v] += dp[u];
        } else if (lvl[u] < lvl[v]) { // aresta acima
            dp[v]++;
        } else if (lvl[u] > lvl[v]) { // aresta abaixo
            dp[v]--;
        }
    }
```

```
    if (parent[v] != -1 and dp[v] == 0) {
        // Encontrou uma ponte
        ans.pb({min(v, parent[v]), max(v, parent[v])});
    }
}
void find_bridges() {
    memset(parent, -1, sizeof(parent));
    for (int v = 0; v < n; v++) {
        if (lvl[v] == 0) {
            lvl[v] = 1; // marca como visitado
            dfs(v);
        }
    }
}
// https://onlinejudge.org/external/7/796.pdf
/*
Ponte e uma aresta que caso seja removida transforma o grafo em desconexo
- Back-edge e uma aresta (u, v) que conecta u (pai) com v (descendente de u).
- Span-edge e uma aresta (u, v) marcada na DFS, representa uma DFS Tree do Grafo
     G.
# Observacoes
1) Uma span-edge (u, v) e uma ponte <-> nao existe nenuma back-edge que conecta
    algum ancestral de uv
                                        com algum descendente de uv.
                                    <-> nao existe um back-edge que "atravessa
                                         por alto" uv.
2) A back-edge nao e nunca ponte.
# Algoritmo:
Dado um grafo G:
1) Construa a DFS Tree de G
2) Para toda span-edge (u, v), caso nao existe nenhuma back-edge que "atravessa
    por alto" (u, v),
                                (u, v) e uma ponte.
*/
```

17.7 Cycles (b74aae)

```
#include <bits/stdc++.h>
using namespace std;
#define pb push_back
const int MAX = 1e5 + 10;
int n, m;
vector<int> adj[MAX];
int parent[MAX], cycle_start, cycle_end;
bool vis[MAX]; // Grafo nao-direcionado
int color[MAX]; // Grafo direcionado
// Grafo nao-direcionado
bool dfs(int v) {
    vis[v] = true;
    for (auto u : adj[v]) {
        if (u==parent[v]) continue;
        if (vis[u]) {
            cycle_end = v;
            cycle_start = u;
            return true;
        }
        parent[u] = v;
        if (dfs(u)) return true;
    }
    return false;
}
// Grafo direcionado
bool dfs(int v) {
    color[v] = 1;
    for (int u : adj[v]) {
        if (color[u] == 0) {
            parent[u] = v;
            if (dfs(u))
                return true;
        } else if (color[u] == 1) {
            cycle_end = v;
            cycle_start = u;
            return true;
        }
    }
    color[v] = 2;
    return false;
}

signed main() {
    cycle_start = -1;
    memset(parent, -1, sizeof(parent));
    memset(color, 0, sizeof(color));
    // Grafo nao-direcionado
    for (int v = 0; v < n; v++) {
        // break -> encontrou um ciclo
        if (!vis[v] and dfs(v)) break;
    }
    // Grafo direcionado
    for (int v = 0; v < n; v++) {
        // break -> encontrou um ciclo
        if ((color[v]==0) and dfs(v)) break;
    }
    // Exibindo o ciclo
    if (cycle_start = -1) {
        cout << "Acyclic" << endl;
    } else {
        vector<int> cycle;
        cycle.pb(cycle_start);
        for (int v = cycle_end; v != cycle_start; v = parent[v])
            cycle.pb(v);
        cycle.pb(cycle_start);
}
// https://cp-algorithms.com/graph/finding-cycle.html
```

17.8 DFS (bf2c67)

```
#include <bits/stdc++.h>
using namespace std;
#define pb push_back
#define MAXN 100005
vector<int> adj[MAXN];
bool visited[MAXN];
int n, m;
// O(v + E)
void dfs(int v) { // dfs(int v, int p=-1) -> Arvores
    visited[v] = true;
    // for (auto u : adj[v]) if (u != p) { -> Arvores
    for (auto u : adj[v]) if (!visited[u]) {
        dfs(u);
    }
}
// https://cp-algorithms.com/graph/depth-first-search.html
```

17.9 Diameter (efdec1)

```
#include <bits/stdc++.h>
using namespace std;
#define pb push_back
const int MAXN = 2e5+5;
int n, dist[MAXN];
vector<int> adj[MAXN];
void dfs(int v, int p) {
    for (auto u : adj[v]) if (u != p) {
        dist[u] = dist[v] + 1;
        dfs(u, v);
    }
}
void solve() {
    cin >> n;
    for (int i = 0; i < n-1; i++) {
       int u, v; cin >> u >> v; u--,v--;
       adj[u].pb(v);
       adj[v].pb(u);
    }
    dfs(0, -1);
```

```
    int d = 0, a = 0, b = 0;
    for (int i = 0; i < n; i++) {
        if (dist[i] > d) {
            d = dist[i];
            a = i;
        }
    }
    memset(dist, 0, sizeof(dist));
    dfs(a, -1);
    d = 0;
    for (int i = 0; i < n; i++) {
        if (dist[i] > d) {
            d = dist[i];
            b = i;
        }
    }
    cout << a << " " << b << endl;
    cout << d << endl;
```

17.10 Dijkstra (df96ee)

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
typedef pair<int, int> pi;
const int INF = 0x3f3f3f3f3f3f3f3fll;
const int MAX = 1e5 + 10;
int n, m;
vector<pi> adj[MAX]; // {vizinho, peso}
bool vis[MAX]; int dist[MAX], parent[MAX];
void dijkstra(int s) {
    fill(vis, vis+MAX, false);
    fill(dist, dist+MAX, INF);
    // priority_queue de {distancia, vertice}
    priority_queue<pi, vector<pi>, greater<pi>> pq;
    pq.push({0, s}); dist[s] = 0; parent[s] = -1;
    while (!pq.empty()) {
        auto [d, v] = pq.top(); pq.pop();
        if (vis[v]) continue;
        vis[v] = true;
        for (auto [u, w] : adj[v]) if (dist[u] > dist[v] + w) {
            dist[u] = dist[v] + w;
            parent[u] = v;
            pq.push({dist[u], u});
        }
    }
}
signed main(){
    ios_base::sync_with_stdio(0);cin.tie(0);
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int a, b, c; cin >> a >> b >> c; a--, b--;
        adj[a].push_back({b, c});
    }
}
// https://cp-algorithms.com/graph/dijkstra.html
```

17.11 EulerTour (865bc4)

```
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 2e5 + 10;
int n, timer=1, tin[MAXN], tout[MAXN], euler[2*MAXN+1];
vector<vector<int>> adj;
// https://courses.edx.org/asset-v1:ITMOx+I2CPx+3T2016+type@asset+block/lecture
    -04.pdf
void dfs(int v) {
    tin[v] = timer;
    euler[timer++] = v;
    // for (auto u : adj[v]) if (u != p) { // Arvore
    for (auto u : adj[v]) if (tin[u] == -1) {
        dfs(u);
    }
    tout[v] = timer;
    euler[timer++] = v;
}
// Verifica se A e ancestral de B
bool is_ancestor(int a, int b) {
    // A e ancestral de B <-> tin[a] < tout[b] < tout[a]
    return (tin[a] < tout[b]) and (tout[b] < tout[a]);
}
int main() {
    memset(tin, -1, sizeof(tin));
    n = 9;
    adj = {
        {2, 1},          // A
        {0, 5, 6},       // B
        {0, 3, 4},       // C
        {2, 4},          // D
        {2, 3},          // E
        {1, 6},          // F
        {1, 5, 7, 8},    // G
        {6, 8},          // H
        {6, 7}           // J
    };
    dfs(0);
    for (int i = 0; i < n; i++) {
        char c = i+'A';
        cout << c << ": tin = " << tin[i]
             << ", tout = " << tout[i] << endl;
    }
    for (int i = 1; i < timer; i++) {
        char c = euler[i]+'A';
        cout << c << " ";
    }
    cout << endl;
    return 0;
}
```

17.12 FindingConnectComponents (7a0e22)

```
#include <bits/stdc++.h>
using namespace std;
#define pb push_back
const int MAXN = 1e5 + 10;
vector<int> adj[MAXN], comp;
bool vis[MAXN];
int n, m;

void dfs(int v) {
    vis[v] = true;
    comp.pb(v);
    for (auto u : adj[v]) if (!vis[u]) {
        dfs(u);
    }
}

signed main() {
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int u, v; cin >> u >> v;
        adj[u].pb(v);
        adj[v].pb(u);
    }
    // Encontrar os componentes conexos (O(V + E))
    for (int v = 0; v < n; v++) {
        if (!vis[v]) {
            comp.clear();
            dfs(v);
            // Percorrendo no componente conexo
            cout << "Component: ";
            for (auto u : comp) {
                cout << u << " ";
            }
            cout << endl;
        }
    }
```

```
}
// https://cp-algorithms.com/graph/search-for-connected-components.html
```

17.13 FloydWarshall (f8ebbc)

```
#include <bits/stdc++.h>
using namespace std;

const int INF = 0x3f3f3f3f;

#define MAXN 10000

// n -> numero de vertices, m -> numero de arestas
int n, m;
// w[i][j] -> matriz com o peso da aresta (i, j)
int w[MAXN][MAXN];
// dist[i][j] -> matriz com a menor distancia entre os vertices i e j
int dist[MAXN][MAXN];

void initialize() {
    for (int i = 0 ; i < n ; i++) {
        for (int j = 0 ; j < n ; j++) {
            if (i == j) {
                dist[i][j] = 0;
            } else {
                dist[i][j] = INF ;
            }
        }
    }
}
void floyd_warshall() {
    for (int k = 0 ; k < n ; k++) {
        for (int i = 0 ; i < n ; i++) {
            for (int j = 0 ; j < n ; j++) {
                dist[i][j] = min(dist[i][j] , dist[i][k] + dist[k][j]) ;
            }
        }
    }
}
signed main() {
    cin >> n >> m;
    initialize();
    for (int i = 0; i < m; i++){
        // a, b -> vertices e c -> custo da aresta (a, b)
        int a, b, c; cin >> a >> b >> c;
        w[a][b] = c;
        dist[a][b] = min(dist[a][b], c);
        // Comente as linhas abaixo, caso o Grafo seja direcionado
        w[b][a] = c;
        dist[b][a] = min(dist[b][a], c);
    }
    floyd_warshall();
}
```

17.14 Hopcroft (d41d8c)

17.15 Kuhn (b585c2)

```
#include <bits/stdc++.h>
using namespace std;
#define pb push_back
#define all(x) (x).begin(), (x).end()
typedef vector<int> vi;
mt19937 rng((int) chrono::steady_clock::now().time_since_epoch().count());
// Algoritmo de Kuhn para encontrar Emparelhamento Maximo (Maximum Matching) em
    O(VE)
struct kuhn {
    // observacao: A = {0, 1, ..., n} e B = {0, 1, ..., m}
    // ou seja estao 0-indexed
    // n -> esquerda, m -> direita
    int n, m;
    // adj[u] = vizinhos (da direita) do esquerdo u
    vector<vi> adj;
    // ma, mb -> o emparelhamento em si
    vi ma, mb;
    vector<bool> vis;
    kuhn(int n_, int m_) : n(n_), m(m_), adj(n),
        vis(n+m), ma(n, -1), mb(m, -1) {}
    void add(int a, int b) {adj[a].pb(b);}
    bool dfs(int v) {
        vis[v] = true;
        for (auto u : adj[v]) if (!vis[u+n]) {
            vis[u+n] = true;
            if (mb[u] == -1 or dfs(mb[u])) {
                // u esta livre ou consigo realocar o dono atual de u?
                ma[v] = u, mb[u] = v;
                return true;
            }
        }
        return false;
    }
    // retorna o tamanho do emparelhamento maximo
    // Para recuperar o matching, basta olhar 'ma' e 'mb'
    int matching() { // O(V*E)
        int ret = 0;
        for (auto &x : adj) shuffle(all(x), rng); // heuristica
        bool aum = true;
        while (aum) {
            for (int u = 0; u < m; u++) vis[u+n] = 0;
            aum = false;
            for (int v = 0; v < n; v++)
                if (ma[v] == -1 and dfs(v)) ret++, aum = true;
        }
        return ret;
    }
};
// recover() responde o problema de minimum vertex cover
// quantidade minima de vertices escolhidos que garante que todas arestas estao
    sendo "observadas"
// Segundo o Teorema de Konig, a cardinalidade do minimum vertex cover e o
    emparelhamento maximo
// esse problema e NP-Dificil para grafos no geral e e possivel resolver em
    tempo polinomial para bipartidos
pair<vi, vi> recover(kuhn &k) { // O(V*E)
    k.matching();
        int n = k.n, m = k.m;
        for (int i = 0; i < n+m; i++) k.vis[i] = 0;
        for (int i = 0; i < n; i++) if (k.ma[i] == -1) k.dfs(i);
        vector<int> ca, cb;
        for (int i = 0; i < n; i++) if (!k.vis[i]) ca.push_back(i);
        for (int i = 0; i < m; i++) if (k.vis[n+i]) cb.push_back(i);
    // {caras da particao A, caras da particao B}
        return {ca, cb};
}
// podemos responder o minimum edge cover (qtd minima de arestas escolhidas que
    garante que todos
// os vertices sao observados) com o Teorema de Gallai:
// minimum edge cover = total de vertices - emparelhamento maximo
// https://codeforces.com/group/Acy2lotTJD/contest/700424/problem/B
signed main() {
    // o grafo precisa ser bipartido
    // n = esquerda, m = direita, e = qtd de arestas
    int n, m, e; cin >> n >> m >> e;
    kuhn k(n, m);
    for (int i = 0; i < e; i++) {
        int a, b; cin >> a >> b; a--, b--;
        k.add(a, b);
    }
    // emparelhamento maximo
    cout << k.matching() << endl;
    // mostrar as arestas escolhidas
    for (int i = 0; i < n; i++) {
        if (k.ma[i] != -1) {
            cout << i << " " << k.ma[i] << endl;
        }
    }
```

```
    // se quiser a cobertura minima por vertices:
    // auto [ca, cb] = recover(K);
    // se quiser a cobertura minima por arestas:
    // cout << (n+m) - k.matching() << endl;
}
/*
# Definicoes
Dado um grafo bipartido G(n, m), o maximum match (emparelhamento maximo)
consiste na quantidade de aresta uv que podemos escolher de tal forma que
os vertices u, v nao pertencem a nenhuma outra aresta escolhida.
- um vertice e saturado se ele pertence a uma aresta de algum matching M
- um caminho ALTERNANDO e um caminho onde as arestas pertencem ou nao (
    alternando) a um match M.
- um caminho AUMENTADO e um caminho ALTERNANDO onde os vertices do inicio e fim
    sao nao-saturados.
Berge's Lemma
-> Um matching M e maximo sse nao existe nenhum caminho AUMENTADO no matching
O algoritmo de Kuhn e uma aplicacao do Berge's Lemma
https://cp-algorithms.com/graph/kuhn_maximum_bipartite_matching.html
*/
```

17.16 LCA (35e6a7)

```
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 2e5+5;
const int LOG = 20;
int n, q, up[MAXN][LOG], depth[MAXN];
vector<int> adj[MAXN];
void dfs(int v, int p) { // O(nlog(n))
    for (auto u : adj[v]) if (u != p) {
        depth[u] = depth[v] + 1;
        up[u][0] = v;
        for (int j = 1; j < LOG; j++) {
            up[u][j] = up[ up[u][j-1] ][j-1];
        }
        dfs(u, v);
    }
}
// Binary Lifting -> Encontrar o kth ancestor O(logn)
int get_kth(int x, int k) {
    if (k > depth[x]) return -1;
    for (int i = 0; i < LOG; i++) {
        if (k & (1 << i))
            x = up[x][i];
    }
    return x;
}
// Encontrar o LCA O(log(n))
int get_lca(int a, int b) {
    if (depth[a] < depth[b]) {
        swap(a, b);
    }
    int k = depth[a] - depth[b];
    for (int j = LOG-1; j >= 0; j--) {
        if (k & (1 << j)) {
            a = up[a][j];
        }
    }
    if (a == b) {
        return a;
    }
    for (int j = LOG-1; j >= 0; j--) {
        if (up[a][j] != up[b][j]) {
            a = up[a][j];
            b = up[b][j];
        }
    }
    return up[a][0];
}
void solve() {
    cin >> n >> q;
    for (int v = 1; v < n; v++) {
        int u; cin >> u; u--;
        adj[v].pb(u);
        adj[u].pb(v);
    }
    dfs(0, -1);
    for (int i = 0; i < q; i++) {
        // int a, b; cin >> a >> b; a--, b--;
        // cout << (get_lca(a, b) + 1) << endl;
        int x, k; cin >> x >> k; x--;
        int ans = get_kth(x, k);
        if (ans == -1) cout << ans;
        else cout << ans+1;
        cout << endl;
    }
}
```

17.17 MST (0308e2)

```
// No problema abaixo, pedia para encontrar a Maximum Spanning Tree
// Caso fosse pedido a Minimum Spanning Treee, a diferenca seria no loop i
struct DSU {
  vector<int> par, rnk, sz;
  int c;
  DSU(int n) : par(n + 1), rnk(n + 1, 0), sz(n + 1, 1), c(n) {
    for (int i = 1; i <= n; ++i)
      par[i] = i;
  }
  int find(int i) { return (par[i] == i ? i : (par[i] = find(par[i]))); }
  bool same(int i, int j) { return find(i) == find(j); }
  int get_size(int i) { return sz[find(i)]; }
  int count() { return c; }
  bool merge(int i, int j) {
    if ((i = find(i)) == (j = find(j)))
      return 0;
    else
      --c;
    if (rnk[i] > rnk[j])
      swap(i, j);
    par[i] = j;
    sz[j] += sz[i];
    if (rnk[i] == rnk[j])
      rnk[j]++;
    return 1;
  }
};

int main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);
  int n; cin >> n;
  DSU dsu(n+1);
  vi id(1<<20,0);
  for(int i=1;i<=n;i++){
    int x; cin >> x;
    id[x]=i;
  }

  ll max_w=0;
  for(int i=1e6;i>=1;i--){
    int lst=-1;
    for(int j=i;j<=1e6;j+=i){
        if(id[j]>0){
            if(lst>0 and !dsu.same(lst,id[j])){
                dsu.merge(lst,id[j]);
                max_w+=i;
            }
            lst=id[j];
        }
    }
  }
  cout<<max_w<<endl;
```

17.18 MaximumBipartiteMatching (9bdd45)

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Kuhn {
    int n, m;
    vector<vector<int>> adj;
    vector<int> match; // match[i] guarda o vertice da ESQUERDA conectado ao
         vertice 'i' da DIREITA
    vector<bool> used; // Marca se o vertice da ESQUERDA ja foi visitado na DFS
         atual

    Kuhn(int n, int m) : n(n), m(m), adj(n), match(m, -1), used(n, false) {}

    void addEdge(int u, int v) {
        adj[u].push_back(v);
    }

    bool dfs(int v) {
        if (used[v]) return false;
        used[v] = true;
        for (int to : adj[v]) {
            if (match[to] == -1 || dfs(match[to])) {
                match[to] = v;
                return true;
            }
        }
        return false;
    }

    int maxMatching() {
        int matches = 0;
        vector<bool> used_left(n, false);
        for (int i = 0; i < n; ++i) {
            for (int to : adj[i]) {
                if (match[to] == -1) {
                    match[to] = i;
                    used_left[i] = true;
                    matches++;
                    break;
                }
            }
        }

        for (int i = 0; i < n; ++i) {
            if (used_left[i]) continue;
            fill(used.begin(), used.end(), false);
            if (dfs(i)) {
                matches++;
            }
        }

        return matches;
    }
};
```

17.19 Sack (dc4882)

```cpp
/*
 DSU on Tree (Sack)
 Utilizado para responder queries offline em subarvores.

 Exemplo de uso classico:
 - "Quantas cores distintas existem na subarvore do vertice U?"
 - "Qual e a cor mais frequente na subarvore do vertice U?"

 Complexidade de Tempo: O(N log N)
 Complexidade de Espaco: O(N)
*/

ll n;
vector<ll> adj[MAXN];
ll heavy[MAXN], sz[MAXN], cnt[MAXN];

void dfs(ll u, ll p = -1) { // precomputa filhos pesados e tamanho das
     subarvores
    sz[u] = 1;
    heavy[u] = -1;
    ll mx = 0;
    for (ll v : adj[u]) {
        if (v == p) continue;
        dfs(v, u);
        sz[u] += sz[v];
        if (sz[v] > mx) heavy[u] = v, mx = sz[v];
    }
}

void add(ll u, ll p, ll val, ll skip) {
    // cnt[cor[u]] += val;
    for (ll v : adj[u]) {
        if (v == p or v == skip) continue;
        add(v, u, val, skip);
    }
}

void dsu(ll u, ll p=-1, bool keep=false) {
    // 1. computa a resposta para os filhos leves e dps descarta
    for (ll v : adj[u]) {
        if (v == p or v == heavy[u]) continue;
        dsu(v, u, false);
    }

    // 2. computa a resposta para o filho pesado e mantem
    if (heavy[u] != -1) dsu(heavy[u], u, true);

    // 3. desce a subarvore dos leves para guardar a resposta
    add(u, p, 1, heavy[u]);

    // nesse instante cnt[i] tem qts vezes a cor i aparece na subarvore de u

    if (!keep) add(u, p, -1, -1); // se for descendente de um leve descarta tudo
          q foi contado na subarvore de u
}
```

17.20 SizeOfAllSubtrees (a2922a)

```cpp
#include <bits/stdc++.h>
using namespace std;
#define pb push_back

const int MAXN = 1e5 + 10;

int n, m, subtree_size[MAXN];
vector<int> adj[MAXN];
bool vis[MAXN];

void dfs(int v) {
    vis[v] = true;
    subtree_size[v] = 1;
    for (auto u : adj[v]) if(!vis[u]) {
        dfs(u);
        subtree_size[v] += subtree_size[u];
    }
}

signed main() {
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int u, v; cin >> u >> v;
        adj[u].pb(v);
        adj[v].pb(u);
    }
    dfs(0);
}
```

17.21 TopoSortBFS (87f16a)

```cpp
#include <bits/stdc++.h>
```

```cpp
using namespace std;
// Kahn Algorithm
// BFS + indegree: grau de incidencia
const int MAXN = 1e5 + 10;
int n, m;
vector<int> adj[MAXN], indegree(MAXN, 0), order;
void topoSort() {
    queue<int> q;
    // priority_queue<int> pq; // Ha uma preferencia nos vertices
    for (int v = 0; v < n; v++) {
        if (indegree[v] == 0) q.push(v);
    }
    while (!q.empty()) {
        int v = q.front(); q.pop();
        // int v = pq.top(); pq.pop();
        order.push_back(v);
        for (auto u : adj[v]) {
            indegree[u]--;
            if (indegree[u] == 0) {
                q.push(u);
            }
        }
    }
}
signed main() {
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int u, v; cin >> u >> v; u--, v--;
        adj[u].push_back(v);
        indegree[v]++;
    }
}
```

17.22 TopoSortDFS (b1833e)

```cpp
#include <bits/stdc++.h>
using namespace std;
// Grafo Aciclico Dirigido (DAG)
// - Arestas = Dependencias -> Ex.: Disciplinas e Pre-requisitos
// - Definicao: E uma permutacao dos vertices tal que
// para toda aresta v -> u, v aparece antes de u e "u depende de v"
// - Ordem de execucao da DP
const int MAXN=1e5+10;
int n, color[MAXN];
vector<int> adj[MAXN]; deque<int> order;
void dfs(int v) {
    color[v] = 1;
    for (auto u : adj[v]) {
        if (color[u] == 0) dfs(u);
        if (color[u] == 1) { // Achou um ciclo
            cout << "IMPOSSIBLE!" << endl;
            exit(0); // Nao e um DAG
        }
    }
    order.push_front(v);
    color[v] = 2;
}
void topoSort() {
    for (int v = 0; v < n; v++) {
        if (color[v] == 0) dfs(v);
    }
}
```

17.23 dinic (ce17ed)

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Dinic {
    struct Edge {
        int to;
        long long cap, flow;
        int rev;
    };

    int n;
    vector<vector<Edge>> adj;
    vector<int> level;
    vector<int> ptr;

    Dinic(int n) : n(n), adj(n), level(n), ptr(n) {}

    void addEdge(int u, int v, long long cap, bool directed = true) {
        adj[u].push_back({v, cap, 0, (int)adj[v].size()});
        adj[v].push_back({u, directed ? 0 : cap, 0, (int)adj[u].size() - 1});
    }

    bool bfs(int s, int t) {
        fill(level.begin(), level.end(), -1);
        level[s] = 0;
        queue<int> q;
        q.push(s);

        while (!q.empty()) {
            int v = q.front();
            q.pop();
            for (auto& e : adj[v]) {
                if (e.cap - e.flow > 0 && level[e.to] == -1) {
                    level[e.to] = level[v] + 1;
                    q.push(e.to);
                }
            }
        }
        return level[t] != -1;
    }

    long long dfs(int v, int t, long long pushed) {
        if (pushed == 0) return 0;
        if (v == t) return pushed;

        for (int& cid = ptr[v]; cid < adj[v].size(); ++cid) {
            auto& e = adj[v][cid];
            int tr = e.to;
            if (level[v] + 1 != level[tr] || e.cap - e.flow == 0) continue;

            long long push = dfs(tr, t, min(pushed, e.cap - e.flow));
            if (push == 0) continue;
            e.flow += push;
            adj[tr][e.rev].flow -= push;
            return push;
        }
        return 0;
    }

    long long maxFlow(int s, int t) {
        long long flow = 0;
        while (bfs(s, t)) {
            fill(ptr.begin(), ptr.end(), 0);
            while (long long pushed = dfs(s, t, 1e18)) {
                flow += pushed;
            }
        }
        return flow;
    }
};
```

17.24 edmondsKarp (9aefbc)

```cpp
#include <bits/stdc++.h>
using namespace std;

struct EdmondsKarp {
    struct Edge {
        int to;
        long long cap, flow;
        int rev;
    };
```

```
    int n;
    vector<vector<Edge>> adj;
    vector<int> parentNode, parentEdgeIndex;

    EdmondsKarp(int n) : n(n), adj(n), parentNode(n), parentEdgeIndex(n) {}

    // Adiciona uma aresta do no 'u' para o no 'v' com capacidade 'cap'
    // Se directed = false, cria uma aresta bidirecional (ida e volta com mesma
        capacidade)
    void addEdge(int u, int v, long long cap, bool directed = true) {
        adj[u].push_back({v, cap, 0, (int)adj[v].size()});
        adj[v].push_back({u, directed ? 0 : cap, 0, (int)adj[u].size() - 1});
    }

    long long bfs(int s, int t) {
        fill(parentNode.begin(), parentNode.end(), -1);
        parentNode[s] = -2;
        queue<pair<int, long long>> q;
        q.push({s, 1e18});
        while (!q.empty()) {
            int cur = q.front().first;
            long long flow = q.front().second;
            q.pop();

            for (int i = 0; i < adj[cur].size(); i++) {
                Edge &e = adj[cur][i];
                if (parentNode[e.to] == -1 && e.cap - e.flow > 0) {
                    parentNode[e.to] = cur;
                    parentEdgeIndex[e.to] = i;
                    long long new_flow = min(flow, e.cap - e.flow);

                    if (e.to == t) return new_flow;
                    q.push({e.to, new_flow});
                }
            }
        }
        return 0;
    }

    long long maxFlow(int s, int t) {
        long long flow = 0, new_flow;
        while ((new_flow = bfs(s, t))) {
            flow += new_flow;
            int cur = t;
            while (cur != s) {
                int prev = parentNode[cur];
                int edgeIdx = parentEdgeIndex[cur];

                adj[prev][edgeIdx].flow += new_flow;
                int revIdx = adj[prev][edgeIdx].rev;
                adj[cur][revIdx].flow -= new_flow;

                cur = prev;
            }
        }
        return flow;
    }
};
```

17.25 tarjan (573bfa)

```
vector<int> g[MAX];
stack<int> s;
int vis[MAX], comp[MAX];
int id[MAX];

// se quiser comprimir ciclo ou achar ponte em grafo nao direcionado,
// colocar um if na dfs para nao voltar pro pai da DFS tree
int dfs(int i, int& t) {
        int lo = id[i] = t++;
        s.push(i);
        vis[i] = 2;

        for (int j : g[i]) {
                if (!vis[j]) lo = min(lo, dfs(j, t));
                else if (vis[j] == 2) lo = min(lo, id[j]);
        }

        // aresta de i pro pai eh uma ponte (no caso nao direcionado)
        if (lo == id[i]) while (1) {
                int u = s.top(); s.pop();
                vis[u] = 1, comp[u] = i;
                if (u == i) break;
        }

        return lo;
}

void tarjan(int n) {
        int t = 0;
        for (int i = 0; i < n; i++) vis[i] = 0;

        for (int i = 0; i < n; i++) if (!vis[i]) dfs(i, t);
}
```

18 Math

18.1 Combinacao (029489)

```
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 100;
int comb[MAXN+1][MAXN+1];
// pre calculando as combinacoes -> O(n)
void pre() {
    comb[0][0] = 1;
    for (int n = 1; n <= MAXN; n++) {
        comb[n][0] = comb[n][n] = 1;
        for (int k = 1; k < n; k++) {
            comb[n][k] = comb[n-1][k-1] + comb[n-1][k];
        }
    }
}
signed main() {
    pre();
}
```

18.2 Crivo (a6487b)

```
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 2e5+10;
bool primo[MAXN];
// Crivo ate raiz de N --> Para descobrir todos os primos ate N, basta achar
// todos os primos que nao ultrapassam N
void crivo() { // O(nlog(log(n)))
    for (int i = 0; i < MAXN; i++) primo[i] = true;
    primo[0] = primo[1] = false;
    for (int i = 2; i * i < MAXN; i++) {
        if (primo[i] and (long long) i * i < MAXN) {
            for (int j = i * i; j < MAXN; j += i) {
                primo[j] = false;
            }
        }
    }
}
// Crivo ate N
// Util quando queremos fatorar numeros ate MAXN^2
void crivo() {
    for (int i = 0; i < MAXN; i++) primo[i] = true;
    primo[0] = primo[1] = false;
    for (int i = 2; i < MAXN; i++) {
        if (primo[i]) {
            // i e primo
```

```
            if ((long long) i * i < MAXN) {
                for (int j = i*i; j < MAXN; j+=i) {
                    primo[j] = false;
                }
            }
        }
    }
}
// https://cp-algorithms.com/algebra/sieve-of-eratosthenes.html#implementation
```

18.3 Divisores (7c1b13)

```
#include <bits/stdc++.h>
using namespace std;
vector<int> divisores[100005];
signed main() {
    // Enumerar todos os divisores dos numeros de i ate N
    int n; cin >> n;
    for (int i = 1; i <= n; i++) { // O(nlog(n))
        for (int j = i; j <= n; j += i) { // j -> multiplo de i
            divisores[j].push_back(i);
        }
    }
}

/*
# Teorema Fundamental da Aritmetica

Todo inteiro N > 1 pode ser representado de forma unica (exceto pela ordem) como
      produto de numeros
primos.

N = (p1 ^ e1) * (p2 ^ e2) * ... * (pk ^ ek)

# Quantidade de Divisores

A quantidade de divisores de um numero N, denotada por d(N), e calculada a
     partir de seus expoentes primos

d(N) = (e1 + 1)(e2 + 1)...(ek + 1)

Obs.: a qtd de divisores de N so e impar, se N for um QUADRADO PERFEITO!
*/
```

18.4 Divisores2 (de6b7b)

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
vector<int> divisores(int n) { // O(sqrt(N))
    vector<int> ans;
    for (int a = 1; a*a <= n; a++) {
        if (n % a == 0) {
            int b = n / a;
            ans.push_back(a);
            if (a != b) ans.push_back(b);
        }
    }
    return ans;
}
```

18.5 Fatoracao (c23ee5)

```
#include <bits/stdc++.h>
using namespace std;
map<int, int> fatorar(int n) { // O(sqrt(n))
    map<int, int> exp;
    for (int i = 2; i * i <= n; i++) {
        while (n % i == 0) {
            exp[i]++;
            n /= i;
        }
    }
    if (n > 1) exp[n]++;
    return exp;
}
```

18.6 LargestPrimeFactor (a499a6)

```
#include <bits/stdc++.h>
using namespace std;
// E possivel encontrar o maior fator primo de um numero
// a partir da conjectura de 'Twin Primes'
int main() {
    int n; cin >> n;
    int ans = -1;
    while (n % 2 == 0) {
        n /= 2;
        ans = 2;
    }
    while (n % 3 == 0) {
        n /= 3;
        ans = 3;
    }
    for (int i = 5; i*i <= n; i++) {
        while (n % i == 0) {
            n /= i;
            ans = i;
        }
        while (n % (i + 2) == 0) {
            n /= (i + 2);
            ans = (i + 2);
        }
    }
    if (n > 4) ans = n;
    cout << ans << endl;
}
```

18.7 Modulo (8c04ba)

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
// Assuma que os inputs estao no range [0, MOD)
const int MOD = 1e9 + 7;
inline int add(int a, int b) {return ((a % MOD) + (b % MOD)) % MOD;}
inline int sub(int a, int b) {return ((a % MOD) - (b % MOD) + MOD) % MOD;}
inline int mult(int a, int b) {return ((a % MOD) * (b % MOD)) % MOD;}
inline int pow(int x, int y, int p) { // (x ^ y) % p
    int res = 1;
    x %= p;
    if (x == 0) return 0;
    while (y > 0) {
        if (y & 1)
            res = (res * x) % p;
        y = y >> 1;
        x = (x * x) % p;
    }
    return res;
}
// Calcular inverso modular: a^-1
inline int inv(int a) {return pow(a, MOD - 2, MOD);}
inline int divi(int a, int b) {return mult(a, inv(b));}
signed main() {
    int a = 10, b = 20; // Substitua o MOD para 7
    cout << add(a, b) << endl; // 2
    cout << sub(a, b) << endl; // 4
    cout << mult(a, b) << endl; // 4
    cout << divi(a, b) << endl; // 4
}
```

18.8 comb (516d9d)

```
struct Comb {
    int n;
    vector<mint> _fac;
    vector<mint> _invfac;
    vector<mint> _inv;

    Comb() : n{0}, _fac{1}, _invfac{1}, _inv{0} {}
    Comb(int n) : Comb() {
        init(n);
    }

    void init(int m) {
        if (m <= n) return;
        _fac.resize(m + 1);
        _invfac.resize(m + 1);
        _inv.resize(m + 1);

        for (int i = n + 1; i <= m; i++) {
            _fac[i] = _fac[i - 1] * i;
        }
        _invfac[m] = _fac[m].inv();
        for (int i = m; i > n; i--) {
            _invfac[i - 1] = _invfac[i] * i;
            _inv[i] = _invfac[i] * _fac[i - 1];
        }
        n = m;
    }

    mint fac(int m) {
        if (m > n) init(2 * m);
        return _fac[m];
    }
    mint invfac(int m) {
        if (m > n) init(2 * m);
        return _invfac[m];
    }
    mint inv(int m) {
        if (m > n) init(2 * m);
        return _inv[m];
    }
    mint binom(int n, int m) {
        if (n < m || m < 0) return mint(0);
        return fac(n) * invfac(m) * invfac(n - m);
    }
} comb;
```

18.9 matrixExp (67c023)

```
// Problema: quantas maneiras conseguimos formar uma linha usando buses(10
     metros) e minibuses(5 metros)
// N = comprimento da linha (sempre multiplo de 5)
// K = numero de diferentes cores para minibuses
// L = numero de diferentes cores para buses
// Recorrencia abaixo
// Cel(1,1) = 0, caso base: 0
// Cel(0,1) = L, minibus
// Cel(1,0) = 1, pois com 5 conseguimos escolher um minibus
// Cel(0,0) = K, bus (precisamos de 2 L)
// A potencia e n/5
// (k, 1)
// (1, 0)
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
const ll MOD = 1000000;

struct M6 {
    ll v;
    M6(ll _v = 0) {
        _v %= MOD;
        if (_v < 0) _v += MOD;
        v = _v;
    }
    M6 operator+(const M6& o) const { return M6(v + o.v); }
    M6 operator*(const M6& o) const { return M6(v * o.v); }
    void operator+=(const M6& o) { v = (v + o.v) % MOD; }
};

struct Matrix {
    M6 mat[2][2];
    Matrix() {
        mat[0][0] = mat[0][1] = mat[1][0] = mat[1][1] = M6(0);
    }
    static Matrix identity() {
        Matrix res;
        res.mat[0][0] = M6(1);
        res.mat[1][1] = M6(1);
        return res;
    }
    Matrix operator*(const Matrix& o) const {
        Matrix res;
        for (int i = 0; i < 2; i++)
            for (int k = 0; k < 2; k++)
                for (int j = 0; j < 2; j++)
                    res.mat[i][j] += mat[i][k] * o.mat[k][j];
        return res;
    }
};

Matrix power(Matrix a, ll b) {
    Matrix res = Matrix::identity();
    while (b > 0) {
        if (b & 1) res = res * a;
        a = a * a;
        b >>= 1;
    }
    return res;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    ll n, k, l;
    cin >> n >> k >> l;

    ll n_norm = n / 5;

    Matrix T;
    T.mat[0][0] = M6(k);  T.mat[0][1] = M6(l);
    T.mat[1][0] = M6(1);  T.mat[1][1] = M6(0);

    T = power(T, n_norm);

    ll result = T.mat[0][0].v;

    printf("%06lld\n", result);

    return 0;
}


// outro template
template<typename T,unsigned int N>
struct Matrix{
        array<array<T,N>,N>dat;
        array<T,N>&operator[](int i){return dat[i];}
        const array<T,N>&operator[](int i)const{return dat[i];}
        Matrix(){for(int i=0;i<N;i++)dat[i].fill(T(0));}
        static Matrix eye(){
                Matrix res;
                for(int i=0;i<N;i++)res[i][i]=1;
                return res;
        }
        Matrix operator+(const Matrix&A)const{
                Matrix res;
                for(int i=0;i<N;i++)for(int j=0;j<N;j++)
                        res[i][j]=dat[i][j]+A[i][j];
```

```
            return res;
        }
        Matrix operator*(const Matrix&A)const{
            Matrix res;
            for(int i=0;i<N;i++)for(int k=0;k<N;k++)for(int j=0;j<N;j++)
                res[i][j]+=dat[i][k]*A[k][j];
            return res;
        }
        Matrix pow(long long n)const{
            Matrix a=*this,res=eye();
            for(;n;a=a*a,n>>=1)if(n&1)res=res*a;
            return res;
        }
};
```

19 Strings

19.1 KMP (c41d90)

```
#include <bits/stdc++.h>
using namespace std;

// n = |padrao| e m = |texto|
// pi: O(n)   match: O(n + m)   automato: O(|sigma| * n)

// Funcao de prefixo pi(i):
// Para todo prefixo s' de s, a funcao calcula o tamanho do maior
// do prefixo proprio de s' que tambem e sufixo.
vector<int> pi(string s) {
    vector<int> p(s.size());
    for (int i = 1, j = 0; i < s.size(); i++) {
        while (j > 0 and s[i] != s[j]) j = p[j-1];
        if (s[i] == s[j]) j++;
        p[i] = j;
    }
    return p;
}

// t -> texto, s -> padrao
// A funcao retorna a quantidade de matchings
vector<int> matching(string &t, string& s) {
    // '$' e um caracter impossivel no padrao,
    // serve para nao tratar o caso de quando acaba o matching
    vector<int> p = pi(s+'$'), match;
    for (int i = 0, j = 0; i < t.size(); i++) {
        while (j > 0 and t[i] != s[j]) j = p[j-1];
        if (t[i] == s[j]) j++;
        if (j == s.size()) match.push_back(i-j+1);
    }
    return match;
}
```

20 Data Structures

20.1 DSU (ed38b0)

```
#include <bits/stdc++.h>
using namespace std;
struct dsu {
    // pai = representante do conjunto
    vector<int> parent, size;
    dsu(int n) {
        parent.resize(n);
        size.resize(n);
        for (int i = 0; i < n; i++) {
            parent[i] = i;
            size[i] = 1;
        }
    }
    // find e uni: O(a(n)) ~= O(1) armotizado
    int find(int i) {
        // Path Compression
        return parent[i] = (parent[i] == i) ? i : find(parent[i]);
    }
    void uni(int a, int b) { // o pai de a se torna pai de b
        a = find(a), b = find(b);
        if (a != b) {
            // a -> maior arvore
            if (size[a] < size[b]) { // Small to Large Otimization
                swap(a, b);
            }
            parent[b] = a;
            size[a] += size[b];
        }
    }
};
// https://cp-algorithms.com/data_structures/disjoint_set_union.html
```

20.2 DynnamicConnectivityOffline (f3c8cf)

```
#include <bits/stdc++.h>
using namespace std;

// Problema dynamic connectivity

#define endl '\n'
#define f first
#define sec second
#define pb push_back

typedef pair<int, int> pii;
typedef vector<int> vi;

const int N = 3e5 + 10;

vector<pii> seg[4 * N];
map<pii, int> entradas; // entradas das arestas na query
int ans[N];

// DSU com rollback
struct persistent_dsu {
  struct state {
    int u, v, rnku, rnkv;
    state() {
      u = -1;
      v = -1;
      rnkv = -1;
      rnku = -1;
    }
    state(int _u, int _rnku, int _v, int _rnkv) {
      u = _u;
      rnku = _rnku;
      v = _v;
      rnkv = _rnkv;
    }
  };

  stack<state> st;
  int par[N], depth[N];
  int comp;

  persistent_dsu() {
    comp = 0;
    memset(par, -1, sizeof(par));
    memset(depth, 0, sizeof(depth));
  }

  int root(int x) {
    if (x == par[x])
      return x;
    return root(par[x]);
  }
```

```
  void init(int n) {
    comp = n;
    for (int i = 0; i <= n; i++) {
      par[i] = i;
      depth[i] = 1;
    }
  }

  bool connected(int x, int y) { return root(x) == root(y); }

  void unite(int x, int y) {
    int rx = root(x), ry = root(y);
    if (rx == ry) {
      st.push(state());
      return;
    }

    st.push(state(rx, depth[rx], ry, depth[ry]));

    if (depth[rx] < depth[ry]) {
      par[rx] = ry;
    } else if (depth[ry] < depth[rx]) {
      par[ry] = rx;
    } else {
      par[rx] = ry;
      depth[ry]++;
    }
    comp--;
  }

  void backtrack(int c) {
    while (!st.empty() && c) {
      if (st.top().u == -1) {
        st.pop();
        c--;
        continue;
      }
      par[st.top().u] = st.top().u;
      par[st.top().v] = st.top().v;
      depth[st.top().u] = st.top().rnku;
      depth[st.top().v] = st.top().rnkv;
      st.pop();
      c--;
      comp++;
    }
  }
};

persistent_dsu d;

void upd(int p, int l, int r, int ql, int qr, pii val) {
  if (l > qr or r < ql)
    return;
  // Seg[p] guarda as arestas contidas em [L,R]
  if (ql <= l and r <= qr) {
    seg[p].pb(val);
    return;
  }
  int m = (l + r) / 2;
  upd(2 * p, l, m, ql, qr, val);
  upd(2 * p + 1, m + 1, r, ql, qr, val);
}

void query(int p, int l, int r) {
  if (l > r)
    return;
  int prev = d.st.size();

  // Uniao de todas arestas pertencentes a seg[p]
  for (auto &edge : seg[p])
    d.unite(edge.f, edge.sec);

  if (l == r) {
    ans[l] = d.comp;
    // Damos rollback no DSU para o DSU sem as arestas de sep[p]
    d.backtrack(d.st.size() - prev);
    return;
  }
  int m = (l + r) / 2;
  query(2 * p, l, m);
  query(2 * p + 1, m + 1, r);

  // Rollback novamente, ja lemos esse node e estamos subindo na seg tree
  d.backtrack(d.st.size() - prev);
}

int main() {
  ios::sync_with_stdio(false);
  cin.tie(NULL);

  freopen("connect.in", "r", stdin);
  freopen("connect.out", "w", stdout);

  int n, k;
  cin >> n >> k;

  vi queries;
  d.init(n);
  for (int i = 1; i <= k; i++) {
    char t;
    cin >> t;
    if (t == '?') {
      queries.pb(i);
    } else if (t == '+') {
      int u, v;
      cin >> u >> v;
      if (u > v)
        swap(u, v);
      entradas[{u, v}] = i;
    } else if (t == '-') {
      int u, v;
      cin >> u >> v;
      if (u > v)
        swap(u, v);
      pii p = {u, v};
      // aresta (u,v) entra em entradas[(u,v)] e sai em i-1
      upd(1, 1, k, entradas[p], i - 1, p);
      // apagamos a aresta
      entradas.erase(p);
    }
  }

  // Adicionamos as arestas que nao sao removidas
  for (auto& p : entradas) {
    // p.second = tempo de entrada, p.first = aresta
    upd(1, 1, k, p.second, k, p.first);
  }
  query(1, 1, k);
  for (auto i : queries) {
    cout << ans[i] << endl;
  }

  return 0;
}
```

20.3 FenwickTree (27422d)

```
// Binary Indexed Tree ou Fenwick Tree
#include <bits/stdc++.h>
using namespace std;

// 0-INDEXED
struct fenw {
    int n;
    vector<int> bit;
    fenw() {}
    fenw(int size) {
        n = size;
        bit.assign(size + 1, 0);
    }
    // query do prefixo a[0] + a[1] + ... + a[r]
    int qry(int r) {
```

```
        int ans = 0;
        for (int i = r + 1; i > 0; i -= i & -i) // i & -i retorna os bits menos
             signativos de i
            ans += bit[i];
        return ans;
    }
    // atualiza o valor a[r] = x
    void upd(int r, int x) {
        for (int i = r + 1; i <= n; i += i & -i) bit[i] += x;
    }
    // busca binaria para o maior indice i (i < n) tal que qry(i) < x
    int bs(int x) {
        int i = 0, k = 0;
        while (1 << (k + 1) <= n) k++;
        while (k >= 0) {
            int nxt_i = i + (1 << k);
            if (nxt_i <= n && bit[nxt_i] < x) {
                i = nxt_i;
                x -= bit[i];
            }
            k--;
        }
        return i - 1;
    }
};
```

20.4 LCAsemStruct (923986)

```
#include<bits/stdc++.h>
using namespace std;

const int N = 3e5 + 9, LG = 18;

vector<int> g[N];
int par[N][LG + 1], dep[N], sz[N];
void dfs(int u, int p = 0) {
  par[u][0] = p;
  dep[u] = dep[p] + 1;
  sz[u] = 1;
  for (int i = 1; i <= LG; i++) par[u][i] = par[par[u][i - 1]][i - 1];
  for (auto v: g[u]) if (v != p) {
    dfs(v, u);
    sz[u] += sz[v];
  }
}
int lca(int u, int v) {
  if (dep[u] < dep[v]) swap(u, v);
  for (int k = LG; k >= 0; k--) if (dep[par[u][k]] >= dep[v]) u = par[u][k];
  if (u == v) return u;
  for (int k = LG; k >= 0; k--) if (par[u][k] != par[v][k]) u = par[u][k], v =
       par[v][k];
  return par[u][0];
}
int kth(int u, int k) {
  assert(k >= 0);
  for (int i = 0; i <= LG; i++) if (k & (1 << i)) u = par[u][i];
  return u;
}
int dist(int u, int v) {
  int l = lca(u, v);
  return dep[u] + dep[v] - (dep[l] << 1);
}
//kth node from u to v, 0th node is u
int go(int u, int v, int k) {
  int l = lca(u, v);
  int d = dep[u] + dep[v] - (dep[l] << 1);
  assert(k <= d);
  if (dep[l] + k <= dep[u]) return kth(u, k);
  k -= dep[u] - dep[l];
  return kth(v, dep[v] - dep[l] - k);
}
int32_t main() {
  int n; cin >> n;
  for (int i = 1; i < n; i++) {
    int u, v; cin >> u >> v;
    g[u].push_back(v);
    g[v].push_back(u);
  }
  dfs(1);
  int q; cin >> q;
  while (q--) {
    int u, v; cin >> u >> v;
    cout << dist(u, v) << '\n';
  }
  return 0;
}
```

20.5 MinQueue (ed1384)

```
#include <bits/stdc++.h>
using namespace std;
typedef pair<int, int> pi;
#define ff first
#define ss second
// push, pop: O(1) armotizado e get_min: O(1)
// Util em problemas de sliding window ou semelhantes
// A ideia e guardar apenas os elementos necessarios para encontrar o minimo
struct MinQueue {
    deque<pi> dq;
    int added = 0, removed = 0;
    // adiciona x no final da fila
    void push(int x) {
        // x > dq.back().ff para MaxQueue
        while (!dq.empty() and x < dq.back().ff) {
            dq.pop_back();
        }
        dq.push_back({x, added});
        added++;
    }
    // remove o elemento do inicio da final
    void pop() { // pop(int x) -> remove x
        // if (!dq.empty() and dq.front().ss == x) {
        if (!dq.empty() and dq.front().ss == removed) {
            dq.pop_front();
        }
        removed++;
    }
    int get_min() {
        return dq.front().ff;
    }
};
// https://noic.com.br/materiais-informatica/curso/data-structures-01/
// https://cp-algorithms.com/data_structures/stack_queue_modification.html
```

20.6 MinQueueAggStack (3748a1)

```
#include <bits/stdc++.h>
using namespace std;
typedef pair<int, int> pi;
#define ff first
#define ss second
// Aggreged Queue e Stack
// - Esse codigo pode ser alterado para min, max, gcd, or, ...
// - E mais flexivel que a MinQueue com deque do arquivo MinQueue.cpp
// - Toda fila pode ser implementada como duas pilhas
// modifique o op para a operacao que vc quer (op deve ser agregavel)
int op(int a, int b) { return a | b; }
// int op(int a, int b) { return min(a, b); }
struct AggStack {
    // no second do pair tem o valor agregado da operacao
    stack<pi> st;
    // adiciona um novo elemento (O(1))
    void push(int x) {
        int cur = st.empty() ? x : op(st.top().ss, x);
        st.push({x, cur});
    }
    // remove o elemento do topo (O(1))
```

```
    void pop() { st.pop(); }
    // retorna o valor agregado do op (minimo, or, ...)
    int agg() { return st.top().ss; };
    // checa se a pilha esta vazia
    bool empty() { return st.empty(); }
    // retorna o topo da pilha
    pi top() { return st.top(); }
};
struct AggQueue {
    AggStack in, out;
    // adiciona um elemento na fila (O(1) amortizado)
    void push(int x) { in.push(x); }
    // remove o primeiro elemento (O(1) amortizado)
    void pop() {
        if (out.empty()) {
            while (!in.empty()) {
                auto [x, cur] = in.top();
                in.pop();
                out.push(x);
            }
        }
        out.pop();
    }
    // retorna o valor agregado da fila (O(1))
    int query() {
        if (in.empty()) return out.agg();
        if (out.empty()) return in.agg();
        return op(in.agg(), out.agg());
    }
};
// https://codeforces.com/blog/entry/143351
```

20.7 Mo (77aa1b)

```
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 2e5 + 10;
int v[MAXN];
// O((N + Q)*SQRT(N)) -> Offline Range Queries
int block_size; // block_size = teto(sqrt(n))
// TODO: Criar as EDs do problema
int freq[MAXN], cnt_freq[MAXN], freq_mode;
struct Query {
    int l, r, idx;
    bool operator<(Query o) const {
        return make_pair(l / block_size, r) <
               make_pair(o.l / block_size, o.r);
    }
};
void add(int idx) {
    int x = v[idx];
    cnt_freq[freq[x]]--;
    freq[x]++;
    cnt_freq[freq[x]]++;
    if (freq[x] > freq_mode) {
        freq_mode = freq[x];
    }
}
void remove(int idx) {
    int x = v[idx];
    if (freq[x] == freq_mode and cnt_freq[freq_mode] == 1) {
        freq_mode--;
    }
    cnt_freq[freq[x]]--;
    freq[x]--;
    cnt_freq[freq[x]]++;

}
int get_answer() {
    return freq_mode;
}
vector<int> mo(vector<Query> qrys) {
    vector<int> ans(qrys.size());
    sort(qrys.begin(), qrys.end());
    // TODO: Inicializar as EDs do problema
    int cur_l = 0, cur_r = -1;
    for (auto [l, r, idx] : qrys) {
        while (cur_l > l) {
            cur_l--;
            add(cur_l);
        }
        while (cur_r < r) {
            cur_r++;
            add(cur_r);
        }
        while (cur_l < l) {
            remove(cur_l);
            cur_l++;
        }
        while (cur_r > r) {
            remove(cur_r);
            cur_r--;
        }
        ans[idx] = get_answer();
    }
    return ans;
}
signed main() {
    int n, q; cin >> n, q;
    block_size = sqrt(n);
    vector<Query> qrys(q);
    for (int i = 0; i < q; i++) {
        int l, r; cin >> l >> r; l--, r--;
        qrys[i] = {l, r, i};
    }
    vector<int> ans = mo(qrys);
    for (auto x : ans) cout << x << endl;
}
// https://cp-algorithms.com/data_structures/sqrt_decomposition.html
// Mo para encontrar a maior frequencia em [l, r]
// Podemos usar esse codigo para problema de soma, minimo, maximo, frequencia,
// valores que obedecem uma condicao, etc...
```

20.8 ModInt (7a26a6)

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
const int MOD = 1e9 + 7;
const int MAXN = 5e5+5;
struct modint {
    using m = modint;
    int val;
    modint(int v = 0) { val = v % MOD; }
    int pow(int y) {
        m x = val, res = 1;
        while (y) {
            if (y & 1)
                res *= x;
            x *= x;
            y >>= 1;
        }
        return res.val;
    }
    int inv() { return pow(MOD - 2); }
    void operator=(int o) { val = o % MOD; }
    void operator=(m o) { val = o.val % MOD; }
    void operator+=(m o) { *this = *this + o; }
    void operator-=(m o) { *this = *this - o; }
    void operator*=(m o) { *this = *this * o; }
    void operator/=(m o) { *this = *this / o; }
    bool operator==(m o) { return val == o.val; }
    bool operator!=(m o) { return val != o.val; }
    int operator*(m o) { return ((val * o.val) % MOD); }
    int operator/(m o) { return (val * o.inv()) % MOD; }
    int operator+(m o) { return (val + o.val) % MOD; }
    int operator-(m o) { return (val - o.val + MOD) % MOD; }
    friend istream& operator >>(istream& in, m& a) {
                int val; in >> val; a = m(val); return in;
        }
```

```
    friend ostream& operator <<(ostream& out, m a) {
                return out << a.val;
        }
};
modint fat[MAXN], inv[MAXN], invfat[MAXN];
void pre() {
    // Calculando Fatorial
    fat[0] = 1;
    for (int i = 1; i < MAXN; i++)
        fat[i] = fat[i-1] * i;
    // Calculando Inverso Fatorial
    inv[1] = 1;
    for (int i = 2; i < MAXN; i++) {
        int val = MOD / i;
        val = (inv[MOD % i] * val) % MOD;
        val = MOD - val;
        inv[i] = val;
    }
    invfat[0] = 1;
    invfat[MAXN - 1] = modint(fat[MAXN - 1]).inv();
    for (int i = MAXN - 2; i >= 1; i--)
        invfat[i] = invfat[i + 1] * (i + 1);
}
// C(n, k) = n! / (k! * (n - k)!)
modint comb(int n, int k) {
    modint ans = fat[n] * invfat[k];
    ans *= invfat[n - k];
    return ans;
}
// A(n, k) = n! / (n - k)!
modint arr(int n, int k) {
    modint ans = fat[n] * invfat[n - k];
    return ans;
}
signed main() {
    pre();
    modint a = 10, b = 3; // substituia o MOD para 7
    cout << a + b << endl; // 6
    cout << a - b << endl; // 0
    cout << a * b << endl; // 2
    cout << a / b << endl; // 1
    cout << a.pow(5) << endl; // 5
    cout << comb(5, 2) << endl; // 10
    cout << arr(5, 2) << endl; // 20
}
```

20.9 OrderStatisticSet (1234d2)

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace std;
using namespace __gnu_pbds;
// Consiste em uma Arvore Binaria Balanceada (std::set + superpoderes)
// Armazena elementos unicos e de forma ordenada
// Operacoes:
// 1 - Todas de um std::set
// 2 - find_by_order(k): retorna um iterator para o k-esimo valor (0-INDEXED) (O
    (log(n)))
// 3 - order_of_key(k): retorna o numero de elementos estritamente menores
// que k. (ou seja, o NUMERO DE INVERSOES)
template <class T>
using ordered_set = tree<
    T, null_type, less<T>,
    rb_tree_tag, tree_order_statistics_node_update>;
// Voce pode fazer um multiset com ordered_set<pair<T, int>>, o segundo elemento
// e um indice por exemplo.
signed main() {
    ordered_set<int> s;
    // Insercao
    for (int i = 0; i < 10; i++) s.insert(i);
    // Leitura
    for (auto i : s) cout << i << endl;
    int k = 0;
    // Posicao do valor k
    cout << *s.find_by_order(k) << endl;
    // Quantos sao menores do que k O(log|s|):
    cout << s.order_of_key(k) << endl;
    // Remocao - Set
    s.erase(5); // apaga o valor 5 (se existir)
    // apaga o elemento pelo iterator
    auto it = s.find(7);
    if (it != s.end()) s.erase(it);
    // remove todos os menores < 5
    auto it_end = s.lower_bound(5);
    while (s.begin() != it_end) {
        s.erase(s.begin());
    }
}
// https://codeforces.com/blog/entry/123624
```

20.10 SegmentTree (b19a9b)

```
#include <bits/stdc++.h>
using namespace std;
#define endl '\n'
const int INF = 0x3f3f3f3f;
#define MAXN 1000000
int n, v[MAXN], seg[4*MAXN]; // a SegTree esta 0-INDEXED
// Funcoes de Apoio
int single(int x) {return x;}
int neutral() {return -INF;}
int merge(int a, int b) {return max(a, b);}
// p -> indice na segtree, [l, r] -> intervalo da subarray
int build(int p=1, int l=0, int r=n-1) { // O(n)
    if (l == r) return seg[p] = single(v[l]);
    int m = (l+r)/2;
    return seg[p] = merge(build(2*p, l, m), build(2*p+1, m+1, r));
}
// query no intervalo [a, b]
int qry(int a, int b, int p=1, int l=0, int r=n-1) { // O(log(n))
    if (b < l or a > r) return neutral();
    if (a <= l and r <= b) return seg[p];
    int m = (l+r)/2;
    return merge(qry(a, b, 2*p, l, m), qry(a, b, 2*p+1, m+1, r));
}
// update -> v[i] = x
int upd(int i, int x, int p=1, int l=0, int r=n-1) { // O(log(n))
    if (i < l or r < i) return seg[p];
    if (l == r) return seg[p] = single(x);
    int m = (l+r)/2;
    return seg[p] = merge(upd(i, x, 2*p, l, m), upd(i, x, 2*p+1, m+1, r));
}
// primeira posicao >= x em [a, b] (ou -1, caso nao exista)
// encontrar o menor indice em [a, b]
// posso passar (x+1) para buscar 'primeira posicao > x'
// Obs.: So funciona na SegTree de Maximos
int first_above(int x, int a, int b, int p=1, int l=0, int r=n-1) { // O(log(n))
    if (b < l or r < a or seg[p] < x) return -1;
        if (l == r) return l;
        int m = (l+r)/2;
        int left = first_above(x, a, b, 2*p, l, m);
        if (left != -1) return left;
        return first_above(x, a, b, 2*p+1, m+1, r);
}
// ultima posicao >= x em [a, b] (ou -1, caso nao exista)
// encontrar o maior indice em [a, b]
// posso passar (x + 1) para buscar 'ultima posicao > x'
// Obs.: So funciona na SegTree de Maximos
int last_above(int x, int a, int b, int p=1, int l=0, int r=n-1) { // O(log(n))
    if (b < l or r < a or seg[p] < x) return -1;
        if (l == r) return l;
        int m = (l+r)/2;
        int right = last_above(x, a, b, 2*p+1, m+1, r);
        if (right != -1) return right;
        return last_above(x, a, b, 2*p, l, m);
}
// retorna a posicao i do vetor do Kth 1
// Obs.: So funciona na SegTree de Soma
// k e 1-INDEXED e o RETORNO e 0-INDEXED
```

```
int find_kth(int k, int p=1, int l=0, int r=n-1) {
    if (k > seg[p]) return -1;
    if (l == r) return l;
    int m = (l + r) / 2;
    if (seg[p*2] >= k)
        return find_kth(k, p*2, l, m);
    else
        return find_kth(k - seg[p*2], p*2+1, m+1, r);
}
signed main() {
    ios_base::sync_with_stdio(0);cin.tie(0);
    cin >> n;
    // Voce pode inicializar v[] com um valor neutro
    // fill(v, v+MAXN, netrual());
    // Leitura de v[]
    for(int i = 0; i < n; i++) cin >> v[i];
    // Construir a SegTree
    build(1, 0, n - 1);
    // Query do intervalo a, b
    int a, b; cin >> a >> b;
    cout << qry(a, b, 1, 0, n - 1) << endl;
    // Update de v[i] = x
    int i, x; cin >> i >> x;
    cout << upd(i, x, 1, 0, n - 1) << endl;
}
/* Segment Tree ou SegTree ou Arvore de Segmentos
# Altura da arvore -> O(log(n))
# Qtd de nos -> 2n - 1 -> O(n)
-> Problemas de RMQ: Range Minimun Query
    -> Descobrir o minimo de um intervalo [l, r]

-> Soma, Minimo, Maximo, Produto, ...
-> SegTree para qualquer operacao ASSOCIATIVA: (A ? B) ? C = A ? (B ? C)
*/
```

20.11 SegmentTreeLazy (e2aa98)

```
#include <bits/stdc++.h>
using namespace std;
#define endl '\n'
#define MAXN 100005
int n, q, v[MAXN], seg[MAXN*4], lazy[MAXN*4], leafs[MAXN];
// Funcoes de Apoio
int single(int x) { return x; }
int neutral() { return 0; }
int merge(int a, int b) { return a + b;}
// Funcao de Propagacao
void prop(int p, int l, int r) { // O(1)
    seg[p] += lazy[p]*(r-l+1);
        if (l != r) {
        lazy[2*p] += lazy[p];
        lazy[2*p+1] += lazy[p];
    }
    lazy[p] = 0;
}
// p -> indice na segtree, [l, r] -> intervalo da subarray
int build(int p=1, int l=0, int r=n-1) { // O(n)
    lazy[p] = 0;
    if (l == r) return seg[p] = single(v[l]);
    int m = (l+r)/2;
    return seg[p] = merge(build(2*p, l, m), build(2*p+1, m+1, r));
}
// query no intervalo [a, b]
int qry(int a, int b, int p=1, int l=0, int r=n-1) {
    prop(p, l, r);
    if (b < l or a > r) return neutral();
    if (a <= l and r <= b) return seg[p];
    int m = (l+r)/2;
    return merge(qry(a, b, 2*p, l, m), qry(a, b, 2*p+1, m+1, r));
}
// update -> Para todo v[i] += x, com a <= i <= b.
int upd(int a, int b, int x, int p=1, int l=0, int r=n-1) {
    prop(p, l, r);
    if (a <= l and r <= b) {
                lazy[p] += x;
                prop(p, l, r);
                return seg[p];
        }
        if (b < l or r < a) return seg[p];
        int m = (l+r)/2;
        return seg[p] = merge(upd(a, b, x, 2*p, l, m), upd(a, b, x, 2*p+1, m+1,
            r));
}
// Funcao para visitar as folhas, ou seja, o v[]
void get_leafs(int p = 1, int l = 0, int r = n - 1) { // O(n)
    prop(p, l, r);
    if (l == r) {
        leafs[l] = seg[p];
        return;
    }
    int m = (l+r)/2;
    get_leafs(2*p, l, m);
    get_leafs(2*p+1, m+1, r);
};
// primeira posicao >= x em [a, b] (ou -1, caso nao exista)
// Obs.: So funciona na SegTree de Maximos
int first_above(int x, int a, int b, int p=1, int l=0, int r=n-1) { // O(log(n))
    if (b < l or r < a or seg[p] < x) return -1;
        if (l == r) return l;
        int m = (l+r)/2;
        int left = first_above(x, a, b, 2*p, l, m);
        if (left != -1) return left;
        return first_above(x, a, b, 2*p+1, m+1, r);
}
// ultima posicao >= x em [a, b] (ou -1, caso nao exista)
// Obs.: So funciona na SegTree de Maximos
int last_above(int x, int a, int b, int p=1, int l=0, int r=n-1) { // O(log(n))
    if (b < l or r < a or seg[p] < x) return -1;
        if (l == r) return l;
        int m = (l+r)/2;
        int right = last_above(x, a, b, 2*p+1, m+1, r);
        if (right != -1) return right;
        return last_above(x, a, b, 2*p, l, m);
}
```

20.12 Trie (62098e)

```
#include<bits/stdc++.h>
using namespace std;

const int N = 3e5 + 9;


struct Trie {
  static const int B = 31;
  struct node {
    node *nxt[2];
    int sz;
    node() {
      nxt[0] = nxt[1] = NULL;
      sz = 0;
    }
  } *root;
  Trie() { root = new node(); }
  void insert(int val) {
    node *cur = root;
    cur->sz++;
    for (int i = B - 1; i >= 0; i--) {
      int b = val >> i & 1;
      if (cur->nxt[b] == NULL)
        cur->nxt[b] = new node();
      cur = cur->nxt[b];
      cur->sz++;
    }
  }
  int query(int x, int k) { // number of values s.t. val ^ x < k
    node *cur = root;
    int ans = 0;
    for (int i = B - 1; i >= 0; i--) {
      if (cur == NULL)
```

```
        break;
      int b1 = x >> i & 1, b2 = k >> i & 1;
      if (b2 == 1) {
        if (cur->nxt[b1])
          ans += cur->nxt[b1]->sz;
        cur = cur->nxt[!b1];
      } else
        cur = cur->nxt[b1];
    }
    return ans;
  }
  int get_max(int x) { // returns maximum of val ^ x
    node *cur = root;
    int ans = 0;
    for (int i = B - 1; i >= 0; i--) {
      int k = x >> i & 1;
      if (cur->nxt[!k])
        cur = cur->nxt[!k], ans <<= 1, ans++;
      else
        cur = cur->nxt[k], ans <<= 1;
    }
    return ans;
  }
  int get_min(int x) { // returns minimum of val ^ x
    node *cur = root;
    int ans = 0;
    for (int i = B - 1; i >= 0; i--) {
      int k = x >> i & 1;
      if (cur->nxt[k])
        cur = cur->nxt[k], ans <<= 1;
      else
        cur = cur->nxt[!k], ans <<= 1, ans++;
    }
    return ans;
  }
  void del(node *cur) {
    for (int i = 0; i < 2; i++)
      if (cur->nxt[i])
        del(cur->nxt[i]);
    delete (cur);
  }

  void erase(int x) {
    node *cur = root;
    cur->sz--;

    for (int i = B - 1; i >= 0; i--) {
      int b = x >> i & 1;

      cur->nxt[b]->sz--;

      if (cur->nxt[b]->sz == 0) {
        del(cur->nxt[b]);
        cur->nxt[b] = NULL;
        return;
      }
      cur = cur->nxt[b];
    }
  }
} t;

int32_t main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  int n, k;
  cin >> n >> k;
  int cur = 0;
  long long ans = 1LL * n * (n + 1) / 2;
  t.insert(cur);
  for (int i = 0; i < n; i++) {
    int x;
    cin >> x;
    cur ^= x;
    ans -= t.query(cur, k);
    t.insert(cur);
  }
  cout << ans << '\n';
  return 0;
}
```

20.13 centroidDec (d056e5)

```
#include<bits/stdc++.h>
using namespace std;

const int N = 1e5 + 9;

vector<int> g[N];
int sz[N];
int tot, done[N], cenpar[N];

// done -> marcar um node que foi escolhido como centroid
// cenpar -> pai de cada vertice na arvore de centroides

void calc_sz(int u, int p) {
  tot ++;
  sz[u] = 1;
  for (auto v : g[u]) {
    if(v == p || done[v]) continue;
    calc_sz(v, u);
    sz[u] += sz[v];
  }
}
int find_cen(int u, int p) {
  for (auto v : g[u]) {
    if(v == p || done[v]) continue;
    else if(sz[v] > tot / 2) return find_cen(v, u);
  }
  return u;
}
void decompose(int u, int pre) {
  tot = 0;
  calc_sz(u, pre);
  int cen = find_cen(u, pre);
  cenpar[cen] = pre;
  done[cen] = 1;
  for(auto v : g[cen]) {
    if(v == pre || done[v]) continue;
    decompose(v, cen);
  }
}
int dep[N];
void dfs(int u, int p = 0) {
  for(auto v : g[u]) {
    if(v == p) continue;
    dep[v] = dep[u] + 1;
    dfs(v, u);
  }
}
```

20.14 dsuOnTree (45cc9f)

```
#include <bits/stdc++.h>

using namespace std;

const int N = 1e5+9;

vector<int> g[N];
int ans[N], col[N], sz[N], cnt[N];
bool big[N];
void dfs(int u, int p) {
  sz[u] = 1;
  for (auto v : g[u]) {
    if (v == p) continue;
    dfs(v, u);
    sz[u] += sz[v];
  }
}
void add(int u, int p, int x) {
  cnt[col[u]] += x;
  for (auto v : g[u]) {
```

```
    if (v == p || big[v] == 1) continue;
    add(v, u, x);
  }
}
void dsu(int u, int p, bool keep) {
  int bigchild = -1, mx = -1;
  for (auto v : g[u]) {
    if (v == p) continue;
    if (sz[v] > mx) mx = sz[v], bigchild = v;
  }
  for (auto v : g[u]) {
    if (v == p || v == bigchild) continue;
    dsu(v, u, 0);
  }
  if (bigchild != -1) dsu(bigchild, u, 1), big[bigchild] = 1;
  add(u, p, 1);
  ans[u] = cnt[u];
  if (bigchild != -1) big[bigchild] = 0;
  if (keep == 0) add(u, p, -1);
}
```

20.15 dsuRollback (b8745d)

```
#include <bits/stdc++.h>
using namespace std;

struct DSU_Rollback {
    vector<int> par, sz;
    vector<pair<int, int>> history;
    int components;

    DSU_Rollback(int n) {
        par.resize(n + 1);
        sz.assign(n + 1, 1);
        components = n;
        iota(par.begin(), par.end(), 0);
    }

    int find(int u) {
        while (u != par[u]) {
            u = par[u];
        }
        return u;
    }

    bool merge(int u, int v) {
        u = find(u);
        v = find(v);

        if (u == v) return false;

        if (sz[u] > sz[v]) swap(u, v);

        history.push_back({u, v});

        par[u] = v;
        sz[v] += sz[u];
        components--;

        return true;
    }

    int time() {
        return history.size();
    }

    void rollback(int t) {
        while (history.size() > t) {
            int u = history.back().first;
            int v = history.back().second;
            history.pop_back();

            sz[v] -= sz[u];
            par[u] = u;
            components++;
        }
    }
};
```

20.16 dynamicSegTree (c01219)

```
#include <bits/stdc++.h>
using namespace std;

struct DynamicSegTree {
    // A estrutura do no fica escondida aqui dentro
    struct Node {
        long long sum;
        Node *l, *r;
        Node() : sum(0), l(nullptr), r(nullptr) {}
    };

    Node* root;
    long long min_val, max_val;

    // Construtor: define o limite minimo e maximo do intervalo global
    DynamicSegTree(long long min_val, long long max_val) : min_val(min_val),
         max_val(max_val) {
        root = new Node();
    }

    // Update interno (Recursivo)
    void update(Node*& node, long long l, long long r, long long idx, long long
         val) {
        if (!node) node = new Node();
        if (l == r) {
            node->sum += val; // Altere para '=' se for substituicao
            return;
        }

        long long m = l + (r - l) / 2; // Evita bugs de overflow e com numeros
             negativos
        if (idx <= m) {
            update(node->l, l, m, idx, val);
        } else {
            update(node->r, m + 1, r, idx, val);
        }

        long long leftSum = node->l ? node->l->sum : 0;
        long long rightSum = node->r ? node->r->sum : 0;
        node->sum = leftSum + rightSum;
    }

    // Query interna (Recursiva)
    long long query(Node* node, long long l, long long r, long long ql, long
         long qr) {
        if (!node || ql > r || qr < l) return 0;
        if (ql <= l && r <= qr) return node->sum;

        long long m = l + (r - l) / 2;
        return query(node->l, l, m, ql, qr) + query(node->r, m + 1, r, ql, qr);
    }

    // --- FUNCOES PARA USAR NA MAIN ---
    void update(long long idx, long long val) {
        update(root, min_val, max_val, idx, val);
    }

    long long query(long long ql, long long qr) {
        return query(root, min_val, max_val, ql, qr);
    }
};

/* COMO USAR NA MAIN:
 * // Cria uma arvore dinamica para um intervalo de 1 ate 1 bilhao
 * DynamicSegTree st(1, 1000000000);
 * st.update(50000, 10);      // Soma 10 no indice 50.000
 * st.update(999999999, 5);  // Soma 5 no indice 999.999.999
 * long long ans = st.query(1, 1000000000); // Retorna 15
 */
```

20.17 implicitTreap (3f66c1)

```cpp
#include <bits/stdc++.h>

using namespace std;

// Gerador de numeros aleatorios robusto
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

struct ImplicitTreap {
    struct Node {
        int val;       // Valor do elemento
        int prior;     // Prioridade (Heap)
        int sz;        // Tamanho da subarvore

        // --- ADICIONE SUAS VARIAVEIS DE QUERY/LAZY AQUI ---
        long long sum; // Exemplo de Query: Soma do subsegmento
        bool rev;      // Exemplo de Lazy: Inverter o subsegmento

        int l, r;      // Filhos esquerdo e direito (usando indices)

        Node() : val(0), prior(0), sz(0), sum(0), rev(false), l(0), r(0) {}
        Node(int v) : val(v), prior(rng()), sz(1), sum(v), rev(false), l(0), r
            (0) {}
    };

    vector<Node> tree;
    int root;

    ImplicitTreap() {
        // O indice 0 age como um no nulo (nullptr)
        tree.push_back(Node());
        root = 0;
    }

    int newNode(int val) {
        tree.push_back(Node(val));
        return tree.size() - 1;
    }

    int sz(int t) { return tree[t].sz; }

    // Atualiza as informacoes do no baseado nos filhos (Cima para Baixo)
    void upd(int t) {
        if (!t) return;
        tree[t].sz = 1 + sz(tree[t].l) + sz(tree[t].r);

        // --- ATUALIZE SUAS QUERIES AQUI ---
        tree[t].sum = tree[t].val + tree[tree[t].l].sum + tree[tree[t].r].sum;
    }

    // Propaga as atualizacoes preguicosas para os filhos (Baixo para Cima)
    void push(int t) {
        if (!t) return;

        // --- APLIQUE E PROPAGUE SUAS LAZY TAGS AQUI ---
        if (tree[t].rev) {
            swap(tree[t].l, tree[t].r); // Inverte os filhos
            if (tree[t].l) tree[tree[t].l].rev ^= true;
            if (tree[t].r) tree[tree[t].r].rev ^= true;
            tree[t].rev = false; // Limpa a tag
        }
    }

    // Divide a arvore t em duas: l (tamanho k) e r (o restante)
    void split(int t, int k, int &l, int &r) {
        if (!t) {
            l = r = 0;
            return;
        }
        push(t); // SEMPRE de push antes de descer na arvore

        int left_sz = sz(tree[t].l);
        if (left_sz >= k) {
            split(tree[t].l, k, l, tree[t].l);
            r = t;
        } else {
            split(tree[t].r, k - left_sz - 1, tree[t].r, r);
            l = t;
        }
        upd(t); // SEMPRE de update depois de modificar a estrutura
    }

    // Une duas arvores l e r em uma unica arvore t
    void merge(int &t, int l, int r) {
        push(l); push(r); // SEMPRE de push antes de mesclar

        if (!l || !r) {
            t = l ? l : r;
            return;
        }
        if (tree[l].prior > tree[r].prior) {
            merge(tree[l].r, tree[l].r, r);
            t = l;
        } else {
            merge(tree[r].l, l, tree[r].l);
            t = r;
        }
        upd(t);
    }

    // --- OPERACOES DE ALTO NIVEL ---

    // Insere um valor na posicao 'pos' (0-indexed)
    void insert(int pos, int val) {
        int t1, t2;
        split(root, pos, t1, t2);
        int novo = newNode(val);
        merge(root, t1, novo);
        merge(root, root, t2);
    }

    // Remove o elemento da posicao 'pos' (0-indexed)
    void erase(int pos) {
        int t1, t2, t3;
        split(root, pos, t1, t2);
        split(t2, 1, t2, t3);
        // t2 e o no a ser removido (fica isolado e e ignorado)
        merge(root, t1, t3);
    }

    // Exemplo: Inverte um subarray [L, R]
    void reverse_range(int L, int R) {
        int t1, mid, t2;
        split(root, L, t1, mid);
        split(mid, R - L + 1, mid, t2);

        tree[mid].rev ^= true; // Aplica a Lazy Tag

        merge(root, t1, mid);
        merge(root, root, t2);
    }

    // Exemplo: Retorna a soma do subarray [L, R]
    long long query_sum(int L, int R) {
        int t1, mid, t2;
        split(root, L, t1, mid);
        split(mid, R - L + 1, mid, t2);

        long long res = tree[mid].sum; // Coleta a resposta

        merge(root, t1, mid);
        merge(root, root, t2);
        return res;
    }

    // Funcao auxiliar para imprimir (in-order)
    void print(int t) {
        if (!t) return;
        push(t);
        print(tree[t].l);
        cout << tree[t].val << " ";
        print(tree[t].r);
    }
```

```
};
```

20.18 lazyDynamicSegTree (a9f7bc)

```
#include <bits/stdc++.h>
using namespace std;

template<typename T>
struct DynamicLazySegTree {
    struct Node {
        T sum, lazy;
        int l, r; // Indices dos filhos no vector
        Node() : sum(0), lazy(0), l(0), r(0) {}
    };

    vector<Node> tree;
    long long min_val, max_val;
    T neutro = 0;

    // Construtor: define o limite do intervalo global (ex: 1 ate 10^9)
    DynamicLazySegTree(long long min_val, long long max_val) : min_val(min_val),
         max_val(max_val) {
        tree.push_back(Node()); // Indice 0 e o no nulo (vazio e imutavel)
        tree.push_back(Node()); // Indice 1 e a raiz global da arvore
    }

    // Funcao encapsulada para criar nos com seguranca
    int create_node() {
        tree.push_back(Node());
        return tree.size() - 1;
    }

    // Resolve a preguica pendente e garante a criacao dos filhos
    void push(int node, long long l, long long r) {
        if (tree[node].lazy != 0) {
            // Aplica a soma no no atual (val * tamanho do intervalo)
            tree[node].sum += tree[node].lazy * (r - l + 1);

            // Se nao for folha, propaga para os filhos
            if (l != r) {
                // Cria os filhos dinamicamente caso nao existam
                if (tree[node].l == 0) {
                    int new_l = create_node();
                    tree[node].l = new_l;
                }
                if (tree[node].r == 0) {
                    int new_r = create_node();
                    tree[node].r = new_r;
                }
                // Passa a preguica para frente
                tree[tree[node].l].lazy += tree[node].lazy;
                tree[tree[node].r].lazy += tree[node].lazy;
            }
            tree[node].lazy = 0;
        }
    }

    // Update de intervalo
    void update(int node, long long l, long long r, long long ql, long long qr,
         T val) {
        push(node, l, r);
        if (ql > r || qr < l) return;

        if (ql <= l && r <= qr) {
            tree[node].lazy += val;
            push(node, l, r);
            return;
        }

        long long mid = l + (r - l) / 2;

        // Garante que os filhos existem antes de descer neles
        if (tree[node].l == 0) {
            int new_l = create_node();
            tree[node].l = new_l;
        }
        if (tree[node].r == 0) {
            int new_r = create_node();
            tree[node].r = new_r;
        }

        // SALVA OS INDICES EM VARIAVEIS ANTES DA RECURSAO (Prevencao de RTE!)
        int esq = tree[node].l;
        int dir = tree[node].r;

        update(esq, l, mid, ql, qr, val);
        update(dir, mid + 1, r, ql, qr, val);

        tree[node].sum = tree[tree[node].l].sum + tree[tree[node].r].sum;
    }

    // Query de intervalo
    T query(int node, long long l, long long r, long long ql, long long qr) {
        // Se o no e 0, significa que essa parte da arvore nunca foi tocada
        if (node == 0 || ql > r || qr < l) return neutro;

        push(node, l, r);
        if (ql <= l && r <= qr) return tree[node].sum;

        long long mid = l + (r - l) / 2;
        return query(tree[node].l, l, mid, ql, qr) + query(tree[node].r, mid +
             1, r, ql, qr);
    }

    // --- FUNCOES DA MAIN ---

    // Adiciona 'val' no intervalo [l, r]
    void update(long long l, long long r, T val) {
        update(1, min_val, max_val, l, r, val);
    }

    // Retorna a soma do intervalo [l, r]
    T query(long long l, long long r) {
        return query(1, min_val, max_val, l, r);
    }
};

/* COMO USAR NA MAIN:
 * // Cria uma arvore para o intervalo de 1 ate 1 bilhao
 * DynamicLazySegTree<long long> st(1, 1000000000LL);
 * * st.update(10, 50000, 5);         // Soma 5 em todos os elementos de 10 ate
     50.000
 * st.update(40000, 1000000000LL, 2); // Soma 2 em todos de 40.000 ate
     1.000.000.000
 * * long long ans = st.query(1, 45000); // Consulta a soma
 */
```

20.19 lazyPropagation (1ab4b7)

```
#include <bits/stdc++.h>
using namespace std;

template<typename T>
struct LazySegTree {
    int n;
    vector<T> tree, lazy;
    T neutro = 0; // Elemento neutro para a soma

    // Construtor
    LazySegTree(int n) : n(n) {
        tree.assign(4 * n, neutro);
        lazy.assign(4 * n, 0); // 0 indica que nao ha atualizacao pendente
    }

    T combina(T a, T b) {
        return a + b;
    }

    // Resolve a preguica pendente no no atual e repassa para os filhos
    void push(int node, int l, int r) {
```

```
        if (lazy[node] != 0) {
            // Aplica a atualizacao no no atual.
            // Como e soma de intervalo, multiplicamos o valor pelo tamanho do
                 intervalo.
            tree[node] += lazy[node] * (r - l + 1);

            // Se nao for folha, repassa a preguica para os filhos
            if (l != r) {
                lazy[2 * node] += lazy[node];
                lazy[2 * node + 1] += lazy[node];
            }

            // Limpa a preguica do no atual
            lazy[node] = 0;
        }
    }

    // Build interno
    void build(int node, int l, int r, const vector<T>& arr) {
        if (l == r) {
            tree[node] = arr[l];
            return;
        }
        int mid = l + (r - l) / 2;
        build(2 * node, l, mid, arr);
        build(2 * node + 1, mid + 1, r, arr);
        tree[node] = combina(tree[2 * node], tree[2 * node + 1]);
    }

    // Update interno em intervalo [ql, qr]
    void update(int node, int l, int r, int ql, int qr, T val) {
        push(node, l, r); // Sempre da push antes de interagir com o no

        if (ql > r || qr < l) return; // Fora do escopo

        if (ql <= l && r <= qr) { // Totalmente dentro do escopo
            lazy[node] += val; // Acumula a preguica
            push(node, l, r);  // Atualiza este no e passa a preguica pra baixo
            return;
        }

        int mid = l + (r - l) / 2;
        update(2 * node, l, mid, ql, qr, val);
        update(2 * node + 1, mid + 1, r, ql, qr, val);

        tree[node] = combina(tree[2 * node], tree[2 * node + 1]);
    }

    // Query interna no intervalo [ql, qr]
    T query(int node, int l, int r, int ql, int qr) {
        push(node, l, r); // Sempre da push antes de ler o valor de um no

        if (ql > r || qr < l) return neutro;
        if (ql <= l && r <= qr) return tree[node];

        int mid = l + (r - l) / 2;
        return combina(query(2 * node, l, mid, ql, qr),
                       query(2 * node + 1, mid + 1, r, ql, qr));
    }

    // --- FUNCOES PARA USAR NA MAIN ---

    void build(const vector<T>& arr) {
        build(1, 0, n - 1, arr);
    }

    // Adiciona 'val' em todos os elementos no intervalo [l, r]
    void update(int l, int r, T val) {
        update(1, 0, n - 1, l, r, val);
    }

    // Retorna a soma do intervalo [l, r]
    T query(int l, int r) {
        return query(1, 0, n - 1, l, r);
    }
};

/* COMO USAR NA MAIN:
 * vector<int> arr = {1, 2, 3, 4, 5};
 * LazySegTree<int> st(arr.size());
 * st.build(arr);
 * * st.update(1, 3, 10); // Adiciona 10 no intervalo [1, 3]. Array fica: {1,
     12, 13, 14, 5}
 * int ans = st.query(1, 4); // Consulta a soma de [1, 4] -> 12 + 13 + 14 + 5 =
     44
 */
```

20.20 persistentSegTree (b68401)

```
#include <bits/stdc++.h>
using namespace std;

template<typename T>
struct PersistentSegTree {
    struct Node {
        T sum;
        int l, r; // Indices para o filho esquerdo e direito no vector 'tree'
        Node(T sum = 0, int l = 0, int r = 0) : sum(sum), l(l), r(r) {}
    };

    int n;
    vector<Node> tree;
    vector<int> roots; // Armazena o indice da raiz de cada versao
    T neutro = 0;      // Elemento neutro para a soma

    // Construtor
    PersistentSegTree(int n) : n(n) {
        // O indice 0 sera o nosso no "nulo" ou vazio
        tree.push_back(Node(neutro, 0, 0));
        roots.push_back(0); // A versao 0 aponta para o no nulo inicialmente
    }

    T combina(T a, T b) {
        return a + b;
    }

    // Cria um novo no copiando os dados de um no existente
    int clone(int node_idx) {
        tree.push_back(tree[node_idx]);
        return tree.size() - 1;
    }

    // Build interno (Constroi a versao inicial baseada em um array)
    int build(int l, int r, const vector<T>& arr) {
        int node = tree.size();
        tree.push_back(Node()); // Cria um novo no

        if (l == r) {
            tree[node].sum = arr[l];
            return node;
        }

        int mid = l + (r - l) / 2;
        tree[node].l = build(l, mid, arr);
        tree[node].r = build(mid + 1, r, arr);
        tree[node].sum = combina(tree[tree[node].l].sum, tree[tree[node].r].sum)
            ;

        return node;
    }

    // Update interno (Retorna o indice da nova raiz)
    int update(int prev_root, int l, int r, int pos, T val) {
        // Criamos uma copia do no atual da versao anterior
        int node = clone(prev_root);

        if (l == r) {
            tree[node].sum += val; // Altere para '=' se for substituicao
            return node;
        }

        int mid = l + (r - l) / 2;
        if (pos <= mid) {
```

```
            // Se a atualizacao for na esquerda, a direita se mantem apontando
                 para o no antigo
            tree[node].l = update(tree[prev_root].l, l, mid, pos, val);
        } else {
            // Se a atualizacao for na direita, a esquerda se mantem apontando
                 para o no antigo
            tree[node].r = update(tree[prev_root].r, mid + 1, r, pos, val);
        }

        tree[node].sum = combina(tree[tree[node].l].sum, tree[tree[node].r].sum)
             ;
        return node;
    }

    // Query interna no intervalo [ql, qr] em uma raiz especifica
    T query(int node, int l, int r, int ql, int qr) {
        if (node == 0 || ql > r || qr < l) return neutro;
        if (ql <= l && r <= qr) return tree[node].sum;

        int mid = l + (r - l) / 2;
        return combina(query(tree[node].l, l, mid, ql, qr),
                       query(tree[node].r, mid + 1, r, ql, qr));
    }

    // --- FUNCOES PARA USAR NA MAIN ---

    // Constroi a versao 0 a partir de um array.
    // Retorna o indice da versao criada (sempre sera 1 se chamado logo apos
         instanciar)
    int build(const vector<T>& arr) {
        int root_idx = build(0, n - 1, arr);
        roots.push_back(root_idx);
        return roots.size() - 1; // Retorna o ID da versao recem-criada
    }

    // Atualiza um elemento partindo de uma versao base.
    // Retorna o ID da nova versao criada.
    int update(int base_version, int pos, T val) {
        int new_root = update(roots[base_version], 0, n - 1, pos, val);
        roots.push_back(new_root);
        return roots.size() - 1; // Retorna o ID da nova versao
    }

    // Consulta no intervalo [l, r] de uma versao especifica
    T query(int version, int l, int r) {
        return query(roots[version], 0, n - 1, l, r);
    }
};

/* COMO USAR NA MAIN:
 * vector<int> arr = {1, 2, 3, 4, 5};
 * PersistentSegTree<int> pst(arr.size());
 * * int v1 = pst.build(arr);              // v1 = 1. Versao 1: {1, 2, 3, 4, 5}
 * int v2 = pst.update(v1, 2, 10);      // v2 = 2. Versao 2: {1, 2, 13, 4, 5} (
      Soma 10 na pos 2)
 * int v3 = pst.update(v2, 4, -2);      // v3 = 3. Versao 3: {1, 2, 13, 4, 3} (
      Soma -2 na pos 4)
 * * int ans1 = pst.query(v1, 1, 3); // Consulta na versao 1 [1,3] -> 2 + 3 + 4
      = 9
 * int ans2 = pst.query(v2, 1, 3); // Consulta na versao 2 [1,3] -> 2 + 13 + 4 =
      19
 * int ans3 = pst.query(v3, 2, 4); // Consulta na versao 3 [2,4] -> 13 + 4 + 3 =
      20
 */
```

20.21 segTreeIterativa (30fb73)

```
#include <bits/stdc++.h>
using namespace std;

template <class T> struct SegTreeIterative {
  int n;
  vector<T> seg;
  T neutro = 0; // Elemento neutro

  SegTreeIterative(int _n) {
    n = 1;
    while (n < _n)
      n *= 2; // Arredonda para a proxima potencia de 2
    seg.assign(2 * n, neutro);
  }

  T combina(T a, T b) { return a + b; }

  // Atualiza a posicao p somando val
  void update(int p, T val) {
    p += n;
    seg[p] += val; // Altere para '=' se for substituicao
    for (p /= 2; p > 0; p /= 2) {
      seg[p] = combina(seg[2 * p], seg[2 * p + 1]);
    }
  }

  // Consulta no intervalo [l, r]
  T query(int l, int r) {
    T res_esq = neutro, res_dir = neutro;
    for (l += n, r += n + 1; l < r; l /= 2, r /= 2) {
      if (l % 2 == 1)
        res_esq = combina(res_esq, seg[l++]);
      if (r % 2 == 1)
        res_dir = combina(seg[--r], res_dir);
    }
    return combina(res_esq, res_dir);
  }
};
```

20.22 structureCentroidD (0b015f)

```
#include <bits/stdc++.h>

using namespace std;

typedef long long ll;

const int N = (int)1e5 + 5;
const int inf = (int)1e9;

struct CentroidDecomposition {
  set<int> G[N];
  map<int, int> dis[N];
  int sz[N], pa[N], ans[N];

  void init(int n) {
    for(int i = 1 ; i <= n ; ++i) G[i].clear(), dis[i].clear(), ans[i] = inf;
  }
  void addEdge(int u, int v) {
    G[u].insert(v); G[v].insert(u);
  }
  int dfs(int u, int p) {
    sz[u] = 1;
    for(auto v : G[u]) if(v != p) {
      sz[u] += dfs(v, u);
    }
    return sz[u];
  }
  int centroid(int u, int p, int n) {
    for(auto v : G[u]) if(v != p) {
      if(sz[v] > n / 2) return centroid(v, u, n);
    }
    return u;
  }
  void dfs2(int u, int p, int c, int d) { // build distance
    dis[c][u] = d;
    for(auto v : G[u]) if(v != p) {
      dfs2(v, u, c, d + 1);
    }
  }
  void build(int u, int p) {
    int n = dfs(u, p);
    int c = centroid(u, p, n);
```

```
    if(p == -1) p = c;
    pa[c] = p;
    dfs2(c, p, c, 0);

    vector<int> tmp(G[c].begin(), G[c].end());
    for(auto v : tmp) {
      G[c].erase(v); G[v].erase(c);
      build(v, c);
    }
  }
  void modify(int u) {
    for(int v = u ; v != 0 ; v = pa[v]) ans[v] = min(ans[v], dis[v][u]);
  }
  int query(int u) {
    int mn = inf;
    for(int v = u ; v != 0 ; v = pa[v]) mn = min(mn, ans[v] + dis[v][u]);
    return mn;
  }
} cd;
```

20.23 treap (b7a3fb)

```
#include <bits/stdc++.h>

using namespace std;

// Gerador de numeros aleatorios robusto
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

struct Treap {
    struct Node {
        int key;       // Chave de busca (mantem a propriedade da BST)
        int prior;     // Prioridade (mantem a propriedade do Heap)
        int sz;        // Tamanho da subarvore (Crucial para Order Statistics)
        int l, r;      // Filhos esquerdo e direito (indices)

        Node() : key(0), prior(0), sz(0), l(0), r(0) {}
        Node(int k) : key(k), prior(rng()), sz(1), l(0), r(0) {}
    };

    vector<Node> tree;
    int root;

    Treap() {
        tree.push_back(Node()); // Indice 0 atua como nullptr seguro
        root = 0;
    }

    int newNode(int key) {
        tree.push_back(Node(key));
        return tree.size() - 1;
    }

    int sz(int t) { return tree[t].sz; }

    // Atualiza as informacoes do no (neste caso, apenas o tamanho)
    void upd(int t) {
        if (!t) return;
        tree[t].sz = 1 + sz(tree[t].l) + sz(tree[t].r);
    }

    // Divide a arvore t em duas: l (chaves <= key) e r (chaves > key)
    void split(int t, int key, int &l, int &r) {
        if (!t) {
            l = r = 0;
            return;
        }
        // Se a chave atual for menor ou igual, ela e todo seu filho esquerdo
             vao para 'l'
        if (tree[t].key <= key) {
            split(tree[t].r, key, tree[t].r, r);
            l = t;
        }
        // Se a chave atual for maior, ela e todo seu filho direito vao para 'r'
        else {
            split(tree[t].l, key, l, tree[t].l);
            r = t;
        }
        upd(t); // Sempre atualize apos modificar a estrutura
    }

    // Une duas arvores l e r. ATENCAO: O maior valor em 'l' deve ser <= ao
         menor em 'r'
    void merge(int &t, int l, int r) {
        if (!l || !r) {
            t = l ? l : r;
            return;
        }
        // O no com maior prioridade vira a raiz
        if (tree[l].prior > tree[r].prior) {
            merge(tree[l].r, tree[l].r, r);
            t = l;
        } else {
            merge(tree[r].l, l, tree[r].l);
            t = r;
        }
        upd(t);
    }

    // --- OPERACOES PADRAO DE BST ---

    // Insere uma nova chave (Esta implementacao permite duplicatas. Comporta-se
         como multiset)
    void insert(int key) {
        int t1, t2;
        split(root, key, t1, t2);
        int novo = newNode(key);
        merge(root, t1, novo);
        merge(root, root, t2);
    }

    // Remove UMA ocorrencia da chave (se existir)
    void erase(int key) {
        int t1, mid, t2;
        // Isola os nos menores que a chave
        split(root, key - 1, t1, mid);
        // Do que sobrou, isola os nos estritamente iguais a chave
        split(mid, key, mid, t2);

        // Se houver algum no igual a chave, removemos apenas a raiz desse grupo
             (uma ocorrencia)
        if (mid) {
            merge(mid, tree[mid].l, tree[mid].r);
        }

        // Junta tudo de volta
        merge(root, t1, mid);
        merge(root, root, t2);
    }

    // --- OPERACOES ESPECIAIS (ORDER STATISTICS) ---

    // Retorna a K-esima menor chave (1-indexed). Retorna -1 se K for invalido.
    int kth_element(int k) {
        int cur = root;
        while (cur) {
            int left_sz = sz(tree[cur].l);
            if (k == left_sz + 1) return tree[cur].key; // Encontrou!

            if (k <= left_sz) {
                cur = tree[cur].l; // Busca na esquerda
            } else {
                k -= (left_sz + 1); // Desconta os elementos ja passados
                cur = tree[cur].r;  // Busca na direita
            }
        }
        return -1;
    }

    // Conta quantos elementos na Treap sao estritamente MENORES que 'key' (
         Equivalente a posicao do elemento)
    int count_less(int key) {
        int t1, t2;
```

```
        split(root, key - 1, t1, t2);
        int res = sz(t1);
        merge(root, t1, t2);
        return res;
    }

    // Funcao auxiliar para imprimir as chaves em ordem crescente
    void print(int t) {
        if (!t) return;
        print(tree[t].l);
        cout << tree[t].key << " ";
        print(tree[t].r);
    }
    void printAll() { print(root); cout << "\n"; }
};
```

21 Techniques

21.1 CoordinateCompression (eca1bb)

```
#include <bits/stdc++.h>
using namespace std;
// Considere que 1 <= ai <= 10^9 e 1 <= |a| <= 10^5.
// Vamos comprimir os valores de ai para  [0, 10^5-1]. -> 0-INDEXED
map<int, int> coord; // mantem a propriedade de ordenacao dos valores
void coordinate_compression(vector<int> &v) {
    set<int> s; for (auto x : v) s.insert(x);
    int idx = 0;
    for (auto x : s) {
        coord[x] = idx;
        idx++;
    }
    for (int i = 0; i < (int) v.size(); i++) {
        v[i] = coord[v[i]];
    }
}
```

21.2 Grid (ed0740)

```
#include <bits/stdc++.h>
using namespace std;
#define endl '\n'
#define ff first
#define ss second
typedef pair<int, int> pi;
const int MAX = 1e3;
int n, m;
char grid[MAX][MAX];
bool vis[MAX][MAX];
// Movimentos: cima, baixo, esquerda, direita
pi mov[16] = {
    // Esquerda, Direita, Cima, Baixo
    {-1, 0}, {1, 0}, {0, -1}, {0, 1},
    // Diagonais
    {-1, -1}, {1, 1}, {-1, 1}, {1, -1},
    // Movimento de cavalo
    {-2, -1}, {2, 1}, {-2, 1}, {2, -1},
    {-1, -2}, {1, 2}, {-1, 2}, {1, -2}
};
// Funcao para validar de new_i, new_j sao validos
bool valid(int i, int j) {
    // Podemos adicionar outras CONDICOES...
    // Por exemplo, "Ja foi visitado", "movimento nao e obstruido"
    return i >= 0 and j >= 0 and i < n and j < m and !vis[i][j];
}
signed main() {
    cin >> n >> m;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            cin >> grid[i][j];
        }
    }
    int i = 0, j = 0; // Posicao Inicial
    for (auto [di, dj] : mov) {
        int ni = i + di, nj = j + dj;
        if (valid(ni, nj)) {
            vis[ni][nj] = true;
            // Faca algo!
        }
    }
}
```

22 Teoria

22.1 STL

Os templates da STL são estruturas de dados e algoritmos já implementados em C++ que facilitam as implementações, além de serem muito eficientes. Em geral, todas estão incluídas no cabeçalho `<bits/stdc++.h>`. As estruturas são templates genéricos, podendo ser usadas com qualquer tipo; todos os exemplos a seguir são com `int` apenas por motivos de simplicidade.

22.1.1 Vector

Um vetor dinâmico (que pode crescer e diminuir de tamanho).

- `vector<int> v(n, x)`: Cria um vetor de inteiros com `n` elementos inicializados com `x` – $O(n)$.
- `v.push_back(x)`: Adiciona o elemento `x` no final do vetor – $O(1)$.
- `v.pop_back()`: Remove o último elemento do vetor – $O(1)$.
- `v.size()`: Retorna o tamanho do vetor – $O(1)$.
- `v.empty()`: Retorna `true` se o vetor estiver vazio – $O(1)$.
- `v.clear()`: Remove todos os elementos do vetor – $O(n)$.
- `v.front()`: Retorna o primeiro elemento do vetor – $O(1)$.
- `v.back()`: Retorna o último elemento do vetor – $O(1)$.
- `v.begin()`: Retorna um iterador para o primeiro elemento do vetor – $O(1)$.
- `v.end()`: Retorna um iterador para o elemento seguinte ao último do vetor – $O(1)$.
- `v.insert(it, x)`: Insere o elemento `x` na posição apontada pelo iterador `it` – $O(n)$.
- `v.erase(it)`: Remove o elemento apontado pelo iterador `it` – $O(n)$.
- `v.erase(it1, it2)`: Remove elementos no intervalo `[it1, it2)` – $O(n)$.
- `v.resize(n)`: Redimensiona o vetor para `n` elementos – $O(n)$.
- `v.resize(n, x)`: Redimensiona o vetor para `n` elementos, todos inicializados com `x` – $O(n)$.

22.1.2 Pair

Um par de elementos (de tipos possivelmente diferentes).

- `pair<int, int> p`: Cria um par de inteiros – $O(1)$.
- `p.first`: Retorna o primeiro elemento do par – $O(1)$.
- `p.second`: Retorna o segundo elemento do par – $O(1)$.

22.1.3 Set

Um conjunto de elementos únicos. Por baixo, é uma árvore de busca binária balanceada.

- `set<int> s`: Cria um conjunto de inteiros – $O(1)$.
- `s.insert(x)`: Insere o elemento `x` no conjunto – $O(\log n)$.
- `s.erase(x)`: Remove o elemento `x` do conjunto – $O(\log n)$.
- `s.find(x)`: Retorna um iterador para `x`; se `x` não existir, retorna `s.end()` – $O(\log n)$.
- `s.size()`: Retorna o tamanho do conjunto – $O(1)$.
- `s.empty()`: Retorna `true` se estiver vazio – $O(1)$.
- `s.clear()`: Remove todos os elementos – $O(n)$.
- `s.begin()`: Retorna um iterador para o primeiro elemento – $O(1)$.
- `s.end()`: Retorna um iterador para após o último elemento – $O(1)$.

22.1.4 Multiset

Basicamente um `set`, mas permite elementos repetidos. Possui todos os métodos de um `set`.

Declaração: `multiset<int> ms`.

Um detalhe é que, ao usar o método `erase`, ele remove todas as ocorrências do elemento. Para remover apenas uma ocorrência, usar `ms.erase(ms.find(x))`.

22.1.5 Map

Um conjunto de pares chave-valor, onde as chaves são únicas. Por baixo, é uma árvore de busca binária balanceada.

- `map<int, int> m`: Cria um mapa de inteiros para inteiros – $O(1)$.
- `m[key]`: Retorna o valor associado à chave `key` – $O(\log n)$.
- `m[key] = value`: Associa o valor `value` à chave `key` – $O(\log n)$.
- `m.erase(key)`: Remove a chave do mapa – $O(\log n)$.
- `m.find(key)`: Retorna um iterador para o par chave-valor com chave `key`, ou `m.end()` se não existir – $O(\log n)$.
- `m.size()`: Retorna o tamanho do mapa – $O(1)$.
- `m.empty()`: Retorna `true` se estiver vazio – $O(1)$.
- `m.clear()`: Remove todos os pares chave-valor – $O(n)$.
- `m.begin()`: Retorna um iterador para o primeiro par chave-valor – $O(1)$.
- `m.end()`: Retorna um iterador para após o último par chave-valor – $O(1)$.

22.1.6 Queue

Uma fila (primeiro a entrar, primeiro a sair).

- `queue<int> q`: Cria uma fila de inteiros – $O(1)$.
- `q.push(x)`: Adiciona o elemento `x` no final da fila – $O(1)$.
- `q.pop()`: Remove o primeiro elemento da fila – $O(1)$.
- `q.front()`: Retorna o primeiro elemento da fila – $O(1)$.
- `q.size()`: Retorna o tamanho da fila – $O(1)$.
- `q.empty()`: Retorna `true` se a fila estiver vazia – $O(1)$.

22.1.7 Priority Queue

Uma fila de prioridade (o maior elemento é o primeiro a sair).

- `priority_queue<int> pq`: Cria uma fila de prioridade de inteiros – $O(1)$.
- `pq.push(x)`: Adiciona o elemento `x` na fila de prioridade – $O(\log n)$.
- `pq.pop()`: Remove o maior elemento da fila – $O(\log n)$.
- `pq.top()`: Retorna o maior elemento da fila – $O(1)$.
- `pq.size()`: Retorna o tamanho da fila de prioridade – $O(1)$.
- `pq.empty()`: Retorna `true` se a fila estiver vazia – $O(1)$.

Para fazer uma fila de prioridade em que o menor elemento sai primeiro, use `priority_queue<int, vector<int>, greater<int>> pq`.

22.1.8 Stack

Uma pilha (último a entrar, primeiro a sair).

- `stack<int> s`: Cria uma pilha de inteiros – $O(1)$.
- `s.push(x)`: Adiciona o elemento `x` no topo da pilha – $O(1)$.
- `s.pop()`: Remove o elemento do topo da pilha – $O(1)$.
- `s.top()`: Retorna o elemento do topo da pilha – $O(1)$.
- `s.size()`: Retorna o tamanho da pilha – $O(1)$.
- `s.empty()`: Retorna `true` se a pilha estiver vazia – $O(1)$.

22.1.9 Bitset

Um conjunto de bits, serve para representar máscaras quando um inteiro não é suficiente. Possui operações bitwise otimizadas pelo processador.

- `bitset<N> a`: Cria um bitset de tamanho `N` (`N` deve ser constante) – $O(1)$.
- `a[i]`: Retorna o valor do bit na posição `i` – $O(1)$.
- `a.count()`: Retorna o número de bits 1 no bitset – $O(N/w)$.
- `a.size()`: Retorna o tamanho do bitset – $O(1)$.
- `a.set(i)`: Seta o bit na posição `i` para 1 – $O(1)$.
- `a.set()`: Seta todos os bits para 1 – $O(N/w)$.
- `a.reset(i)`: Seta o bit na posição `i` para 0 – $O(1)$.
- `a.reset()`: Seta todos os bits para 0 – $O(N/w)$.
- `a.flip(i)`: Inverte o bit na posição `i` – $O(1)$.
- `a.flip()`: Inverte todos os bits – $O(N/w)$.
- `a.to_ullong()`: Retorna o valor do bitset como um inteiro – $O(N/w)$.

O bitset também suporta operações como `&`, `|`, `^`, `~`, `<<`, `>>`, entre outros.
Obs: `w` é o tamanho da palavra do processador, em geral 32 ou 64 bits.

22.1.10 Funções úteis

- `min(a, b)`: Retorna o menor entre `a` e `b` – $O(1)$.
- `max(a, b)`: Retorna o maior entre `a` e `b` – $O(1)$.
- `abs(a)`: Retorna o valor absoluto de `a` – $O(1)$.
- `swap(a, b)`: Troca os valores de `a` e `b` – $O(1)$.
- `sqrt(a)`: Retorna a raiz quadrada de `a` – $O(\log a)$.
- `ceil(a)`: Retorna o menor inteiro maior ou igual a `a` – $O(1)$.
- `floor(a)`: Retorna o maior inteiro menor ou igual a `a` – $O(1)$.
- `round(a)`: Retorna o inteiro mais próximo de `a` – $O(1)$.

22.1.11 Funções úteis para vetores

Para usar em `std::vector`, sempre passar `v.begin()` e `v.end()` como argumentos para essas funções.

Se for um vetor estilo C, usar `v` e `v + n`. Exemplo:

```
int v[10];    sort(v, v + 10);
```

Lembrete: `v.end()` é um iterador para o elemento seguinte ao último do vetor, então não é um iterador válido.

As funções de vetor em geral são da forma `[L, R)`, ou seja, `L` é incluso e `R` é exclusivo.

- `fill(v.begin(), v.end(), x)`: Preenche o vetor `v` com o valor `x` – $O(n)$.
- `sort(v.begin(), v.end())`: Ordena o vetor `v` – $O(n \log n)$.
- `reverse(v.begin(), v.end())`: Inverte o vetor `v` – $O(n)$.
- `accumulate(v.begin(), v.end(), 0)`: Soma todos os elementos do vetor `v` – $O(n)$.
- `max_element(v.begin(), v.end())`: Retorna um iterador para o maior elemento do vetor `v` – $O(n)$.
- `min_element(v.begin(), v.end())`: Retorna um iterador para o menor elemento do vetor `v` – $O(n)$.
- `count(v.begin(), v.end(), x)`: Retorna o número de ocorrências do elemento `x` no vetor `v` – $O(n)$.
- `find(v.begin(), v.end(), x)`: Retorna um iterador para a primeira ocorrência de `x` no vetor `v`, ou `v.end()` se não existir – $O(n)$.
- `lower_bound(v.begin(), v.end(), x)`: Retorna um iterador para o primeiro elemento maior ou igual a `x`; o vetor deve estar ordenado – $O(\log n)$.
- `upper_bound(v.begin(), v.end(), x)`: Retorna um iterador para o primeiro elemento estritamente maior que `x`; o vetor deve estar ordenado – $O(\log n)$.
- `next_permutation(a.begin(), a.end())`: Rearranja os elementos do vetor `a` para a próxima permutação lexicograficamente maior – $O(n)$.

22.1.12 Pragmas

Os pragmas são diretivas para o compilador, que podem ser usadas para otimizar o código.

Temos os pragmas de otimização, como por exemplo:

- `#pragma GCC optimize("O2")`: Otimizações de nível 2 (padrão de competições)
- `#pragma GCC optimize("O3")`: Otimizações de nível 3 (seguro para usar)
- `#pragma GCC optimize("Ofast")`: Otimizações agressivas (perigoso!)
- `#pragma GCC optimize("unroll-loops")`: Otimiza os loops mas pode levar a *cache misses*

E também os pragmas de *target*, que são usados para otimizar o código para um certo processador:

- `#pragma GCC target("avx2")`: Otimiza instruções para processadores com suporte a AVX2
- `#pragma GCC target("sse4")`: Parecido com o de cima, mas mais antigo

- `#pragma GCC target("popcnt")`: Otimiza o *popcount* em processadores que suportam
- `#pragma GCC target("lzcnt")`: Otimiza o *leading zero count* em processadores que suportam
- `#pragma GCC target("bmi")`: Otimiza instruções de *bit manipulation* em processadores que suportam
- `#pragma GCC target("bmi2")`: Mesmo que o de cima, mas mais recente

Em geral, esses pragmas são usados para otimizar o código em competições, mas é importante usá-los com certa sabedoria, em alguns casos eles podem piorar o desempenho do código.

Uma opção relativamente segura de se usar é a seguinte:

```
#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
```

22.1.13 Constantes em C++

Constante	Nome em C++	Valor
π	M_PI	3.141592...
$\pi/2$	M_PI_2	1.570796...
$\pi/4$	M_PI_4	0.785398...
$1/\pi$	M_1_PI	0.318309...
$2/\pi$	M_2_PI	0.636619...
$2/\sqrt{\pi}$	M_2_SQRTPI	1.128379...
$\sqrt{2}$	M_SQRT2	1.414213...
$1/\sqrt{2}$	M_SQRT1_2	0.707106...
e	M_E	2.718281...
$\log_2 e$	M_LOG2E	1.442695...
$\log_{10} e$	M_LOG10E	0.434294...
$\ln 2$	M_LN2	0.693147...
$\ln 10$	M_LN10	2.302585...

22.2 Matemática

22.2.1 Definições

Algumas definições e termos importantes:

Funções

- **Comutativa:** Uma função $f(x, y)$ é comutativa se $f(x, y) = f(y, x)$.
- **Associativa:** Uma função $f(x, y)$ é associativa se $f(x, f(y, z)) = f(f(x, y), z)$.
- **Idempotente:** Uma função $f(x, y)$ é idempotente se $f(x, x) = x$.

22.2.2 Números primos

Números primos são muito úteis para funções de hashing (dentre outras coisas). Aqui vão vários números primos interessantes:

Primos com truncamento à esquerda Números primos tais que qualquer sufixo deles é um número primo:

$$33{,}333{,}31$$
$$357{,}686{,}312{,}646{,}216{,}567{,}629{,}137$$

Primos gêmeos (Twin Primes) Pares de primos da forma $(p, p + 2)$ (aqui tem só alguns pares aleatórios, existem muitos outros).

Números primos de Mersenne São os números primos da forma $2^m - 1$, onde m é um número inteiro positivo.

22.2.3 Operadores lineares

Rotação no sentido anti-horário por θ

$$\begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

Reflexão em relação à reta $y = mx$

$$\frac{1}{m^2+1}\begin{bmatrix} 1-m^2 & 2m \\ 2m & m^2-1 \end{bmatrix}$$

Inversa de uma matriz 2×2 A

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{\det(A)}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

Cisalhamento horizontal por K

$$\begin{bmatrix} 1 & K \\ 0 & 1 \end{bmatrix}$$

Cisalhamento vertical por K

$$\begin{bmatrix} 1 & 0 \\ K & 1 \end{bmatrix}$$

Mudança de base $\vec{a}_\beta$ são as coordenadas do vetor $\vec{a}$ na base β.
$\vec{a}$ são as coordenadas do vetor $\vec{a}$ na base canônica.
$\vec{b}_1$ e $\vec{b}_2$ são os vetores de base para β.
C é uma matriz que muda da base β para a base canônica.

$$C\vec{a}_\beta = \vec{a}$$
$$C^{-1}\vec{a} = \vec{a}_\beta$$
$$C = \begin{bmatrix} b_{1x} & b_{2x} \\ b_{1y} & b_{2y} \end{bmatrix}$$

Propriedades das operações de matriz

$$(AB)^{-1} = B^{-1}A^{-1}$$
$$(AB)^T = B^T A^T$$
$$(A^{-1})^T = (A^T)^{-1}$$
$$(A+B)^T = A^T + B^T$$
$$\det(A) = \det(A^T)$$
$$\det(AB) = \det(A)\det(B)$$

Seja A uma matriz $N \times N$:

$$\det(kA) = k^N \det(A)$$

22.2.4 Sequências numéricas

Sequência de Fibonacci Primeiros termos: 1, 1, 2, 3, 5, 8, 13, 21, 34, ...
Definição:

$$F_0 = F_1 = 1$$
$$F_n = F_{n-1} + F_{n-2}$$

Matriz de recorrência:

$$\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} F_{n-2} \\ F_{n-1} \end{bmatrix} = \begin{bmatrix} F_{n-1} \\ F_n \end{bmatrix}$$

Sequência de Catalan Primeiros termos: 1, 1, 2, 5, 14, 42, 132, 429, 1430, ...
Definição:

$$C_0 = C_1 = 1$$
$$C_n = \sum_{i=0}^{n-1} C_i \cdot C_{n-1-i}$$

Definição analítica:

$$C_n = \frac{1}{n+1}\binom{2n}{n}$$

Propriedades úteis:

- C_n é o número de árvores binárias com $n+1$ folhas.
- C_n é o número de sequências de parênteses bem formadas com n pares de parênteses.

22.2.5 Análise combinatória

Fatorial O fatorial de um número n é o produto de todos os inteiros positivos menores ou iguais a n.
O fatorial conta o número de permutações de n elementos.

$$n! = n \cdot (n-1)!$$

Em particular, $0! = 1$.

Combinação Conta o número de maneiras de escolher k elementos de um conjunto de n elementos.

$$\binom{n}{k} = \frac{n!}{k! \cdot (n-k)!}$$

Arranjo Conta o número de maneiras de escolher k elementos de um conjunto de n elementos, onde a ordem importa.

$$P(n,k) = \frac{n!}{(n-k)!}$$

Estrelas e barras Conta o número de maneiras de distribuir n elementos idênticos em k recipientes distintos.

$$\binom{n+k-1}{k-1}$$

Princípio da inclusão-exclusão O princípio da inclusão-exclusão é uma técnica para contar o número de elementos em uma união de conjuntos.

$$\left|\bigcup_{i=1}^{n} A_i\right| = \sum_{k=1}^{n}(-1)^{k+1} \sum_{1 \le i_1 < \cdots < i_k \le n} |A_{i_1} \cap \cdots \cap A_{i_k}|$$

Princípio da casa dos pombos Se n pombos são colocados em m casas, então pelo menos uma casa terá $\lceil \frac{n}{m} \rceil$ pombos ou mais.

22.2.6 Teoria dos números

Pequeno teorema de Fermat Se p é um número primo e a é um inteiro não divisível por p, então $a^{p-1} \equiv 1 \pmod{p}$.

Teorema de Euler Se m e a são inteiros positivos coprimos, então $a^{\phi(m)} \equiv 1 \pmod{m}$, onde $\phi(m)$ é a função totiente de Euler.

Aritmética modular Quando estamos trabalhando com aritmética módulo um número p, todos os valores existentes estão entre $[0, p-1]$.
Algumas propriedades e equivalências úteis para usar aritmética modular em código são:

- $(a+b) \bmod p \equiv ((a \bmod p) + (b \bmod p)) \bmod p$
- $(a-b) \bmod p \equiv ((a \bmod p) - (b \bmod p)) \bmod p$
  - Note que o resultado pode ser negativo, nesse caso é necessário adicionar p ao resultado. De forma geral, geralmente fazemos $(a-b+p) \bmod p$ (assumindo que a e b já estão no intervalo $[0, p-1]$).
- $(a \cdot b) \bmod p \equiv ((a \bmod p) \cdot (b \bmod p)) \bmod p$
- $a^b \bmod p \equiv ((a \bmod p)^b) \bmod p$
- $a^b \bmod p \equiv ((a \bmod p)^{(b \bmod \phi(p))}) \bmod p$ se a e p são coprimos

---

22.3 Grafos

22.3.1 Definições Iniciais

- **Grafo:** Um grafo é um conjunto de vértices e um conjunto de arestas que conectam os vértices. Formalmente, $G(V, E)$ onde V é o conjunto de vértices e E o conjunto de arestas.
- **Caminho:** Um caminho P de x_0 para x_k é um subgrafo não vazio da forma $V = \{x_0, x_1, \ldots, x_k\}$ e $E = \{\{x_0, x_1\}, \{x_1, x_2\}, \ldots, \{x_{k-1}, x_k\}\}$ em que todos os x_i são distintos.
- **Comprimento de um caminho:** Definido como o número de arestas em E.
- **Vizinho:** Para cada vértice, os vértices que podemos atingir a partir das suas arestas são conhecidos como vizinhos.
- **Grau:** O número de vizinhos de um vértice.

22.3.2 Ciclos

- **Ciclo:** Sendo P um caminho de x_0 para x_k de comprimento ≥ 2, um ciclo é definido como $P + \{x_k, x_0\}$, ou seja, adiciona uma aresta fechando o ciclo.
- **Ciclo Simples:** É um tipo especial de ciclo onde:
  - Nenhum vértice (além do inicial e final) se repete.
  - Nenhuma aresta se repete.
  - Tem ao menos 3 vértices distintos (em grafos simples), ou seja, comprimento ≥ 3.

22.3.3 Conectividade

- **Grafo Conexo:** Um grafo é conexo se existe um caminho entre todos os pares de vértices. Caso contrário, o grafo é desconexo.
- **Componente Conexo:** Um subgrafo conexo maximal de um grafo G.
- **Grafo Bipartido:** Um grafo é bipartido se é possível dividir os vértices em dois conjuntos disjuntos de forma que todas as arestas conectem um vértice de um conjunto com um vértice do outro conjunto, ou seja, não existem arestas que conectem vértices do mesmo conjunto.

22.3.4 Árvores

- **Árvore:** Um grafo acíclico (sem ciclo) e conexo.
- **Folhas:** Vértices de grau 1.
- **Raiz:** Uma árvore pode ser enraizada por um vértice qualquer (introduz noção de pai e filhos).
- **Árvore Geradora Mínima (AGM):** Uma árvore que conecta todos os vértices de um grafo e possui o menor custo possível, também conhecido como *Minimum Spanning Tree (MST)*.

Afirmações equivalentes para uma árvore G:

- G é uma árvore $\iff$ G é um grafo conexo e $|E| = |V| - 1$.
- G é uma árvore $\iff$ Para todo $v, w \in G$ existe EXATAMENTE UM caminho de v para w.
- G é uma árvore $\iff$ G é conexo e a remoção de qualquer aresta o torna desconexo.
- G é uma árvore $\iff$ G não contém ciclos, mas adicionar qualquer aresta a G cria um ciclo.

Diâmetro Consiste no maior caminho de uma árvore.
Algoritmo:

1. Roda BFS a partir de um vértice qualquer u para encontrar o vértice v mais distante.
2. Roda BFS a partir do vértice v para encontrar o vértice w mais distante. O caminho de v a w é o diâmetro.

---

23 Utils

23.1 Makefile (a879a5)

```
CXX = g++
CXXFLAGS = -fsanitize=address,undefined -fno-omit-frame-pointer -g -Wall -
    Wshadow -std=c++20 -Wno-unused-result -Wno-sign-compare -Wno-char-
    subscripts
```

23.2 custom hash (13b6af)

```
#include <bits/stdc++.h>
using namespace std;
// importas para o hash_table
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
typedef long long ll;
// o unordered_map e facil de hackear:
// as operacoes ficam O(N), podemos melhorar
// isso com uma funcao de hash melhor
struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    // posso alterar o int para ll e etc...
    size_t operator()(int x) const {
        static const int FIXED_RANDOM =
            chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + FIXED_RANDOM);
    }
    size_t operator()(pair<int, int> x) const {
        static const uint64_t FIXED_RANDOM =
            chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x.first + FIXED_RANDOM)
            ^ splitmix64(x.second + FIXED_RANDOM);
    }
};
// criando algumas estruturas "seguras"
template <typename K, typename V>
using umap = unordered_map<K, V, custom_hash>;
// o hash table e mais performatico em media
template <typename K, typename V>
using ht = gp_hash_table<K, V, custom_hash>;
// https://codeforces.com/blog/entry/62393
```

23.3 hash (d78ff6)

```
# Para usar (hash das linhas [l1, l2]):
# bash hash.sh arquivo.cpp l1 l2
sed -n $2',$3' p' $1 | sed '/^#w/d' | cpp -dD -P -fpreprocessed | tr -d '[:
    space:]' | md5sum | cut -c-6
```

23.4 mistakes (6dae29)

```
(dicas da lib do collares - traduzido)

# Dicas Gerais de Estrategia

- Todos devem ter lido todos os problemas ate o final da segunda hora
    de competicao.
- Na ultima hora, apenas uma questao deve ser tentada por vez.
- Sempre use *vectors* em vez de *arrays* no caso unidimensional.
- Use *vectors* em vez de *arrays* no caso bidimensional se a dimensao
    mais externa (numero de linhas) nao for muito grande. Isso
    permite uma melhor depuracao (*debugging*).

# Ao Pensar (Antes de Programar)

1. Seja organizado(a)!
2. Nao toque no computador a menos que a solucao esteja pronta,
    incluindo os detalhes de implementacao: evite pensar demais em
    frente ao computador.
O tempo gasto no papel detalhando uma solucao e tempo bem gasto.

3. Um corolario do ponto acima:
Nao toque no computador se tiver duvidas sobre a sua ideia.

# Caso Voce Nao Tenha Ideias de Solucao

1. (Inclua os truques de Polya aqui) -> TODO: Gabrielzinho faca essa
    parte

# Em Caso de *Wrong Answer* (Resposta Errada)

1. Certifique-se de que o algoritmo esta correto (ou seja, esboce uma
    prova de corretude) o mais rapido possivel. Leve o tempo que
    precisar e verifique com cuidado.
Se voce tem certeza de que a ideia esta correta, nao duvide dela ao
    procurar por *bugs*, mesmo que nao encontre nenhum erro de
    implementacao em lugar nenhum.

2. Se o codigo usa *vectors*, adicione `#define _GLIBCXX_DEBUG` bem no
    inicio do codigo-fonte e submeta novamente. Um erro de execucao
    (*runtime error*) geralmente significa acesso fora dos limites do
    *array* (*out-of-bounds*) ou outro uso incorreto da STL.

3. Depure no papel e nao volte para o computador a cada *bug* que
    encontrar: verifique a solucao inteira pelo menos mais uma vez
    apos encontrar cada novo *bug*.

4. Pense em casos de teste capciosos.

5. Leia o problema novamente. Para cada restricao encontrada,
    verifique o codigo-fonte impresso.

6. Se voce nao conseguir encontrar nenhum *bug* em cinco minutos, va
    ao banheiro.
    6.1. Se ainda assim nao conseguir encontrar o *bug*, va para outro
        problema e volte para a solucao errada mais tarde.
    6.2. Depurar um unico programa por muito tempo leva a encontrar
        muitos falsos *bugs* e faz com que seja facil deixar passar
        erros simples.

7. Use o `assert(condicao)` para verificar:
    - Se um algoritmo construtivo seu bate com as condicaos que o
        problema pede.
    - Para testar possiveis cenarios de casos de testes. Gaste um WA/
        RE para verificar se os casos de teste sao do jeito que voce
        imagina.

# Em Caso de *Runtime Error* (Erro de Execucao)

1. Verifique possivel overflow nas operacoes.
2. Verifique divisoes e operacoes de modulo.
3. Verifique os indices dos *arrays* (tanto nas declaracoes quanto nos
    acessos).
4. Verifique se ha recursoes infinitas ou *loops* `while` infinitos.
```

23.5 random (5b3cc9)

```
#include <bits/stdc++.h>
using namespace std;
#define all(x) (x).begin(), (x).end()
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
signed main() {
    vector<int> a(5);
    iota(all(a), 0);
    // ordena aleatoriamente
    shuffle(all(a), rng);
    // gera valor aleatorio de [0, x)
    cout << rng() % 10 << endl; // [0, 10)
    // valor aleatorio entre [a, b]
    int r = uniform_int_distribution<int>(3, 99)(rng);
    cout << r << endl;
}
```

23.6 template (c1a851)

```
#include <bits/stdc++.h>
using namespace std;
#define ff first
#define ss second
#define pb push_back
#define int long long
#define endl '\n' //<< flush
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define dbg(x) cerr << #x << " = " << x << endl
#define pdbg(x) cerr << #x << " = " << x.ff << "," << x.ss << endl
#define uniq(v) sort(all(v)); v.erase(unique(all(v)), v.end())
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
typedef long long ll;
typedef long double ld;
typedef pair<int, int> pi;
// const ld EPS = 1e-9;
// const int MOD = 1e9 + 7; // 998244353;
// const int INF = 0x3f3f3f3f;
// const ll LINF = 0x3f3f3f3f3f3f3f3fll;
// const int MAXN = 2e5+5;
signed main() {
    ios_base::sync_with_stdio(0);cin.tie(0);
    cout.precision(10); cout.setf(ios::fixed);
}
```

23.7 vimrc (54a8c1)

```
set ts=4 sw=4 sts=4 expandtab mouse=a nu ai si undofile
```